



EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

DEPT. OF PROGRAMMING LANGUAGES AND COMPILERS

AI Cookbook with Ingredient-Based Search

Supervisor:

Gregory Reynolds Morse

Lecturer

Author:

Ulkar Chobanova

Computer Science BSc

Budapest, 2026

Thesis Topic Registration Form

Student's Data:

Student's Name: Chobanova Ulkar

Student's Neptun code: WQ1C71

Educational Information:

Training programme: Computer Science BSc

I have an internal supervisor

Internal Supervisor's Name: *Gregory Reynolds Morse*

Supervisor's Home Institution: *Department of Programming Languages and Compilers*

Address of Supervisor's Home Institution: *1117, Budapest, Pázmány Péter sétány 1/C.*

Supervisor's Position and Degree: *Lecturer and MSc in Computer Science*

Thesis Title: AI Cookbook with Ingredient-Based Search

Topic of the Thesis:

(Upon consulting with your supervisor, give a 150-300-word-long synopsis of your planned thesis.)

Problem Statement:

People often face challenges in deciding what to cook with available ingredients while considering dietary preferences or allergies. This project proposes an AI-powered cookbook that generates personalized recipes from a list of ingredients. The system will employ OCR to extract and structure nutritional information from food labels and provide structured recipe outputs with basic nutritional insights, enhancing usability and supporting informed cooking decisions.

Proposed Complex Features:

1. OCR-Based Nutrition Label Processing

Users can upload images of food labels. Text is extracted using an OCR framework (e.g., Google Vision API) and refined into structured JSON format by an LLM for consistent interpretation of nutritional values.

2. Structured Recipe Generation and Personalization

The application generates clear, structured recipes with sections for ingredients, preparation steps, and nutritional details using retrieval-augmented generation (RAG). Recipe suggestions are further personalized based on user preferences such as dietary restrictions or allergens.

3. Data Persistence and Backend Integration

The system will include a lightweight backend to handle recipe generation requests and manage user data. A database will ensure data persistence, allowing storage of user preferences, generated recipes, and interaction history for improved continuity and usability.

Budapest, 2025. 10. 13.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Gregory Reynolds Morse, for his guidance, feedback, and continuous support throughout the development of this thesis. I am also deeply grateful to my family for their patience, encouragement, and constant support during my studies and during the preparation of this work.

Contents

Acknowledgements	2
1 Introduction	4
1.1 Motivation and Problem Statement	4
1.2 Goals and Objectives	5
1.3 Scope and Constraints	5
1.4 Thesis Roadmap	6
2 User documentation	7
2.1 Installation, setup, and configuration guide	8
2.1.1 System requirements	8
2.1.2 Getting the source code and installing dependencies	10
2.1.3 Environment configuration	12
2.1.4 Verifying the installation	14
2.1.5 Initializing the database and local retrieval data	14
2.1.6 Starting the backend and frontend	16
2.1.7 Post-startup verification	16
2.2 User manual	17
2.2.1 Quick-start walkthrough from launch to first meaningful action	17
2.2.2 Feature-by-feature guide with annotated key screens	18
2.2.3 Step-by-step workflows for the main features	25
2.2.4 Accessibility, localisation, and access notes	27
2.3 Troubleshooting, limitations, and frequently asked questions	28
2.3.1 Known issues and recommended workarounds	28
2.3.2 System and functional limitations	29
2.3.3 Frequently asked questions from realistic usage scenarios	31
3 Developer Documentation	33

3.1	Design	34
3.1.1	Design Drivers and Requirements	34
3.1.2	Architecture and System Design	43
3.1.3	Data Design	55
3.1.4	UI/UX Design	62
3.1.5	Design alternatives considered	66
3.1.6	Section Summary	67
3.2	Implementation	68
3.2.1	Development environment and technology stack	68
3.2.2	Repository structure and code organization	68
3.2.3	Core application logic and complex features	71
3.2.4	Frontend implementation	81
3.2.5	Persistence implementation	85
3.2.6	Known implementation limitations	92
3.2.7	Section Summary	95
3.3	Testing	96
3.3.1	Testing Strategy and Methodology	96
3.3.2	Unit and Functional Testing	97
3.3.3	UI Testing	100
3.3.4	Integration and End-to-End Testing	102
3.3.5	Advanced Reliability Testing	107
3.3.6	Manual Scenario-Based Testing	108
3.3.7	Advanced AI-Related Evaluation	111
3.3.8	Threats to Validity	115
3.3.9	Section Summary	115
4	Conclusion	117
4.1	Summary of achieved goals versus original proposal commitments . .	117
4.2	Lessons learned and challenges encountered	118
4.3	Current limitations	119
4.4	Proposed future work and development directions	119
	Bibliography	121
	List of Figures	125

CONTENTS

List of Tables	127
List of Codes	128

Chapter 1

Introduction

1.1 Motivation and Problem Statement

Deciding what to cook from whatever ingredients happen to be available is a common practical problem. Users typically know what is in their kitchen, but translating that into a concrete, safe, and personalized meal requires browsing multiple recipe websites, manually filtering results, and adapting instructions to fit dietary restrictions or allergies. This process is repetitive and effortful, yet users still often lack a simple tool that accepts a list of ingredients and returns a structured recipe adapted to personal constraints.

A related inconvenience arises with packaged food products. Nutritional information is printed on product labels in a format that is easy to read in the moment but difficult to store, compare, or reuse digitally. Users who want to track or inspect the nutritional content of packaged goods have no practical way to capture that data without manually transcribing it, which is both time-consuming and error-prone.

This thesis addresses both problems through a single integrated application. The core feature allows users to submit a custom ingredient list along with optional dietary preferences and allergy constraints, and receive a structured recipe generated specifically from that input. A secondary feature accepts an image of a food product label, extracts the printed nutritional text automatically, and allows users to store the result in a structured digital form. Together, these features form a coherent tool for ingredient-aware cooking decisions and nutritional data capture within a single personalized cooking assistant.

1.2 Goals and Objectives

The main goal of the thesis is to deliver a working web application that generates personalized recipes from user-provided ingredients and supports OCR-based nutrition label processing. The project commits to four concrete and verifiable objectives.

The first objective is to implement an ingredient-based recipe generation workflow. The system accepts a list of ingredients and optional dietary preferences and allergies, validates the request, and returns a structured recipe containing a title, ingredient list, preparation steps, nutritional estimate, and system warnings. To improve the relevance of generated results, the workflow uses retrieval-augmented generation: before producing a recipe, the system retrieves relevant entries from an indexed recipe corpus and uses them as contextual grounding for the language model.

The second objective is to implement user-specific personalization and persistence. Authenticated users can create accounts, save dietary preferences and allergies, store generated recipes, and access both saved recipes and their full generation history. This ensures the application supports repeated and personalized use rather than one-time generation.

The third objective is to implement OCR-based nutrition label extraction. The system accepts uploaded label images, validates and preprocesses them, extracts text through an OCR provider, transforms the raw output into structured nutritional fields, and allows authenticated users to save their labels and extracted data.

The fourth objective is to implement and document the backend and data model that support these workflows. This covers the system architecture, request handling, authentication, persistent storage, media handling, and the service boundaries required to coordinate recipe generation, OCR processing, and history management.

1.3 Scope and Constraints

The scope of the thesis covers the implemented *AI Cookbook with Ingredient-Based Search* application and all features available in the final system: ingredient-based recipe generation, preference- and allergy-aware personalization, OCR-based nutrition label extraction, user authentication, and storage of recipe and label his-

tory. It also covers the technical documentation of the system, including the application architecture, main request flows, persistent data model, and testing material.

The thesis does not aim to provide a general introduction to artificial intelligence, nutrition science, or OCR theory beyond what is necessary to explain the implemented solution.

Several constraints apply. Recipe generation, nutritional structuring, and image generation depend on external AI services, meaning that response quality and availability are not fully controlled by the local application. OCR accuracy depends on the quality and readability of uploaded images. Dietary preferences, allergy-related handling, and nutritional values displayed by the system, whether extracted from labels or estimated for generated recipes, are intended as application output for user guidance and do not constitute certified dietary or medical advice.

The project also does not aim to provide production-grade infrastructure, clinical nutritional analysis, or guaranteed factual correctness for every AI-generated or OCR-derived value. Its purpose is to demonstrate a working and well-documented implementation of the designed workflows within the scope of a bachelor thesis.

The project is also constrained by its scope as a bachelor thesis. The system was designed to be functionally complete and documentable within limited development time. The thesis therefore focuses on the implemented features and their software engineering realization rather than production-scale concerns such as distributed deployment or expert-level nutritional analytics. Additional feature expansion and broader implementation improvements are therefore treated as future work.

1.4 Thesis Roadmap

Chapter 2 presents the user documentation and describes how the application is configured, launched, and used from the perspective of its intended users.

Chapter 3 presents the developer documentation. It is divided into the design, implementation, and testing sections, covering the software requirements, system architecture, data model, implementation details, and validation results of the application.

Chapter 4 concludes the thesis by summarizing the achieved results, discussing the limitations of the current implementation, and outlining directions for future improvement.

Chapter 2

User documentation

This chapter documents the user-side aspects of the implemented *AI Cookbook with Ingredient-Based Search* application. It describes how the system is prepared, configured, launched, and used in a local thesis and development environment. It first presents the setup and configuration steps required before the application can be used, then documents the main workflows from the user’s perspective. Because the application integrates a FastAPI backend, a React frontend, a local relational database, retrieval-based recipe generation, and OCR-based nutrition label processing, proper initial setup is required to ensure reliable execution of all workflows.

The chapter is organized into three sections. Section 2.1 outlines the technical prerequisites and configuration steps, including hardware and software requirements, dependency installation, environment variables, and initial verification. Section 2.2 provides a user manual describing the main workflows, such as recipe generation, personalization features, recipe storage, and OCR-based food label extraction. Section 2.3 discusses known issues, system limitations, and frequently asked questions based on practical usage.

2.1 Installation, setup, and configuration guide

This section describes the requirements and steps necessary to run the application in a local development environment. It covers the hardware and software prerequisites, required external services, and the configuration steps needed before the backend, frontend, database, vector store, and AI-supported features can be used.

2.1.1 System requirements

This subsection defines the conditions that should be satisfied before the application is installed and executed in a local development environment. Because the system combines a Python-based backend, a React frontend, local persistent storage, and multiple AI-supported services, successful operation depends on both local development tools and external API access. The requirements are therefore grouped into three categories: hardware requirements, software requirements, and external service requirements.

Hardware requirements

- **Operating system:** Windows 10 or newer, a recent Linux distribution, or a recent macOS version.
- **Processor:** Any modern 64-bit multi-core processor is sufficient for development and local execution.
- **Memory (RAM):** At least 8 GB is required, while 16 GB is recommended for smoother use when the backend, frontend, browser, and indexing-related tasks are active at the same time.
- **Storage:** At least 4 GB of free disk space is recommended. This covers the project source code, Python and Node.js dependencies, the local SQLite database, uploaded OCR images, generated recipe thumbnails, and the persisted Chroma vector store.
- **Internet connection:** Required during both setup and normal use. Dependency installation requires network access, and the AI-supported work-

flows depend on external APIs for recipe generation, embedding generation, OCR processing, and nutrition-data structuring.

Software requirements

- **Python 3.11 or newer.** Required for the FastAPI backend, virtual environment management, dependency installation, Alembic database migrations, recipe indexing scripts, and backend testing.

Download: <https://www.python.org/downloads/>.

- **Node.js and npm.** Required for installing frontend dependencies and running the React application through the Vite development server. A recent LTS version is recommended for local development.

Download: <https://nodejs.org/en/download>.

- **Git.** Recommended for cloning, updating, and managing the repository in a reproducible way.

Installation guide:

<https://git-scm.com/book/en/v2/Getting-Started-Installing-Git>.

- **Modern web browser.** Required to access the frontend locally. Google Chrome, Mozilla Firefox, and Microsoft Edge are all suitable for development and normal use.

For platform-specific Python installation, interpreter usage, and environment-management details, the official *Python Setup and Usage* guide can be consulted [1].

For platform-specific Node.js and npm installation details, the official Node.js installation guide can be consulted [2].

External service requirements

In addition to the local software stack, the AI-supported features require access to two external services. The application can start without these credentials, but the affected AI-supported workflows will not function correctly until they are configured.

OpenAI API access is required for recipe generation, embedding generation for the retrieval pipeline, structuring OCR-extracted nutrition data into JSON, producing nutritional estimates for generated recipes, and generating recipe thumbnail images. For full local deployment, the application expects a valid OpenAI API key to be provided through environment configuration by the maintainer or developer operating the system. Ordinary end users do not enter this value through the application interface. Further information is available at:

<https://help.openai.com/en/articles/4936850-where-do-i-find-my-openai-api-key>

Google Cloud Vision credentials are required for OCR-based nutrition label extraction. The application expects credentials to be provided through the `GOOGLE_APPLICATION_CREDENTIALS` environment variable, following the standard Google Cloud authentication flow. As with the OpenAI key, these credentials are supplied during environment setup by the maintainer or developer rather than by ordinary end users. The authentication setup is described at:

<https://cloud.google.com/vision/docs/authentication>.

2.1.2 Getting the source code and installing dependencies

This subsection explains how the project is obtained and how the development dependencies are prepared for first use. The repository contains a Python backend, a React frontend, migration files, testing material, and thesis-related assets, so the complete project root should be available locally before configuration begins.

The source code is hosted in a Git repository. Cloning the repository is the preferred approach because it preserves the full project structure and allows future updates to be pulled in a consistent way. If the project is already available locally as a thesis submission archive or extracted folder, the same setup steps can be followed from that project root without cloning again. The repository can be obtained with the following commands:

```
1 git clone https://github.com/ulkarchbnv/ai-cookbook-thesis.git
2 cd ai-cookbook-thesis
```

Code 2.1: Cloning the project repository

After cloning, the project root should contain the main application directories used during setup and development, most importantly `backend`, `frontend`, and `alembic`. The `backend` directory contains the FastAPI application and its service

logic, the **frontend** directory contains the React user interface, and the **alembic** directory contains the database migration files used to initialize and update the local schema.

After the repository has been cloned, the backend environment should be prepared first. The application uses Python for the backend, so it is recommended to create a dedicated virtual environment in the project root. This avoids interference with globally installed Python packages and keeps the local setup reproducible.

```
1 python -m venv .venv
```

Code 2.2: Creating the Python virtual environment

On Windows PowerShell, the virtual environment can be activated as follows:

```
1 .\venv\Scripts\Activate.ps1
```

Code 2.3: Activating the virtual environment in PowerShell

On Linux or macOS shells, the typical activation command is:

```
1 source .venv/bin/activate
```

Code 2.4: Activating the virtual environment on Linux or macOS

Once the virtual environment is active, the backend requirements can be installed from the dependency list stored in **backend/requirements.txt**. This step installs the web framework, database and migration libraries, AI-service client libraries, OCR-related packages, retrieval-pipeline dependencies, and project support tools such as the backend testing framework.

```
1 python -m pip install --upgrade pip
2 python -m pip install -r backend/requirements.txt
```

Code 2.5: Installing backend dependencies

At this point, the backend runtime, database libraries, migration tools, OCR-related dependencies, AI client libraries, and testing packages are available inside the local virtual environment.

The frontend is maintained separately in the **frontend** directory and must also be initialized before the application can be started. Its dependencies are installed with **npm**, which downloads the packages required for the React user interface, routing, the Vite development server, and frontend tests.

```
1 cd frontend
2 npm install
3 cd ..
```

Code 2.6: Installing frontend dependencies

After both installation steps have finished, the local machine contains all project files and package dependencies needed for the next stage of setup. The environment is then ready for configuration of the variables used by the backend and frontend services, which is described in the following subsection.

2.1.3 Environment configuration

After the repository and package dependencies have been prepared, the next step is to define the environment variables required by the application. The backend loads its configuration from a `.env` file located in the project root. The frontend reads its API base URL through Vite environment handling, and in the current implementation it also falls back to `http://127.0.0.1:8000` when `VITE_API_BASE_URL` is not explicitly provided.

The project already provides a template file named `.env.example`. This file should be copied to a new file named `.env`, and the placeholder values should then be replaced with valid local configuration values.

```
1 copy .env.example .env
```

Code 2.7: Creating the local environment file

On Linux or macOS, the equivalent command is typically:

```
1 cp .env.example .env
```

Code 2.8: Creating the local environment file on Linux or macOS

The most important variables for local execution are the following:

- `OPENAI_API_KEY` specifies the API key used by the recipe-generation, embedding, nutrition-structuring, and image generation features.
- `GOOGLE_APPLICATION_CREDENTIALS` points to the local path of the Google Cloud service-account JSON file used by the OCR provider.

- `DATABASE_URL` defines the relational database connection. If this value is not overridden, the backend defaults to a local SQLite database file named `ai_cookbook.db` in the project root.
- `SECRET_KEY` is required for JWT creation and validation and must not be left empty.
- `FRONTEND_ORIGIN` defines the allowed frontend origin for backend CORS handling. In the default local setup, this should match the Vite development server address.

In addition to these core values, the configuration file also contains optional settings for model names, retrieval parameters, upload limits, media directories, rate-limiting rules, and vector store persistence paths. Many of these defaults are already suitable for a standard local development setup, so they do not need to be changed unless the deployment environment or experimental requirements differ from the default configuration. However, required values such as `SECRET_KEY`, `OPENAI_API_KEY`, and, for OCR usage, `GOOGLE_APPLICATION_CREDENTIALS`, must still be configured explicitly.

If the frontend should target a backend address other than the default `http://127.0.0.1:8000`, the `VITE_API_BASE_URL` variable can be provided through the frontend's Vite environment configuration.

A typical local `.env` file follows the structure shown in Code 2.9. The values below are examples only and should be replaced with valid credentials and paths. The example is intentionally limited to the most important backend settings for a standard local setup.

```
1 OPENAI_API_KEY=your_openai_key
2 OCR_PROVIDER=google_vision
3 GOOGLE_APPLICATION_CREDENTIALS=C:\path\to\your-google-service-
  account.json
4 DATABASE_URL=sqlite:///./ai_cookbook.db
5 SECRET_KEY=your_secret_key
6 ALGORITHM=HS256
7 FRONTEND_ORIGIN=http://localhost:5173
```

Code 2.9: Example local environment configuration

When this file has been created and updated, the backend can load its runtime settings correctly, and the frontend can target the intended local API endpoint.

2.1.4 Verifying the installation

Before launching the application, it is useful to verify that the main tools and local files are available as expected. This helps detect common setup mistakes early, such as a missing virtual environment, an unavailable Node.js installation, or an incorrectly named configuration file.

The first check is to confirm that the required command-line tools are accessible and return valid version information.

```
1 python --version
2 node --version
3 npm --version
4 git --version
```

Code 2.10: Verifying the local toolchain

At this stage, Python should report version 3.11 or newer, while Node.js and npm should also return installed versions without error. Git is not required for runtime execution after cloning, but it should still be available if the repository will be updated or maintained locally.

The second check is to confirm that the expected local setup artifacts exist in the project root. In a correctly prepared environment, the repository should already contain the application source directories, the `.venv` virtual environment directory, and the `.env` configuration file.

```
1 Get-ChildItem .venv, backend, frontend, alembic, .env
```

Code 2.11: Verifying key local files and directories

If these files and directories are present, the local installation can be considered structurally complete. The next step is to initialize the database and other local data resources required by the application.

2.1.5 Initializing the database and local retrieval data

After the installation has been verified, the local data layer should be initialized. In the default configuration, the backend uses a SQLite database defined through

`DATABASE_URL`, and its schema is managed through Alembic migration files located in the `alembic` directory. Running the migration step ensures that the required tables and indexes are created before the application is started.

The migration command should be executed from the project root while the Python virtual environment is active:

```
1 alembic upgrade head
```

Code 2.12: Applying database migrations

When this command completes successfully, the local database schema contains the tables required for users, generated recipes, and saved OCR extraction history. In the default SQLite-based setup, the file `ai_cookbook.db` is created automatically in the project root if it does not already exist.

In addition to the relational database, the recipe-generation workflow also depends on local retrieval data. The project already includes a prepared recipe subset in `backend/data/recipes_subset.json`, which serves as the source corpus for the retrieval-augmented generation pipeline. This subset was prepared from the Food.com dataset distributed through Kaggle. To make this corpus searchable, it must be embedded and indexed into the local Chroma vector store.

```
1 python -m backend.scripts.index_recipes
```

Code 2.13: Indexing the recipe corpus for retrieval

This step reads the recipe subset, generates embeddings for the recipe documents, and stores them in the Chroma persistence directory specified by `CHROMA_PERSIST_DIRECTORY`. It should be executed at least once before recipe generation is tested on a new local setup.

The project also includes a preparation script that can regenerate the cleaned recipe subset when the raw dataset is available locally. For normal local setup, however, this preprocessing step is not required because the prepared subset used by the application is already included in the repository.

Once the database migrations and vector-store indexing have both been completed, the system is ready to launch the backend and frontend services.

2.1.6 Starting the backend and frontend

After the local data layer has been initialized, the application can be started. The backend and frontend run as separate development services, so they are typically launched in two terminal windows.

The backend should be started first from the project root while the Python virtual environment is active:

```
1 uvicorn backend.main:app --reload
```

Code 2.14: Starting the FastAPI backend

In the default local configuration, the backend starts on `http://127.0.0.1:8000`. If the startup is successful, the terminal displays the Uvicorn server message and begins watching the project files for code changes.

If a different host or port is needed, or if the backend must later be deployed behind HTTPS with an SSL certificate, these operational details should be taken from the official FastAPI deployment guide rather than duplicated here [3].

The frontend is started separately from the `frontend` directory:

```
1 cd frontend
2 npm run dev
```

Code 2.15: Starting the React frontend

In the default setup, the Vite development server starts on `http://localhost:5173`. The frontend sends its API requests to the backend URL defined by `VITE_API_BASE_URL`, which should therefore match the backend address used during startup.

When both services are running, the application can be opened in a web browser by visiting the frontend address. The backend exposes the API used by the user interface, while the frontend provides access to the pages for recipe generation, nutrition label extraction, saved items, history, and account management.

2.1.7 Post-startup verification

After both services have been started, a short smoke test should be performed before the full user workflows are exercised. The purpose of this check is not detailed feature validation, but quick confirmation that the local setup is operational.

The backend can be checked by opening `http://127.0.0.1:8000/` in a browser. In a successful setup, the service returns a small JSON response containing the message "AI Cookbook backend is running".

The frontend can be checked by opening `http://localhost:5173`. In a successful setup, the home page of the application is displayed and the main navigation bar is visible. It should then be possible to open the **Generate** and **Food Labels** pages without an immediate runtime error.

Once this smoke test succeeds, the local setup can be considered complete and the application is ready for functional use. The following section describes the user workflows supported by the system.

2.2 User manual

This section describes how the application is used after the local environment has been configured and both services are running. The manual covers both the first-use path through the application and the main workflows available from the implemented user interface. It is based on the actual frontend structure of the system and focuses on the features that are most important from the user perspective: accessing the system, managing an account, generating recipes, processing food labels, and reviewing saved or historical results.

Although recipe generation can be used without authentication, creating an account unlocks the persistence features of the system. Authenticated users can store dietary preferences and allergy defaults, save generated recipes, and keep a history of both recipe generation and OCR-based food label extraction.

2.2.1 Quick-start walkthrough from launch to first meaningful action

This subsection provides a short first-use path through the application. Its purpose is to help a new evaluator or user reach a meaningful result quickly after startup, without yet exploring every feature in detail.

Once the backend and frontend have been started as described in the previous section, the application is opened at `http://localhost:5173`. The home page

appears first and gives access to the two main workflows of the system: recipe generation and food-label extraction.

The quickest meaningful action is recipe generation as a guest user. The user opens the **Generate** page, enters a comma-separated list of ingredients such as **chicken, rice, onion**, and submits the form with the **Generate Recipe** button. If the backend is configured correctly and the required AI services are available, the page returns a structured recipe containing a title, ingredients, preparation steps, a nutrition estimate, and any warning messages. This confirms that the main backend, retrieval, and frontend response flow is operational.

An alternative first-use path is the OCR-based workflow. The user opens the **Food Labels** page, selects an image of a nutrition label, and clicks **Extract Nutrition**. A successful response returns both raw OCR text and structured nutrition data. This confirms that file upload, OCR processing, and AI-assisted structuring are all working.

If the user wants to test persistence features as part of the initial exploration, the next step is to open the **Login** page, create an account, and log in. After authentication, generated recipes can be saved, profile defaults can be configured, and recipe or OCR history becomes available.

2.2.2 Feature-by-feature guide with annotated key screens

This subsection presents the main screens of the implemented user interface one by one. Each screenshot is accompanied by a short textual annotation describing the most important visible areas and the user actions supported on that screen.

Home page and basic navigation

When the frontend is opened in the browser, the application displays the home page. This page functions as the main entry point and gives the user a concise overview of the system. It introduces the three main capabilities of the application: recipe generation, OCR-supported nutrition label extraction, and data persistence.

The page also provides two primary call-to-action buttons:

- **Generate a Recipe**, which opens the recipe-generation page.
- **Scan a Food Label**, which opens the OCR-based nutrition label page.

At the top of the interface, the navigation bar provides access to the main pages of the system:

- **Home** returns to the landing page.
- **Generate** opens the recipe-generation workflow.
- **Food Labels** opens the nutrition label extraction page.
- **Saved** displays recipes that have been explicitly saved by the authenticated user.
- **History** displays the user’s generated recipe history, including unsaved entries.
- **Login** or **Account** opens the authentication and profile-management page.

If the user is already authenticated, the navigation bar also displays a **Logout** button. This immediately removes the stored session token on the client side and returns the application to guest mode.

The main visible areas of the screen are the headline section, the two primary action buttons, and the top navigation bar. Together, these elements guide the user toward either the recipe-generation or OCR workflow and provide access to the rest of the application. This layout is shown in Figure 2.1.

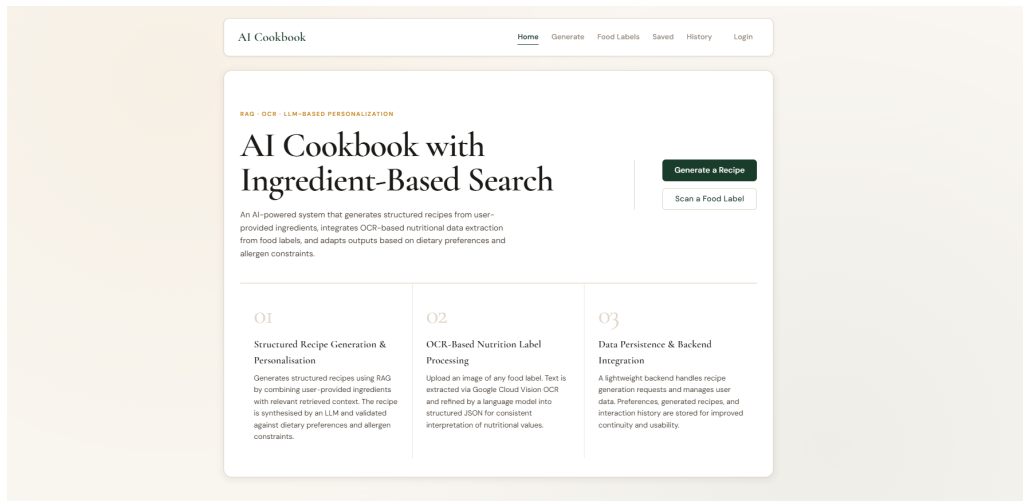


Figure 2.1: Home page of the AI Cookbook application

Account creation, login, and profile defaults

The `/login` page serves both as the authentication page for guest users and as the account-management page for authenticated users. Before login, the page presents

two tabs: **Login** and **Sign Up**. This allows a new user to create an account without leaving the page. The guest-facing authentication view is shown in Figure 2.2.

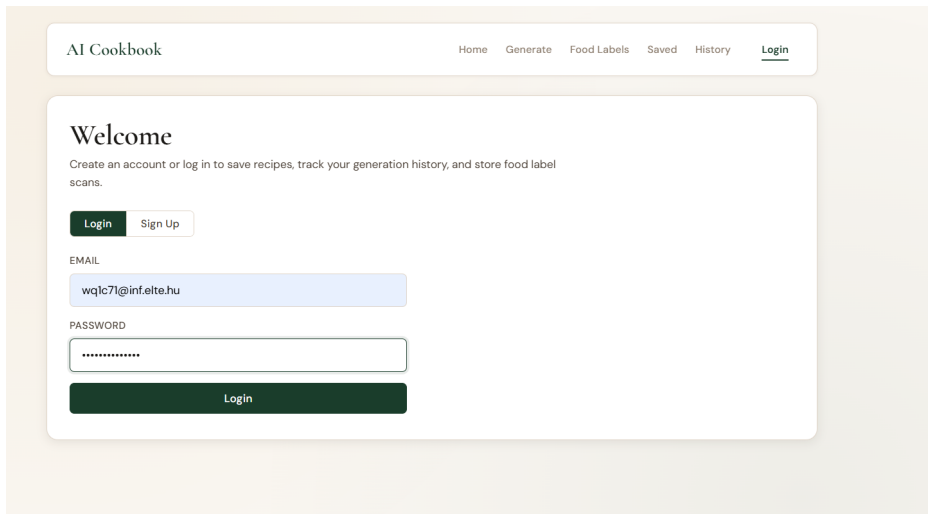
The screenshot shows a web interface for 'AI Cookbook'. At the top, there is a navigation bar with links: Home, Generate, Food Labels, Saved, History, and Login (which is underlined). Below the navigation bar is a 'Welcome' section with the text 'Create an account or log in to save recipes, track your generation history, and store food label scans.' There are two buttons: 'Login' (dark green) and 'Sign Up' (light blue). Below these buttons are two input fields: 'EMAIL' with the value 'wqlc7l@inf.elte.hu' and 'PASSWORD' with masked characters '*****'. At the bottom of the form is a dark green 'Login' button.

Figure 2.2: Login and registration page

To register a new account, the user switches to the **Sign Up** tab, enters an email address and password, and submits the form. If the email address is not already registered, the system creates the account and returns a confirmation message. The user can then switch back to the login tab and authenticate with the same credentials.

To log in, the user enters the registered email address and password and submits the login form. On successful authentication, the backend returns a JWT access token. The frontend stores this token locally and uses it automatically in subsequent authenticated requests.

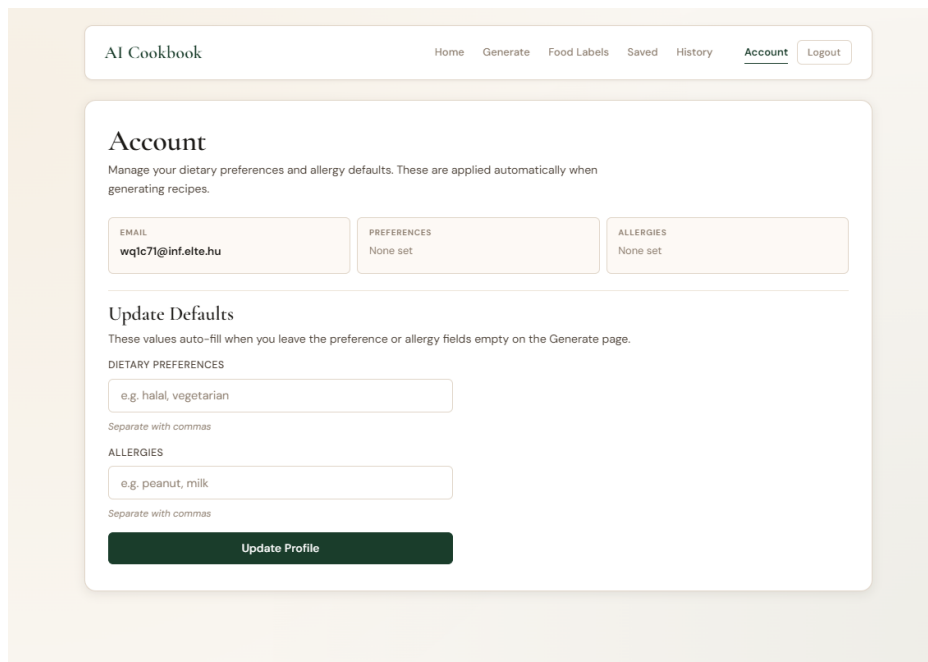
After login, the same page changes into an account-management view. Instead of the login form, the user sees a summary of the current profile data:

- the account email address,
- the saved list of dietary preferences,
- the saved list of allergies.

Below the profile summary, the page provides a form for updating default dietary preferences and allergy values. These defaults are important because they can be reused later on the recipe-generation page. If the user leaves the optional preference and allergy fields empty during recipe generation, the application automatically falls back to the saved profile values.

This makes the account page an important bridge between authentication and later personalized recipe-generation behavior, because the values stored here directly affect future requests when the user relies on saved defaults.

The key visible parts of this screen are the tab selector, the credential form, and, after login, the profile summary and update form. This page therefore functions both as the entry point for authentication and as the place where personalization defaults are maintained. The authenticated account-management view is shown in Figure 2.3.



The screenshot shows the 'Account' page of the 'AI Cookbook' application. At the top, a navigation bar includes links for 'Home', 'Generate', 'Food Labels', 'Saved', 'History', 'Account' (which is highlighted), and 'Logout'. The main content area is titled 'Account' and includes a subtitle: 'Manage your dietary preferences and allergy defaults. These are applied automatically when generating recipes.' Below this, there are three summary boxes: 'EMAIL' with the value 'wq1c71@inf.elte.hu', 'PREFERENCES' with 'None set', and 'ALLERGIES' with 'None set'. Further down, the 'Update Defaults' section explains that values auto-fill when preference or allergy fields are empty on the 'Generate' page. It contains two input fields: 'DIETARY PREFERENCES' with the example text 'e.g. halal, vegetarian' and 'ALLERGIES' with 'e.g. peanut, milk'. Both fields have a note 'Separate with commas' below them. At the bottom of this section is a dark green button labeled 'Update Profile'.

Figure 2.3: Account page after login, showing saved dietary defaults

Recipe generation

The recipe-generation page is the main workflow of the application. The user enters a comma-separated list of available ingredients and may optionally specify dietary preferences and allergies. If the user is already authenticated and has saved default preferences or allergy values in the account page, these defaults are applied automatically when the optional fields are left empty.

After the form is submitted, the frontend sends the request to the backend recipe-generation endpoint. The backend validates the input, retrieves relevant recipe context from the indexed corpus, invokes the language model, and returns a structured response. The result page shows the generated recipe title, ingredient list, any ad-

ditional ingredients that may be required, preparation steps, a nutrition estimate, and warning messages where applicable.

The screen is organized around three main areas: the input form, the recipe result card, and the save action shown for authenticated users. This makes it possible to move directly from ingredient entry to inspection of the returned recipe and, when logged in, to persistence of the result. The implemented screen is shown in Figure 2.4.

If the recipe was generated while the user was logged in, the page also offers a **Save Recipe** action. This stores the generated recipe in the user’s saved collection and also preserves it in the generation history.

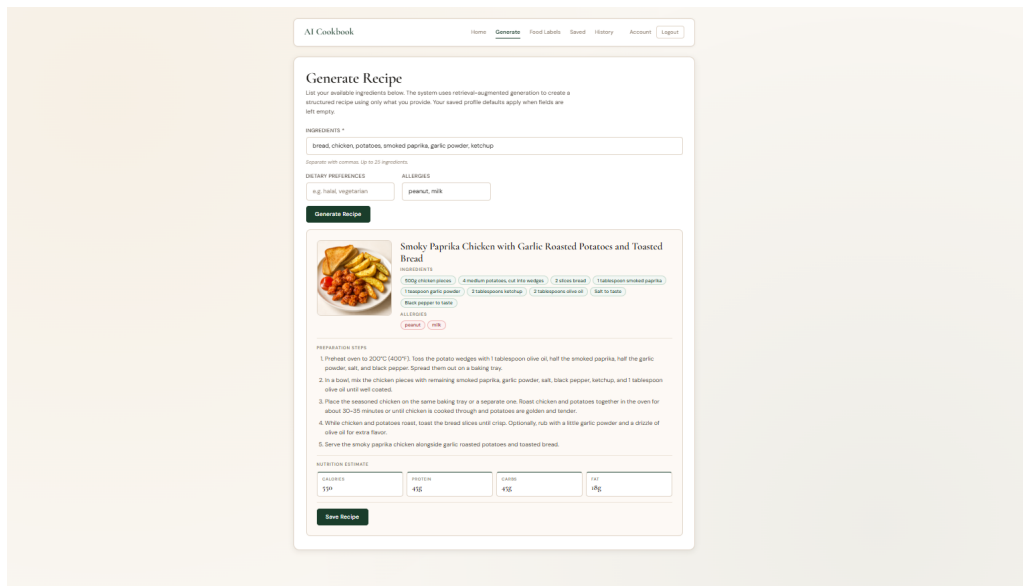


Figure 2.4: Recipe-generation page with a generated result

Food label extraction

The **Food Labels** page provides the OCR-based nutrition-label workflow. The user selects an image file containing a food label and submits it for processing. The page first shows a local preview of the selected image, then displays the extracted structured nutrition values together with the raw OCR text returned by the backend.

If the user is logged in, the extracted result can be saved to the personal label history. Saved entries remain available on the same page and can be browsed with pagination when multiple records exist. For unauthenticated users, extraction is still available, but persistent storage is disabled.

The main visible regions on this page are the upload area, the extraction result block, and the saved label history shown for authenticated users. This page therefore

combines immediate OCR usage with later review of previously saved extraction results. The screen is shown in Figure 2.5.

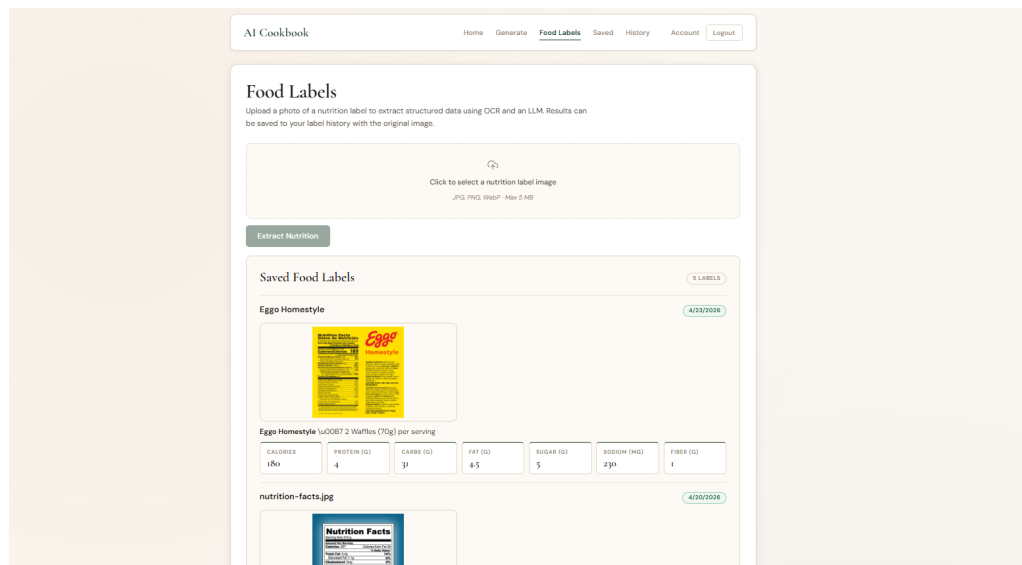


Figure 2.5: Food label extraction page with OCR result

Saved recipes

The **Saved** page lists recipes that the authenticated user has explicitly bookmarked after generation. Each entry is displayed as a card containing the recipe title, key ingredients, optional preferences and allergy markers, and the date on which the recipe was saved.

The cards can be expanded interactively to reveal the preparation steps and nutrition estimate. If the user is not logged in, the page does not display private content and instead instructs the visitor to authenticate first.

The key visible elements are the list of saved recipe cards, the saved date indicator, and the expandable details section. This keeps the page focused on compact review first and detailed inspection only when requested by the user. The page is shown in Figure 2.6.

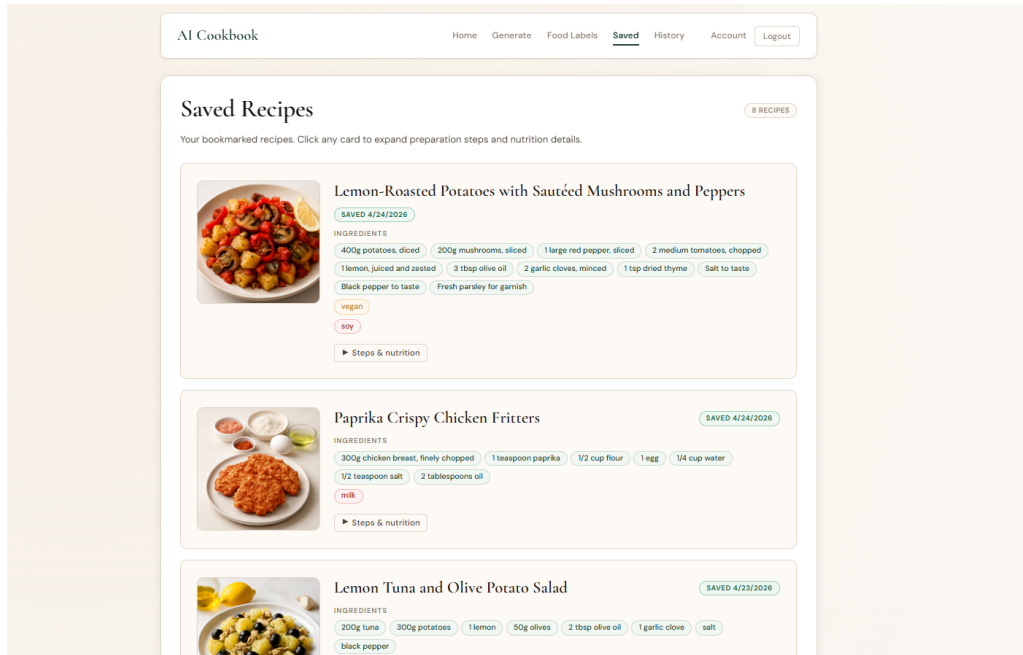


Figure 2.6: Saved recipes page

Recipe history

The **History** page provides a broader view than the saved recipes page. It includes every recipe generated by the authenticated user, including entries that were not explicitly saved. This makes the page useful for reviewing previous outputs, comparing generated results, and returning to earlier recipe ideas.

Each history card shows whether the recipe is merely generated or also saved, along with the creation timestamp and any warning messages returned during generation. As on the saved recipes page, each card can be expanded to display the preparation steps and nutrition estimate in full detail.

The visible structure of the page emphasizes recipe status, creation time, and warning information, so that the user can distinguish quickly between temporary and explicitly saved outputs before opening the full details. The page is shown in Figure 2.7.

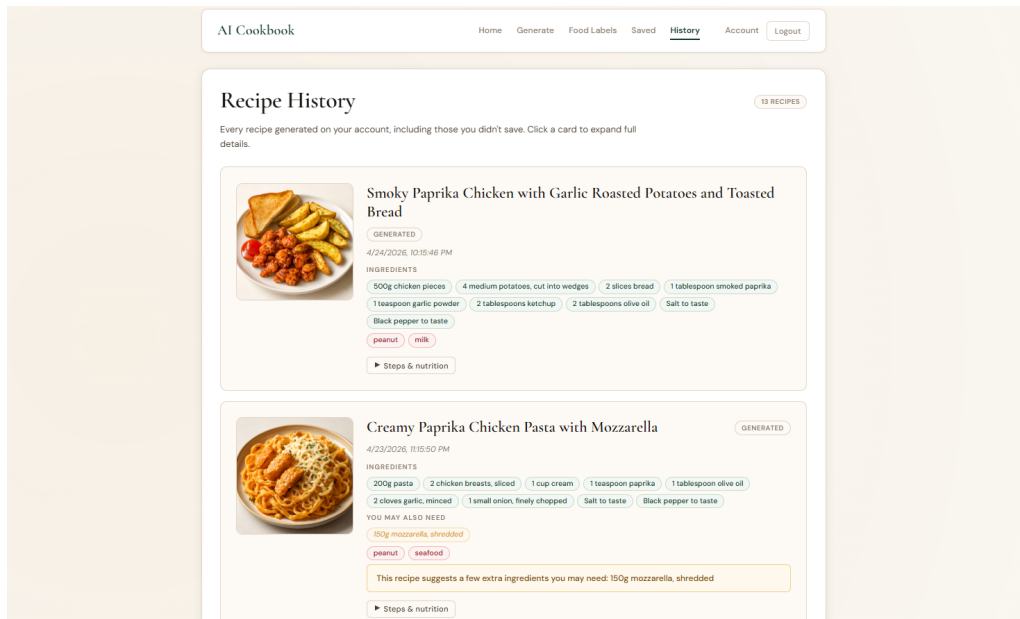


Figure 2.7: Recipe history page

2.2.3 Step-by-step workflows for the main features

This subsection collects the most important multi-step workflows of the application in a task-oriented format. These workflows correspond to the main interactive features promised by the project and implemented in the final system.

Creating an account and setting profile defaults

1. Open the **Login** page from the navigation bar.
2. Select the **Sign Up** tab.
3. Enter a valid email address and password, then submit the form.
4. Return to the **Login** tab and authenticate with the same credentials.
5. After login, review the account summary shown on the same page.
6. Enter dietary preferences and allergies in the update form and save the changes.

After this workflow, the profile contains stored defaults that can be reused during recipe generation when the optional preference and allergy fields are left empty.

Generating a recipe from ingredient input

1. Open the **Generate** page.
2. Enter a comma-separated ingredient list.
3. Optionally enter dietary preferences and allergies, or leave these blank to use the saved profile defaults when authenticated.
4. Click **Generate Recipe**.
5. Review the returned title, ingredients, additional ingredients, preparation steps, nutrition estimate, and any warning messages.

This is the core workflow of the application and demonstrates the interaction between frontend input, backend validation, retrieval, and LLM-based recipe synthesis.

Saving a generated recipe and reviewing it later

1. Log in before generating the recipe.
2. Complete the recipe-generation workflow on the **Generate** page.
3. Click **Save Recipe** in the result area.
4. Open the **Saved** page to verify that the recipe appears in the saved collection.
5. Open the **History** page to verify that the same recipe also appears in the generation history.

This workflow demonstrates the persistence behavior of the application. Saved recipes remain accessible from the dedicated saved collection, while all generated recipes remain visible in the broader history view.

Extracting nutrition data from a food label

1. Open the **Food Labels** page.
2. Select an image containing a readable nutrition label.
3. Submit the extraction request with **Extract Nutrition**.

4. Review the extracted structured nutrition fields and the raw OCR text returned by the backend.

This workflow exercises the upload, preprocessing, OCR, and AI-assisted structuring path of the system.

Saving an OCR extraction for later access

1. Log in before or after a successful extraction.
2. Complete the nutrition-label extraction workflow.
3. Click **Save to Food Labels**.
4. Verify that the extraction appears in the saved label history on the same page.
5. If multiple saved entries exist, use the pagination controls to browse them.

This workflow shows the persistence and later review path for OCR-derived nutrition data.

2.2.4 Accessibility, localisation, and access notes

This subsection summarizes practical usage characteristics that affect how different users can access the system.

Access control. The application does not implement multiple user roles such as administrator, editor, or moderator. The effective access model is binary: guest or authenticated user. Guests can use the recipe-generation and OCR-extraction workflows, but persistence-related features are restricted to authenticated users. Saved recipes, recipe history, and saved OCR history are therefore user-scoped features.

Localisation. The implemented user interface is available in English only. No runtime language-switching or multilingual interface layer is currently implemented.

Accessibility. The interface uses standard web form controls, buttons, and page navigation patterns. The file-selection area on the OCR page also supports keyboard-triggered activation through the implemented **Enter** and **Space** handlers. However, the project does not currently include a dedicated accessibility settings panel, screen-reader-specific optimization layer, or a formal accessibility compliance evaluation. Accessibility improvements are therefore a reasonable area for future development.

2.3 Troubleshooting, limitations, and frequently asked questions

This section summarizes the most important practical limitations of the application and collects the issues that a user is most likely to encounter during local use. Because the system depends on external AI services and local configuration, not every problem is caused by the frontend interface itself. In many cases, unsuccessful operation can be traced back to missing credentials, service availability, or incomplete local setup.

2.3.1 Known issues and recommended workarounds

The frontend loads, but requests fail immediately. This usually indicates that the backend service is not running or that `VITE_API_BASE_URL` does not match the actual backend address. The user should confirm that the FastAPI backend is running on the expected port and that the frontend environment points to the same API base URL.

Login or signup fails unexpectedly. If account-related requests fail, the backend should be checked first. The database migrations may not have been applied yet, or the `SECRET_KEY` required for token creation may be missing from the environment configuration.

Recipe generation returns an error. Recipe generation depends on the OpenAI API key and on the retrieval index prepared from the local recipe corpus. If the API key is missing, invalid, or rate-limited, the request may fail. The same can happen if the recipe corpus has not yet been indexed into the Chroma vector store.

OCR extraction fails or returns poor output. This may happen when the uploaded image is blurred, poorly cropped, rotated, too large, or contains limited readable text. OCR extraction also depends on valid Google Cloud Vision credentials, so incorrect credential paths can prevent the feature from working at all.

Recipe generation rejects the request even though ingredients were entered. This can happen when all provided ingredients conflict with the chosen allergy settings or dietary preferences. In that case, the backend rejects the request

rather than generating an unsafe recipe. The recommended workaround is to remove the conflicting ingredients or adjust the selected dietary constraints.

Saving features are unavailable. Saving recipes and OCR extraction results requires authentication. A guest user can still generate a recipe or process a food label, but the persistent save actions remain disabled until the user logs in.

Recipe thumbnail generation is missing for an otherwise valid recipe. The recipe-generation workflow attempts to attach an image thumbnail, but thumbnail generation depends on an external image-generation API. If that request fails, the backend still returns the generated recipe without an image and adds a warning message instead. In practice, the recommended workaround is simply to retry the request later, while treating recipe generation itself as the primary feature and the thumbnail as an optional enhancement.

An authenticated session appears to be lost after a restart or after token expiry. The frontend stores the access token locally and reloads the user profile at application startup. If the token is no longer valid, the frontend clears it and returns to guest mode. The recommended workaround is to log in again. Since the application does not implement refresh tokens or server-side session management, this is expected behavior of the current implementation.

2.3.2 System and functional limitations

The application is intended as a functional AI-assisted cooking and nutrition tool, but it also has clear limitations that should be understood by the user.

- **AI output is not guaranteed to be perfect.** Generated recipes, nutritional estimates, OCR-structured values, and generated thumbnails all depend on external AI services. The quality of these outputs may vary between requests.
- **Nutritional values are approximate.** The nutrition estimate attached to generated recipes is intended for guidance only. It should not be treated as a certified nutritional analysis.
- **Dietary filtering is rule-based rather than medically comprehensive.** The application handles allergies and preferences through keyword-based restrictions and prompt-level controls, which improves safety but does not replace expert dietary validation.

- **OCR accuracy depends heavily on image quality.** Clear, well-lit, and correctly oriented label images produce better results than low-resolution or visually noisy inputs.
- **OCR image persistence is best-effort rather than guaranteed.** During OCR extraction, the system attempts to store the uploaded label image before processing it. However, if that media-write step fails, the OCR workflow may still continue and return a successful structured nutrition result. In such cases, the extraction can still be used, but the saved history entry may not include a usable stored-image reference.
- **The software does not provide medical, clinical, or certified dietary decision support.** Neither generated recipes nor extracted nutrition values should be treated as medically verified output.
- **The software does not support editing or deleting saved content.** In the current implementation, users can create accounts, update profile defaults, generate recipes, save recipes, and save OCR extractions, but there is no interface for removing or editing saved recipes or saved OCR history entries after they have been stored.
- **The software does not implement advanced account management features.** There is no email verification, password-reset workflow, refresh-token support, or multi-role authorization model.
- **Recipe images are optional enhancements rather than guaranteed outputs.** A generated recipe can still be valid and usable even if no thumbnail image is attached. When image generation is unavailable, the system returns the recipe itself and treats the missing thumbnail as a secondary issue.
- **Recipe history and saved recipes are not the same thing.** The **History** page contains all generated recipe entries for the authenticated user, while the **Saved** page contains only the subset that was explicitly saved. Users should expect these two views to differ.
- **Saved OCR results are shown separately from saved recipes.** Food-label extractions do not appear on the main **Saved** page. They remain available on the **Food Labels** page under the saved OCR history section.

2.3.3 Frequently asked questions from realistic usage scenarios

Can the application be used without creating an account? Yes. Guests can generate recipes and use food-label OCR extraction. However, saved recipes, recipe history, saved food labels, and profile defaults require authentication.

Why does the recipe sometimes include additional ingredients? The recipe-generation workflow prioritizes the provided ingredients, but it may introduce a small number of extra ingredients when the language model determines that a practical recipe would otherwise be incomplete. These extra items are shown separately in the result.

Why do saved preferences appear even when I leave the fields empty on the Generate page? If the user is authenticated, the system automatically reuses the saved dietary preferences and allergy defaults from the account page when the optional fields are left blank.

Do I need to re-index the recipe corpus every time I start the project? No. The indexing step is normally required only for the initial local setup, or when the recipe corpus has changed and the retrieval index needs to be regenerated.

Are the OCR results stored automatically? No. OCR extraction and OCR saving are separate actions. The user must explicitly choose to save the extraction result if persistent history is desired.

Why can I generate recipes as a guest, but I cannot save them? The application separates feature access from persistence. Recipe generation and OCR extraction are available without authentication, but any data that is stored per user account requires login so that it can be associated with a specific user.

Why does the Saved page not show food-label results? Saved recipes and saved OCR extractions are stored separately in the current implementation. Recipes appear on the **Saved** and **History** pages, while saved food-label results are shown in the **Food Labels** page under the saved label history section.

Why does the same recipe sometimes show the same thumbnail even after changing dietary options? The current thumbnail cache is derived mainly from the ingredient list. This reduces repeated image-generation cost, but it may cause visually similar requests to reuse the same cached image even when dietary preferences differ.

Why was my recipe request rejected after I selected allergies or dietary preferences? If every provided ingredient conflicts with the selected restrictions, the system refuses the request instead of generating an unsafe result. In practice, this means at least one safe ingredient must remain after filtering for recipe generation to continue.

Chapter 3

Developer Documentation

This chapter documents the technical contribution of the thesis through three implemented thesis contributions. The first contribution is an OCR-based nutrition-label processing workflow that accepts uploaded food-label images, extracts text, and converts the result into a structured nutritional representation. The second contribution is a structured recipe-generation and personalization workflow that combines ingredient-based input, retrieval-augmented generation, and user constraints such as dietary preferences and allergens. The third contribution is the backend and persistence layer that integrates these workflows into one application, supports authenticated use, and stores user-specific data such as preferences, generated recipes, and OCR history.

The rest of the chapter explains these contributions from three perspectives. The design section presents the planned structure and responsibilities of the system, the implementation section shows how that design was realized in code, and the testing section evaluates the resulting application behavior.

3.1 Design

This section presents the design of the *AI Cookbook with Ingredient-Based Search* system. The goal of the design is not to repeat implementation details, but to explain how the system was structured before discussing concrete modules and code in the later implementation section. The design had to cover three core concerns of the project: user-facing interaction, backend application logic, and persistent data management.

The system solves two closely related problems. First, it helps users generate recipes from available ingredients while respecting dietary preferences and allergy constraints. Second, it extracts structured nutrition data from food-label images so that the user does not need to transcribe labels manually. These two features share the same application shell, authentication model, and persistence layer, but they differ in workflow shape: recipe generation is a retrieval- and generation-oriented pipeline, while OCR is an image-processing and information-extraction pipeline. The design therefore emphasizes clear separation between routes, services, providers, and storage.

3.1.1 Design Drivers and Requirements

Design goals and principles

The design of the system is guided by a small set of principles that reflect the actual risks and responsibilities of an AI-assisted cooking application.

- **Safety-oriented generation.** Allergy and dietary constraints should influence validation, retrieval, prompt construction, output checking, and warning generation instead of being treated as decorative metadata.
- **Explainability and transparency.** The system should make AI-supported decisions visible through warnings, excluded ingredients, raw OCR text, and structured nutrition output rather than hiding them behind a black-box interface.
- **Modularity.** Routes, services, providers, persistence models, and frontend task pages should remain separated by responsibility so that changes in one subsystem do not force unrelated rework elsewhere.

- **Reproducibility.** The application should be rebuildable from versioned source, migrations, dependency manifests, environment settings, and indexing scripts, which is important both for thesis evaluation and for repeatable local deployment.
- **User trust.** Users should be able to understand what was generated, what was inferred, and what limitations still apply. In this project, trust is built through predictable workflows and visible evidence rather than through stylistic polish alone.

These principles shape the rest of the design chapter. The requirements, workflows, architecture, data model, and user interface all follow the same priorities, so the chapter remains focused on one system instead of reading as a set of separate technical topics.

System purpose

The application is designed as an interactive web system rather than a single-run AI script. Users should be able to move between recipe generation, OCR-based nutrition extraction, account management, and history views through a stable browser interface. The backend is responsible for validating requests, enforcing ownership rules, orchestrating AI-supported workflows, and persisting user-specific results.

From a design perspective, the most important objective is controlled workflow execution. The application accepts user input that can be unsafe or incomplete, and it also depends on external AI services that may return imperfect outputs. For that reason, the design does not treat generation as a single black-box API call. Instead, it introduces validation, filtering, retrieval, output checking, optional retry, and selective persistence as explicit stages.

Primary actors and major use cases

The system has two human-facing roles and one external-service role.

- **Guest user.** Can generate recipes and run OCR extraction, but cannot persist private data.
- **Authenticated user.** Can use the guest-level functions and can also manage profile defaults, save recipes, and browse recipe and OCR history.

- **External AI services.** Provide text generation, embeddings, OCR text extraction, and recipe-image generation.

The main user-visible use cases are registration, login, profile update, recipe generation, recipe saving, browsing saved recipes, browsing recipe history, OCR extraction, and OCR result saving. The table below lists only external actors; the backend itself is part of the system and is discussed later in the architecture section.

Actor	Responsibilities
Guest user	Provides ingredients or nutrition-label images, triggers recipe generation and OCR extraction, reviews returned results, and explores the application without private persistence features.
Authenticated user	Owns guest capabilities and additionally manages stored profile defaults, saves generated recipes, browses recipe history, and persists OCR extraction history under a private account scope.
External AI services	Supply embeddings, recipe generation, recipe-thumbnail generation, and OCR text extraction through provider-facing interfaces used by the backend workflows.

Table 3.1: Actors and responsibilities in the AI Cookbook system

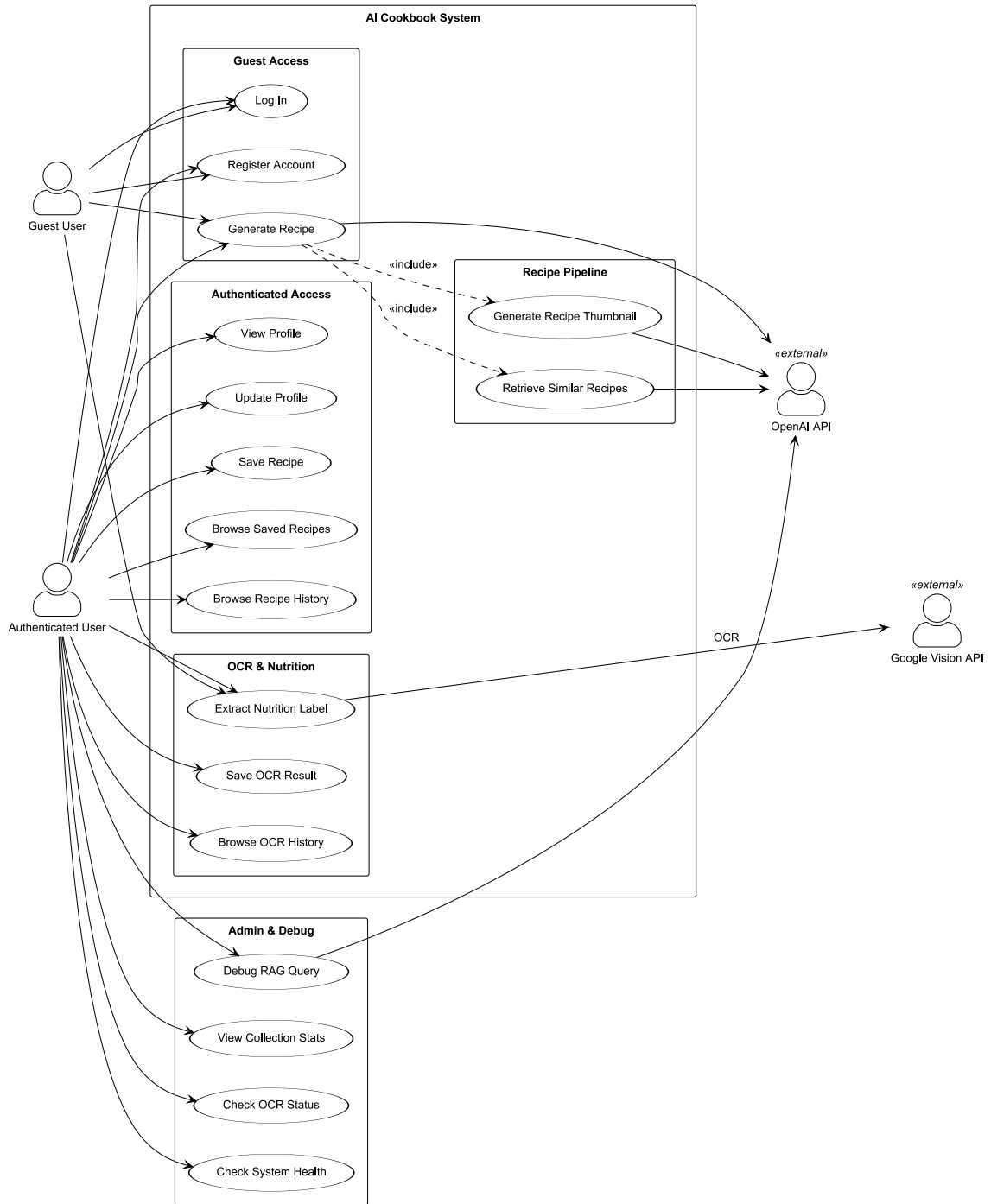


Figure 3.1: Use case diagram of the main user-visible system functions

Functional requirements

The main functional requirements of the system are the following.

1. **FR-1: Ingredient-based recipe generation.** The system shall accept a non-empty ingredient list and return a structured recipe containing a title, ingredient list, preparation steps, and a nutrition estimate.

2. **FR-2: Preference- and allergy-aware generation.** The system shall allow the request to include dietary preferences and allergy constraints, and these constraints shall influence the recipe generation workflow.
3. **FR-3: Retrieval-grounded generation.** Before recipe generation, the system shall retrieve relevant recipe context from an indexed recipe corpus and use that context to ground the language model.
4. **FR-4: User authentication.** The system shall support user registration, login, and retrieval of the current authenticated profile.
5. **FR-5: Profile defaults.** The system shall allow authenticated users to store dietary preferences and allergies as profile defaults.
6. **FR-6: Recipe history.** The system shall persist generated recipes for authenticated users and make them available through a history view.
7. **FR-7: Explicit recipe saving.** The system shall allow an authenticated user to mark a generated recipe as saved and display saved recipes separately from full history.
8. **FR-8: OCR extraction.** The system shall accept a food label image, validate it, extract raw text, and return structured nutrition fields.
9. **FR-9: OCR history.** The system shall allow authenticated users to save OCR results and later browse those extractions through a history view.
10. **FR-10: Protected ownership.** The system shall ensure that saved recipes, generation history, OCR history, and profile data are visible only to the owning authenticated user.

The design makes these requirements testable through explicit input limits, response-shape guarantees, and route behavior. Those measurable criteria are summarized in Table 3.2.

ID	Requirement focus	Acceptance criterion derived from the design
FR-1	Recipe request and response structure	A recipe request contains 1–25 ingredients; each ingredient string is at most 80 characters after trimming. A successful response contains a title, ingredient list, step list, and nutrition estimate object.
FR-2	Constraint-aware generation	Preference and allergy lists may each contain at most 10 entries. Ingredients conflicting with declared allergies or dietary preferences are excluded before generation and surfaced as warnings in the response.
FR-3	Retrieval grounding	For every recipe request, the backend queries the vector store for up to 10 candidates, filters unsafe matches, ranks the remainder, and forwards the best 3 contexts to the generation prompt.
FR-4	Authentication lifecycle	The system exposes dedicated signup, login/token, and current-user profile retrieval routes. Passwords are never stored in plaintext, and authenticated identity is reconstructed from a signed JWT.
FR-5	Profile defaults	The authenticated user can persist default preferences and allergies, and those values are returned as part of the profile representation.
FR-6	Recipe history	When the requester is authenticated, every successful recipe generation creates a history row owned by that user. History listing is paginated with page size in the range 1–50.
FR-7	Explicit saving	Saving a recipe requires authentication and a valid <code>generated_recipe_id</code> owned by the current user. The operation sets <code>is_saved</code> and records <code>saved_at</code> .
FR-8	OCR extraction pipeline	OCR upload accepts common image formats, rejects empty uploads, enforces a maximum size of 5 242 880 bytes and a maximum of 20 000 000 pixels, then returns both raw OCR text and structured nutrition fields.
FR-9	OCR history	Saved OCR results are persisted only for authenticated users and listed through a paginated history endpoint with page size in the range 1–50.
FR-10	Ownership protection	Recipe save/list/history and OCR save/history queries are always filtered by the authenticated user identifier, so one user’s records are not visible to another.

Table 3.2: Measurable acceptance criteria for the functional requirements

Non-functional requirements

The design also had to satisfy several non-functional requirements.

1. **NFR-1: Usability.** The main workflows should be discoverable through a small and stable set of pages.
2. **NFR-2: Safety-oriented behavior.** The recipe-generation workflow should explicitly consider allergies and dietary preferences rather than treating them as informational metadata only.
3. **NFR-3: Maintainability.** The system should separate presentation logic, request handling, service orchestration, persistence, and external-provider integration.
4. **NFR-4: Reproducibility.** A new local environment should be able to run the system from declared dependencies, environment configuration, migration files, and indexing scripts.

5. **NFR-5: Basic security.** The design should include password hashing, token-based authentication, user-scoped data access, and route-level rate limiting.
6. **NFR-6: Thesis-scale performance.** The system should remain responsive for ordinary interactive use with a moderate local vector index and paginated history access.

Table 3.3 translates these quality goals into concrete design targets that can be checked against the implemented system.

ID	Quality goal	Concrete design target
NFR-1	Usability	Top-level navigation is limited to six stable routes: Home, Generate, Food Labels, Saved, History, and Login/Account. Each major user task has a dedicated page rather than a shared overloaded view.
NFR-2	Safety-oriented behavior	Conflicting input ingredients are removed before prompt construction, unsafe generated outputs are checked against restriction keywords, and the recipe workflow may issue one corrective retry before failing.
NFR-3	Maintainability	Route handlers do not call provider SDKs directly. External OCR, image generation, embeddings, and LLM calls are isolated behind service or provider modules. API schemas are separated from ORM entities.
NFR-4	Reproducibility	Backend configuration is centralized in environment-backed settings, relational changes are represented through migration files, and recipe-corpus preparation plus vector indexing are scriptable offline steps.
NFR-5	Security	Passwords are hashed with a dedicated password-hashing library, access tokens expire after 60 minutes, and route-specific rate limits are enforced: 5 requests / 15 minutes for auth routes, 15 / 15 minutes for recipe generation, and 10 / 15 minutes for OCR extraction.
NFR-6	Thesis-scale performance	Saved and history queries are paginated with page size capped at 50. Retrieval is bounded to 10 vector candidates and 3 final context items, limiting per-request downstream prompt size.

Table 3.3: Concrete design targets for the non-functional requirements

Narrative use-case descriptions

The use-case diagram in Figure 3.1 shows the full actor-function landscape. The following three narrative descriptions focus on the workflows that shape the system design most strongly.

UC-1: Generate a recipe from ingredients. **Primary actor:** Guest user or authenticated user.

Preconditions: The user opens the Generate page and provides at least one non-blank ingredient. Optional dietary preferences and allergies may also be provided directly or inherited from the profile defaults of an authenticated user.

Main success scenario:

1. The user submits a recipe-generation request.
2. The backend validates the request structure and trims the string values.
3. The workflow removes ingredients that conflict with declared allergies or dietary preferences.
4. The system retrieves relevant safe recipe context from the vector store and builds a grounded generation prompt.
5. The language model returns a structured recipe, which is checked for restriction violations and supplemented with warnings if needed.
6. The system attempts to generate a thumbnail image.
7. If the user is authenticated, the generated recipe is stored in the recipe-history table.
8. The user receives the final recipe, optional warnings, and the thumbnail if image generation succeeded.

Alternative flows:

1. If every provided ingredient conflicts with the declared constraints, the request is rejected with a validation-style error and no generation is attempted.
2. If the first model output still violates the restrictions, a corrective retry is executed once.
3. If thumbnail generation fails, the recipe is still returned, but a warning is attached and the response contains no image URL.

Postconditions: A structured recipe has been delivered. If the requester was authenticated, the unsaved generated result exists in the history table and can later be explicitly saved.

UC-2: Save a generated recipe. Primary actor: Authenticated user.

Preconditions: The user is logged in and already owns a generated recipe entry created by the recipe-generation workflow.

Main success scenario:

1. The user selects the Save action on a generated recipe.
2. The frontend sends the stored `generated_recipe_id` to the backend.
3. The backend verifies that the referenced recipe belongs to the current authenticated user.
4. The system marks the recipe as saved and records the save timestamp.
5. The saved-recipes view becomes able to display the entry separately from full history.

Alternative flow: If the identifier does not exist or belongs to another user, the save request is rejected with a not-found response.

Postconditions: The recipe remains in history and is also visible through the saved-recipes listing.

UC-3: Extract structured nutrition data from a food label. Primary actor: Guest user or authenticated user.

Preconditions: The user selects an image file in a supported format and the upload satisfies the configured size and pixel limits.

Main success scenario:

1. The user uploads a nutrition-label image from the Food Labels page.
2. The backend validates file type, non-emptiness, size, and image dimensions.
3. The system attempts to store the upload in the backend-managed media directory before OCR processing continues.
4. The image is preprocessed for OCR and forwarded to the OCR provider.
5. The raw OCR text is returned and then converted into a fixed nutrition JSON structure.
6. The user sees both the raw extracted text and the structured nutrition fields.
7. If the user is authenticated and chooses to save the result, the extraction is persisted to the OCR history table.

Alternative flows:

1. If the uploaded file is empty or exceeds the configured limits, the request is rejected before provider calls are made.
2. If OCR succeeds but nutrition structuring fails, the backend rejects the request because the workflow requires a complete structured nutrition result.

Postconditions: The user receives either a validated nutrition payload or a clear failure response. Authenticated users may persist the result for later review.

3.1.2 Architecture and System Design

Architectural style

The system follows a layered client-server architecture. The frontend is responsible for presentation, route navigation, and local interaction state. The backend exposes an HTTP API for validation, authentication, persistence, and orchestration of the AI workflows. Storage is split across three backend-managed layers: a relational database, a vector store, and the local filesystem for media assets.

This structure follows directly from what the application has to do. User-scoped history persistence, OCR validation and preprocessing, retrieval from the recipe corpus, prompt construction, and provider calls all belong on the backend side. The browser mainly handles forms, navigation, conditional rendering, and result presentation. Keeping these responsibilities separate makes the main workflows easier to control and easier to extend.

Layered architecture

The layered style is the main organizational rule of the system. The browser layer is responsible for forms, navigation, local interaction state, and result presentation. It does not validate business constraints beyond basic user-interface checks, and it never communicates directly with the relational database, vector store, filesystem, or provider SDKs.

The backend API layer serves as the controlled entry point into the system. Route modules receive JSON requests, validate them through schemas, resolve authentication state, and delegate non-trivial work to the service layer. The service layer then coordinates recipe generation, OCR processing, retrieval, image generation, and response assembly in a fixed order. Below that, the storage and provider layers separate

persistent state from external integrations: relational tables store user-owned data, the vector store supports retrieval, the local filesystem stores media, and provider adapters isolate third-party API details.

This layering is especially important because the system combines ordinary web-application responsibilities with AI-assisted workflows. Without explicit layers, validation, persistence, provider logic, and result presentation would quickly become entangled.

Architectural decomposition rationale

The structural decomposition of the system follows directly from the two main workflows introduced earlier. Recipe generation requires a request–validation boundary, a retrieval subsystem, a generation orchestration layer, persistence for user-owned history, and optional media generation. OCR-based nutrition extraction requires an upload boundary, image preprocessing, provider-facing extraction logic, semantic structuring of the extracted text, and optional persistence of results. These workflows share authentication, ownership control, and the browser-facing navigation shell, but they differ enough internally that they are kept as separate backend modules.

This decomposition also keeps later changes manageable. The frontend is limited to page interaction and result presentation. The backend controls validation, workflow order, and access rules. Persistent state is split into relational data, retrieval data, and media assets. Provider-specific behavior is kept behind separate boundaries, so changes in OCR or image generation do not ripple through the user-facing API or the persistence model.

High-level component model

At the highest level, the system is decomposed into the following components:

- **Single-page frontend** for user-facing pages and interaction.
- **Backend API layer** for request routing and workflow orchestration.
- **Authentication/profile module** for user identity and default constraints.
- **Recipe-generation module** for RAG-based recipe creation and recipe persistence.

- **OCR module** for label-image processing and nutrition structuring.
- **Relational database** for transactional persistence.
- **Vector store** for retrieval support.
- **Filesystem media storage** for OCR uploads and generated thumbnails.
- **External AI providers** for OCR, text generation, embeddings, and image generation.

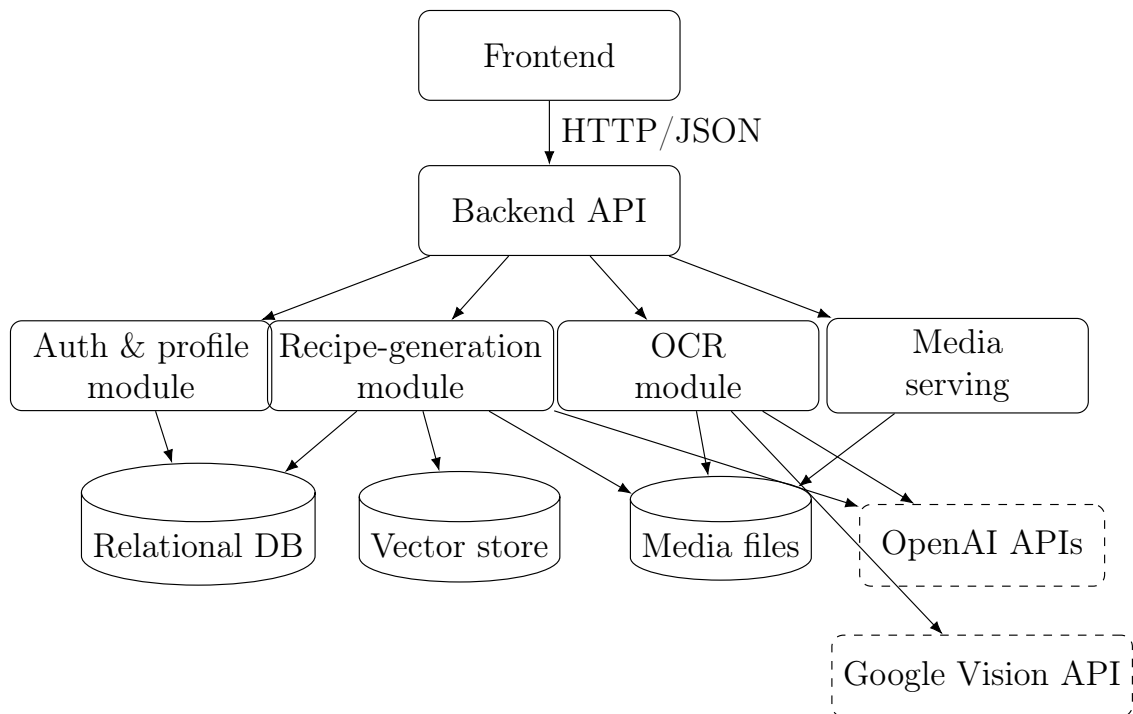


Figure 3.2: Component diagram of the system architecture

The component model also makes the layer boundaries visible. The frontend never accesses the relational database, vector store, or provider APIs directly. It talks only to the backend API. Within the backend, route modules remain thin and delegate non-trivial work to services, providers, and storage abstractions. This same decomposition reappears in the more detailed UML artifacts presented next.

Layer	Implemented units	Primary responsibility
Frontend interaction layer	React pages, navigation shell, shared auth context, reusable presentation components	Collect user input, maintain page-level interaction state, invoke backend routes, and render generated or retrieved results.
API boundary	FastAPI route modules and Pydantic request/response schemas	Validate request shape, enforce authentication or ownership checks, and expose a stable JSON contract to the browser.
Workflow and orchestration layer	Recipe-generation, OCR, retrieval, and image services	Execute multi-step application logic in a fixed order and isolate external-provider interactions from route handlers.
Provider integration layer	OCR and image-provider abstractions plus concrete adapters	Hide vendor-specific SDK calls behind application-facing interfaces such as <code>extract_text()</code> and <code>generate_png()</code> .
Transactional persistence layer	SQLAlchemy entities, database session handling, Alembic migrations	Store user accounts, generated recipes, OCR history, ownership relations, and pagination-supporting indexes.
Retrieval and media layer	Chroma vector store, recipe-indexing scripts, filesystem media storage	Persist retrieval documents and embeddings, store OCR uploads and thumbnails, and serve media files back to the frontend.

Table 3.4: Layer responsibilities in the implemented system

Technology selection rationale

The selected technologies follow the needs of the implemented architecture instead of being chosen as a generic full-stack template. The frontend is built as a React single-page application because the main user tasks—recipe generation, OCR extraction, saved results, and history browsing—share one navigation shell but require different forms, loading states, and result layouts. A page-oriented React frontend keeps those task flows separate while still allowing shared authentication state and reusable result components.

The backend is built with FastAPI because the application depends on clear request validation and on keeping route handlers thin. Recipe generation, OCR extraction, save operations, and history listing all accept structured input and return structured output, so schema-based request and response handling fits the project well. This is especially useful in the recipe workflow, where ingredient lists, allergies, preferences, warnings, and nutrition fields all have to remain well-formed across several processing stages.

Relational persistence is handled through SQLAlchemy with migration support, because the project stores ownership-sensitive data rather than anonymous prompt logs. User accounts, generated recipes, save state, and OCR history need stable identifiers, foreign-key relationships, and queryable timestamps. These requirements match a relational model more closely than a document-only store.

The retrieval layer uses a persistent vector store because recipe generation is grounded in a local recipe corpus. The system does not use retrieval as an optional

enhancement; it depends on retrieval to supply context before prompt construction. For that reason, embeddings and similarity search are treated as part of the architecture itself.

Media files are stored on the local filesystem because OCR uploads and generated thumbnails act as supporting assets around the main application data. The project needs these files to be saved, reused, and served back to the frontend, but it does not need cloud-scale media infrastructure for the bachelor-thesis scope. External AI providers are used only where the project needs capabilities that are not implemented locally, namely text generation, embeddings, OCR, and thumbnail generation.

Object-oriented modelling artifacts

Although part of the backend is implemented in a function-oriented style, the system still has a meaningful object-oriented structure. The most important implemented classes fall into three groups:

- **Persistent domain classes**, such as `User`, `GeneratedRecipe`, and `OcrExtraction`.
- **Schema/data-transfer classes**, such as request and response models used at API boundaries.
- **Provider abstraction classes**, such as `OcrProvider`, `GoogleVisionOcrProvider`, `RecipeImageProvider`, and `OpenAIRecipeImageProvider`.

The following class diagrams expand the earlier component view into the implemented backend structures. The overview is split into two figures to keep the diagrams readable at thesis page scale: one diagram focuses on domain classes, provider abstractions, and the module-level service boundaries around them, while the other focuses on schemas, configuration, and infrastructure support.

These diagrams describe the implemented structure of the system. The persistent entities correspond to the relational model, the schema classes define the API boundary, and the provider abstractions separate vendor-specific OCR and image-generation logic from the rest of the application. They are mixed on purpose: some parts of the backend are class-based, while others are clearer as service modules.

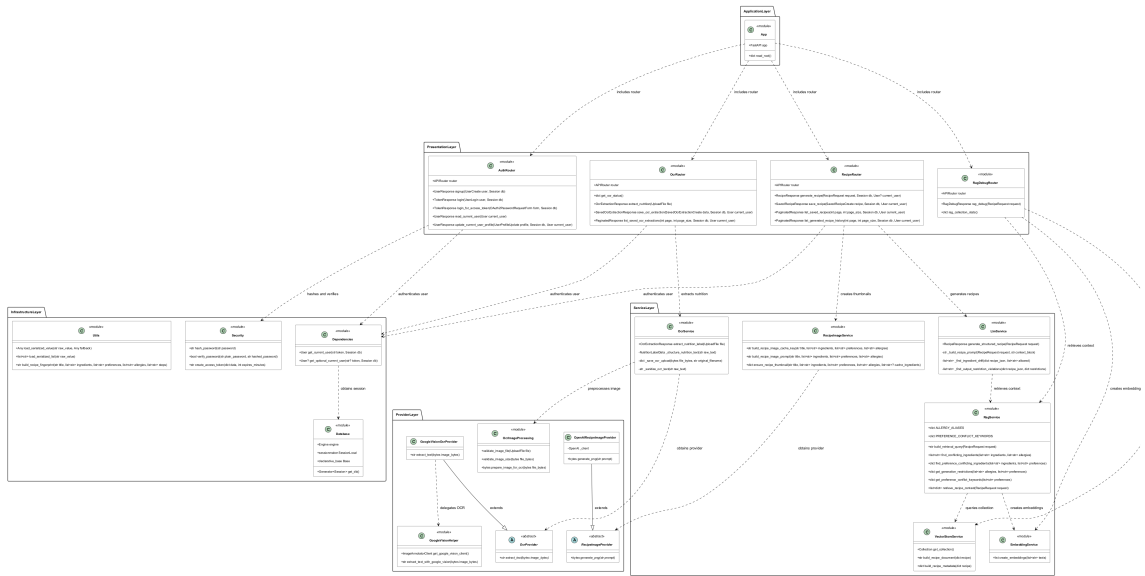


Figure 3.3: Class diagram of the domain and backend-service structure, showing implemented entity classes, provider abstractions, and their relationships



Figure 3.4: Class diagram of the infrastructure and schema layer, showing API contracts, configuration, middleware, and rate-limiting components

The diagrams follow that split instead of pretending that every part of the system fits one style equally well.

To make this correspondence explicit, Table 3.5 links the main structural artifacts to the implementation units they describe most directly.

Diagram artifact	Most visible implemented elements	Implementation area
Domain/service class view	<code>User</code> , <code>GeneratedRecipe</code> , <code>OcrExtraction</code> , provider abstractions, route-to-service dependencies	Domain model, recipe workflow services, OCR workflow services, and provider abstractions.
Infrastructure/schema class view	<code>Settings</code> , <code>RateLimitMiddleware</code> , <code>RateLimitRule</code> , request and response schemas	Configuration, middleware, and schema-definition modules.
Sequence view of recipe generation	Route entry, generation service, retrieval service, provider calls, conditional persistence	Recipe route handling together with retrieval and generation services.
State-transition view	Validation, filtering, retrieval, retry, image generation, authenticated persistence	Request validation, recipe orchestration, and thumbnail-generation modules.
Use-case and navigation views	User-visible routes and persistence-related page separation	Frontend routing, task pages, and authenticated navigation views.
ER and relational views	Entity ownership, nullable vs non-null foreign keys, indexes, and denormalized payload fields	ORM entity definitions and schema-migration history.

Table 3.5: Direct correspondence between the design artifacts and the implementation units

Core workflow design

The most important backend behavior is not a static entity relation but a controlled multi-stage workflow. Recipe generation especially benefits from dynamic modelling because its correctness depends on the order of operations: validation must happen before retrieval, retrieval before prompt construction, and output checking before persistence.

The following sequence diagram captures the expected interaction path for a recipe request. It shows how the frontend, backend route, service layer, retrieval subsystem, and provider calls cooperate during a single end-to-end generation.

The sequence view is complemented by a state-oriented and a data-flow view. The state machine is useful because a recipe request may pass through correction states before it becomes acceptable, and because thumbnail generation is intentionally modelled as a non-fatal step. The data flow diagram is useful because it clarifies which data is produced or transformed at each stage and which stores participate in the pipeline. Together, these views document the static structure and the dynamic behavior of the system from complementary angles.

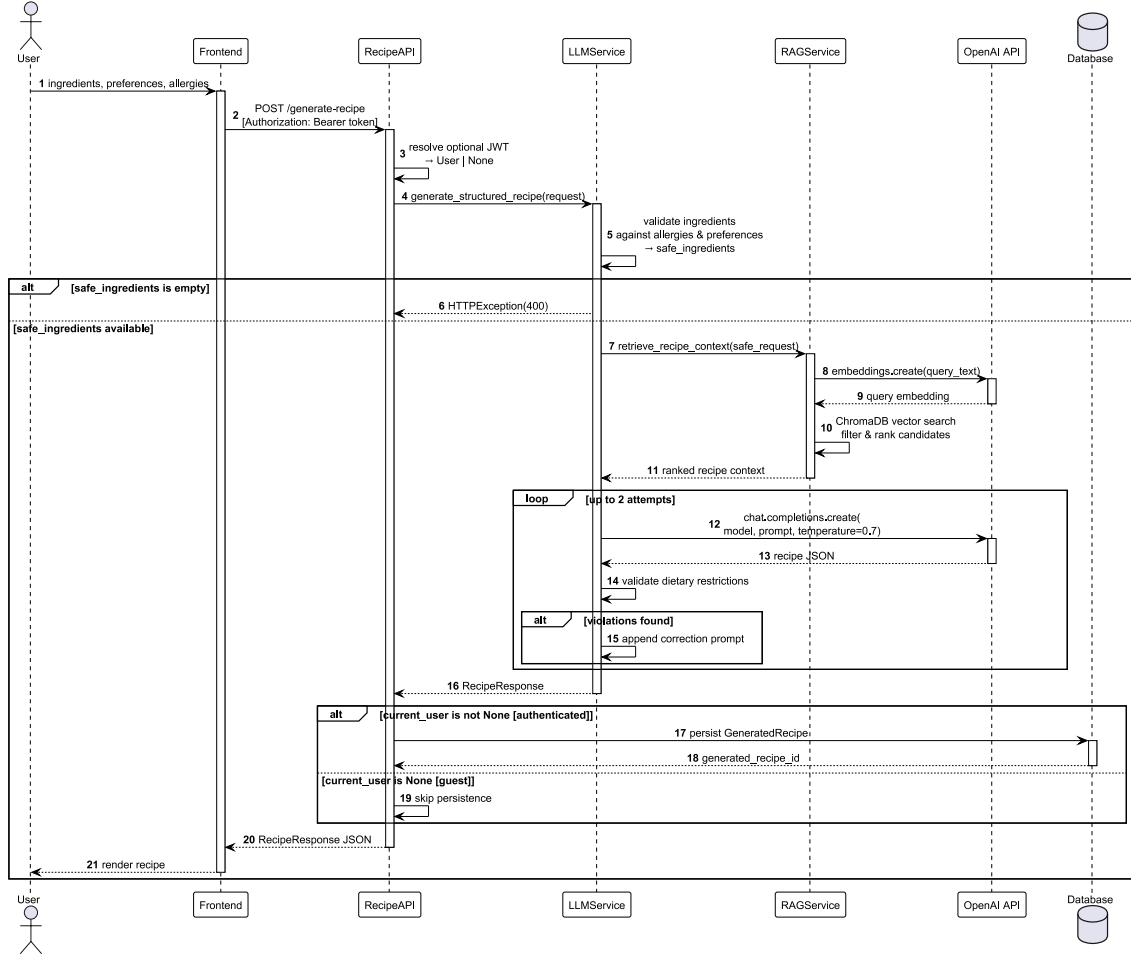


Figure 3.5: Sequence diagram of the recipe-generation workflow, illustrating the call chain from user input through context retrieval, LLM invocation, and conditional database persistence

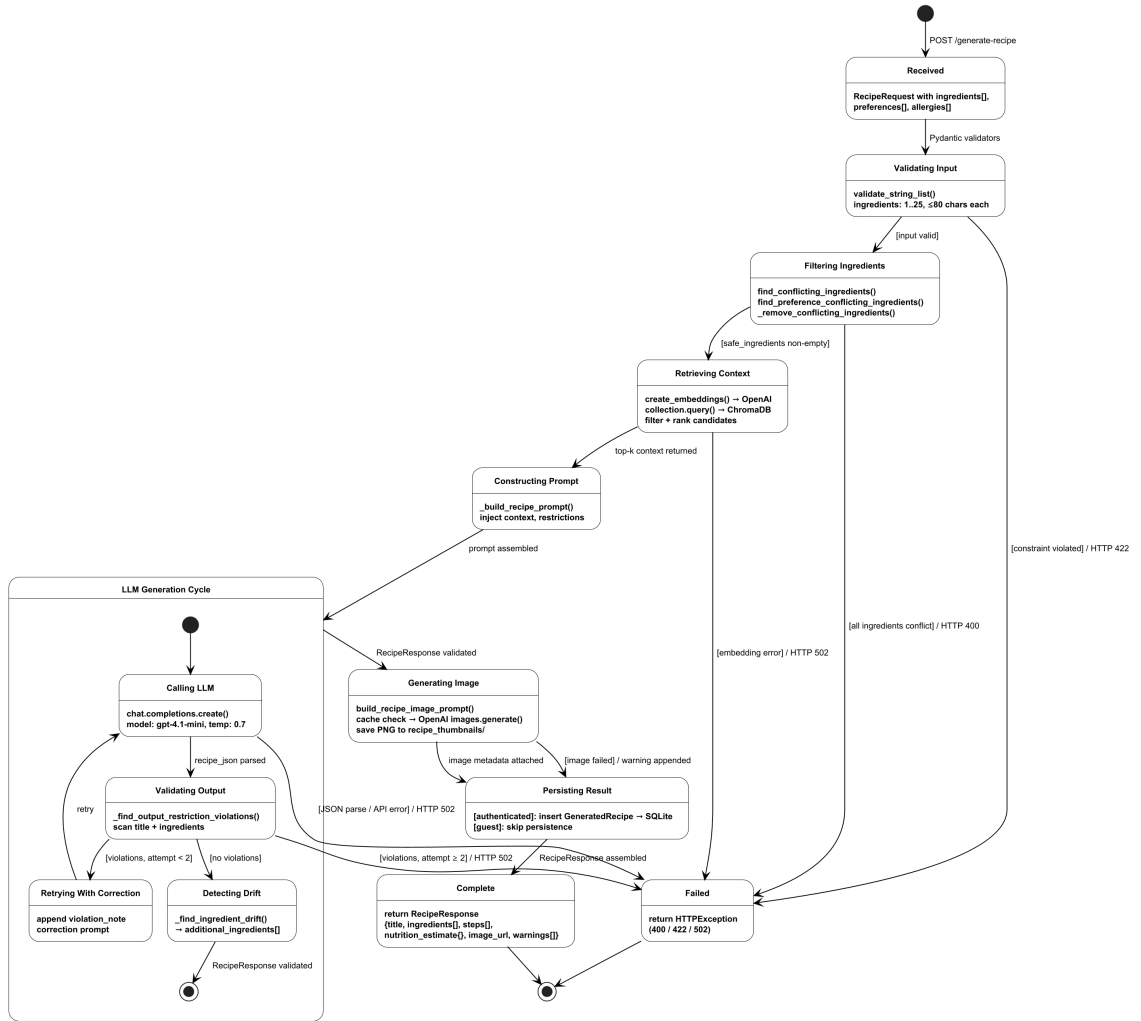


Figure 3.6: State-transition diagram of the recipe request lifecycle, showing the path from initial submission through input validation, ingredient filtering, retrieval-augmented generation, output verification, thumbnail generation, and persistence

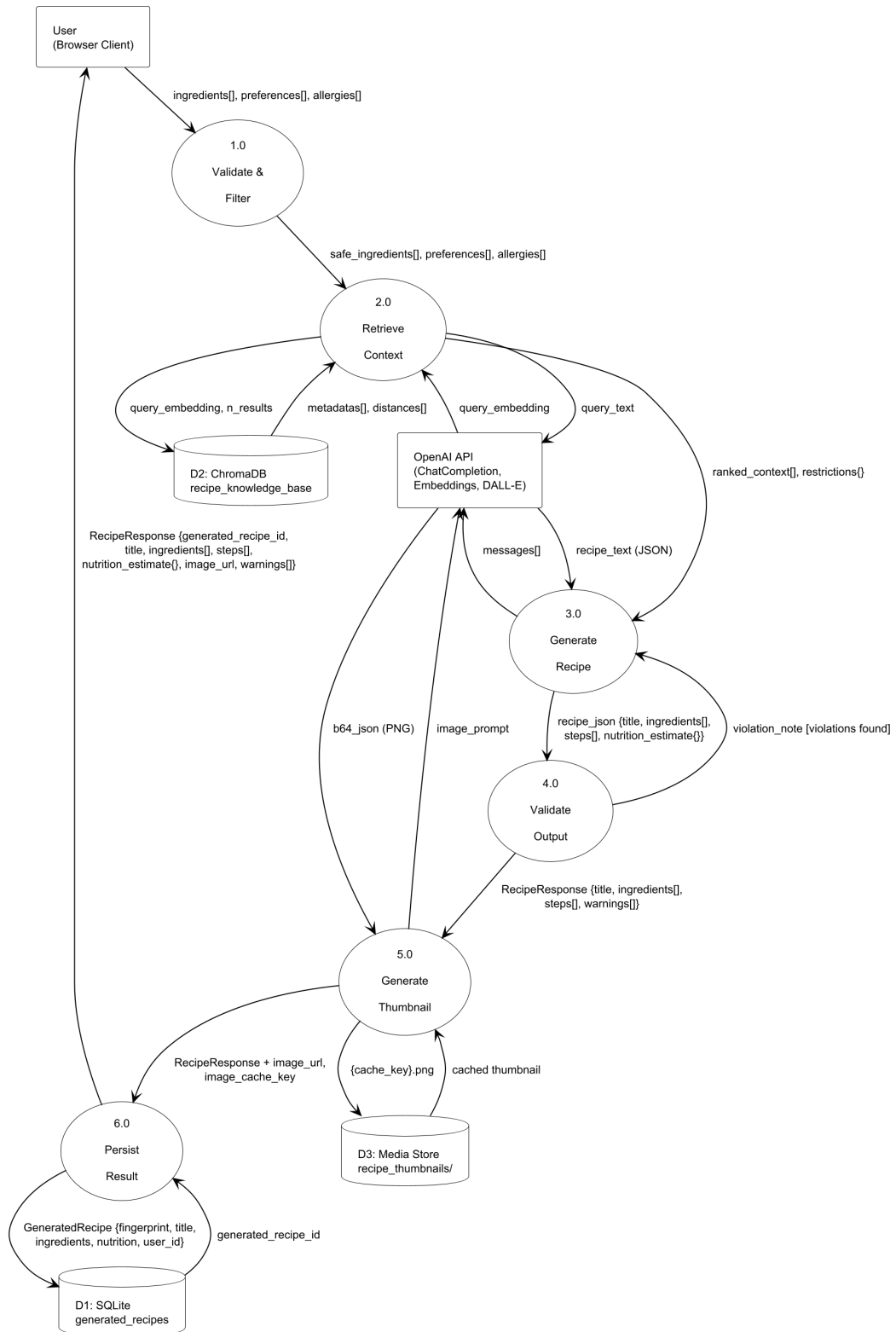


Figure 3.7: Level 1 data flow diagram of the recipe-generation pipeline, showing processes, data stores, external entities, and field-level flows derived from the application codebase

Two design decisions are especially important at workflow level. First, recipe generation is not a single prompt call. The workflow performs input cleaning, conflict detection, safe-context retrieval, prompt construction, output validation, optional retry, optional image generation, and conditional persistence. Second, OCR is explicitly split into optical extraction and semantic structuring. This separation keeps failure causes easier to diagnose and allows the OCR provider and the nutrition-structuring logic to evolve independently.

API design and interaction model

The frontend communicates with the backend almost entirely through JSON API requests. The only exception is OCR upload, where the image itself is transferred as multipart form data before the backend returns a structured JSON result. In all cases, the interaction model remains the same: the frontend sends user input to a dedicated backend route, the backend validates the request, executes the relevant workflow, and returns a structured response that the frontend can render directly.

This interaction model keeps the frontend simple. Each page only needs to know which route to call and what response structure to expect. The backend remains responsible for authentication, ownership checks, workflow order, provider calls, and persistence.

API area	Purpose and example responsibilities
Authentication API	Supports signup, login, token issuance, current-user retrieval, and profile-default updates. Protected profile operations require authentication and always operate on the resolved current user rather than on client-supplied ownership identifiers.
Recipe API	Accepts recipe-generation requests, returns structured recipes with warnings and optional image metadata, persists generated history for authenticated users, and exposes saved-recipe and full-history listings filtered by the authenticated owner.
OCR API	Accepts nutrition-label uploads, validates image payloads, returns raw OCR text together with structured nutrition fields, and allows authenticated users to save and browse OCR history under their own account scope.
Media/static API	Exposes backend-managed OCR uploads and generated thumbnails as static files addressable from frontend result views and history pages.

Table 3.6: Main API areas and their responsibilities

Decision transparency and user-visible evidence

Because the application relies on external AI services, the design does not hide intermediate evidence or corrective decisions from the user. Transparency appears in several concrete forms. Recipe generation can return warning messages when some provided ingredients were excluded, when the generated result requires additional ingredients, or when thumbnail generation fails and the recipe is returned without

an image. The OCR workflow displays the raw extracted text and the structured nutrition interpretation side by side, which allows the user to judge whether the semantic structuring step remained faithful to the visible label content.

This transparency is part of making the system usable. OCR and language-model services can still return imperfect results, so the application should show what was read directly from the input and what was inferred later in the workflow.

Recurring design mechanisms

The project uses a layered architecture together with a small number of recurring design mechanisms that appear across multiple modules. This is a better description of the system than forcing every decision into a formal pattern taxonomy. The most important structural pressures in the project are provider substitution, explicit API contracts, workflow orchestration, and controlled persistence boundaries.

Mechanism	Implemented form	Implementation area	Why it fits the system
Provider substitution	OCR and image-generation interfaces with concrete implementations	Provider abstractions for OCR and recipe-image generation.	External providers can change without forcing route or workflow logic to depend on vendor-specific SDK calls.
Centralized provider construction	Backend-owned provider selection and reuse points	Provider setup and service initialization logic.	Route handlers and service code ask for capabilities, not concrete classes, which keeps creation logic out of application flow.
SDK adaptation	Thin wrappers around OpenAI and Google Vision calls	Concrete provider adapter modules.	External APIs are translated into application-level operations such as text extraction and image generation.
Service orchestration boundary	Route modules delegate non-trivial workflows to narrow service entry points	Route modules and workflow-service modules.	Multi-stage workflows stay reusable and testable instead of being duplicated across endpoints.
Validated data contracts	Request and response schemas separated from ORM entities	API schema layer.	API contracts remain explicit, validated, and independent from persistence concerns.
Cross-cutting middleware	Shared rate-limiting and CORS configuration installed at bootstrap time	Application bootstrap and middleware configuration.	Concerns that apply to many routes are kept out of endpoint logic and enforced uniformly.

Table 3.7: Recurring design mechanisms used throughout the system

These decisions matter mainly for maintainability. Route handlers do not need to know how provider authentication works or how OCR and image payloads are encoded; they rely on service or provider interfaces. Likewise, the frontend depends

on API contracts instead of the database schema. That separation is useful in this project because recipe generation, OCR, retrieval, and persistence all meet in the same application. Some parts are best described through classes and relationships, while others are better described as step-by-step workflows, and the design chapter follows that same split.

3.1.3 Data Design

Storage responsibilities

The system uses three forms of storage because the data has three different roles:

- **Relational storage** for users and historical user data.
- **Vector storage** for recipe-context retrieval.
- **Filesystem storage** for OCR upload images and generated thumbnails.

This storage split follows the way the application actually uses data. User accounts and ownership relations fit a relational model. Retrieval needs embeddings and similarity search. Uploaded images and thumbnails are easiest to keep as files referenced from the application data. The dynamic behavior around recipe generation is documented earlier through the sequence, state-transition, and data-flow diagrams in Section 3.2, because the non-trivial data transformations in this project arise from workflow execution rather than from the static relational model alone.

Entity–relationship model

The relational core of the system contains three entities: **User**, **GeneratedRecipe**, and **OcrExtraction**.

- A **User** stores identity and profile-default information.
- A **GeneratedRecipe** stores the result of a recipe generation request and whether the result was explicitly saved.
- An **OcrExtraction** stores the result of a nutrition-label extraction associated with a user.

The entity relationships are intentionally simple, because the main complexity of the application lies in workflow logic rather than in a large relational domain model:

- one user may have many generated recipes,
- one user may have many OCR extractions,
- each generated recipe belongs to at most one user,
- each OCR extraction belongs to exactly one user.

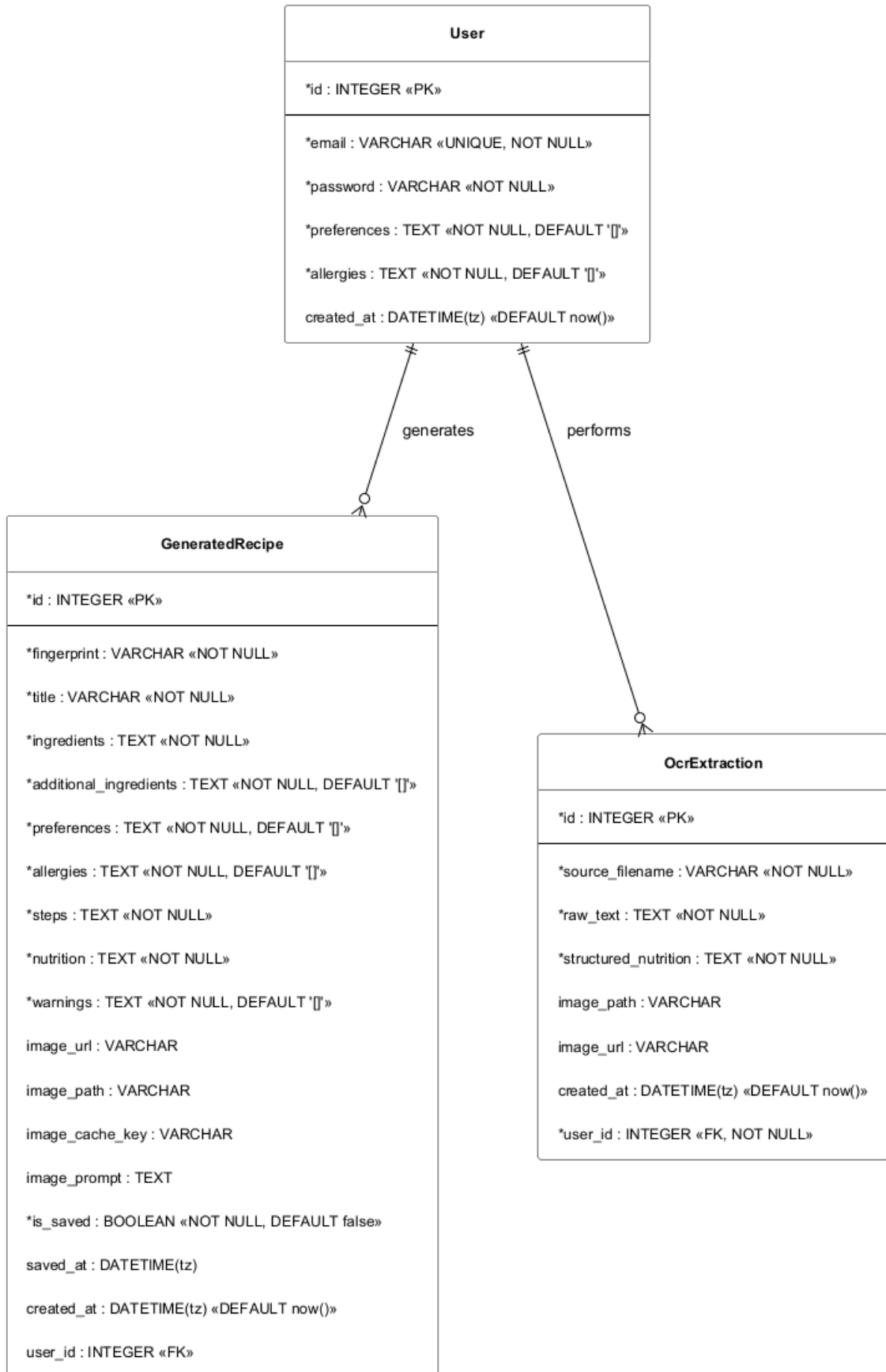


Figure 3.8: Entity-relationship diagram of the AI Cookbook system database, showing all entities, attributes, primary keys, foreign keys, and cardinality constraints

Derived relational schema

From the ER model, the relational schema is derived as shown in Figure 3.9. Every table is presented with its complete column set, data types, key markers, integrity constraints, and index definitions. The diagram makes the transformation from conceptual entities to implementable tables explicit, which is important because the application uses ORM classes but still relies on a concrete relational design underneath.

The transformation from conceptual model to relational model is direct. The `User` entity becomes the `users` table, the `GeneratedRecipe` entity becomes the `generated_recipes` table, and the `OcrExtraction` entity becomes the `ocr_extractions` table. The two one-to-many relationships from `User` are realized by the `user_id` foreign keys in `generated_recipes` and `ocr_extractions`. In other words, the ER cardinality “one user may own many generated recipes” is implemented as many rows in `generated_recipes` referencing one row in `users`, and the same transformation is used for OCR history.

One detail deserves explicit mention. The `generated_recipes.user_id` column is nullable at schema level. This comes from the guest-generation flow, not from a different ownership rule. The application allows guests to generate recipes, but only authenticated users can persist them. Keeping the foreign key nullable makes the schema compatible with that flow, while the application logic still decides when persistence is allowed. By contrast, `ocr_extractions.user_id` is non-null because OCR history entries are created only through an authenticated save action.

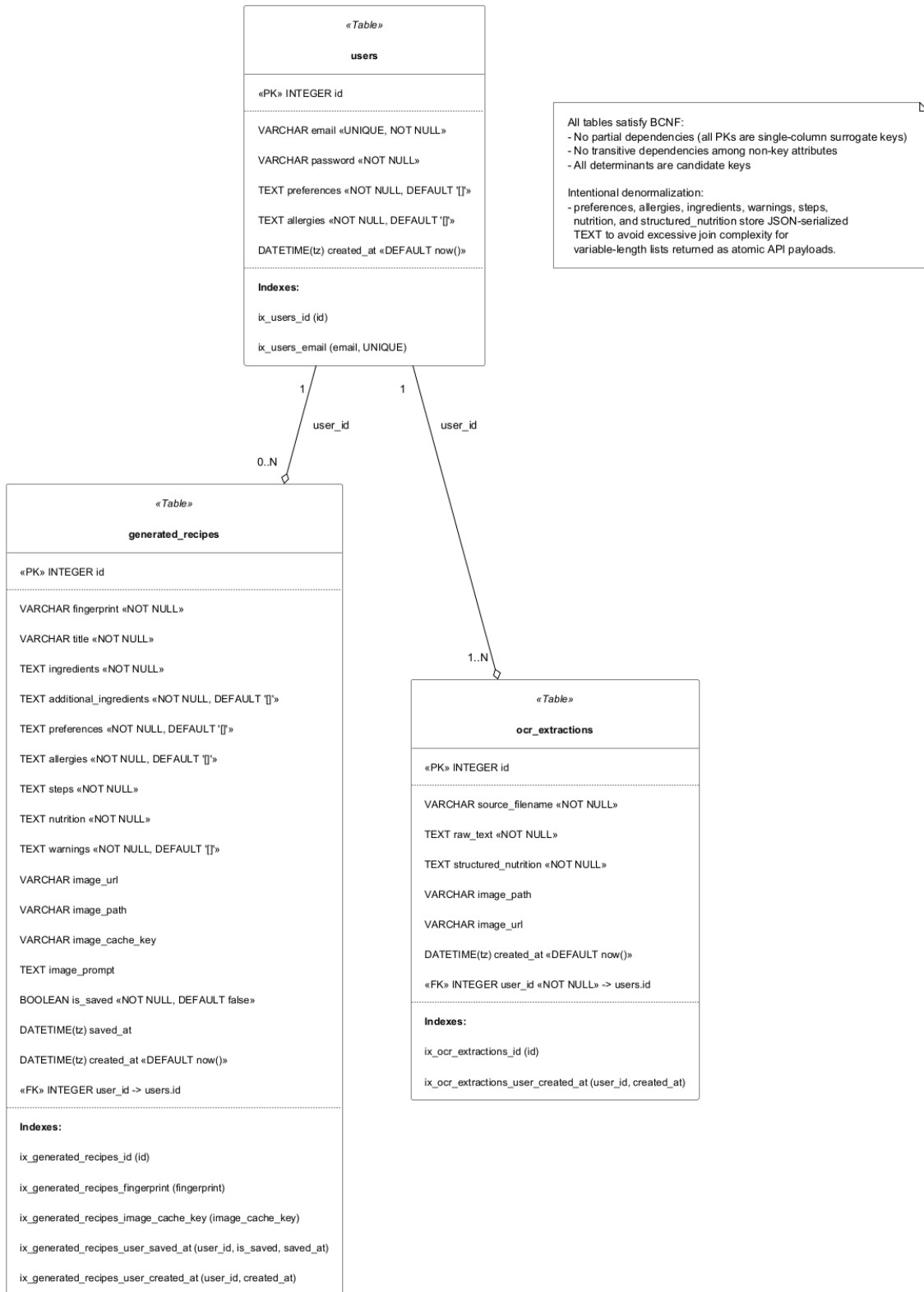


Figure 3.9: Relational schema derived from the ER model, showing all tables with primary keys, foreign keys, constraints, indexes, and inter-table relationships

Normalization analysis

The database schema of the AI Cookbook system consists of three relations: `users`, `generated_recipes`, and `ocr_extractions`. Each relation uses a single-column surrogate integer primary key (`id`), which makes partial dependencies structurally impossible and keeps ownership relations easy to express through foreign keys.

At the relational level, the schema satisfies the usual normalization requirements up to BCNF. The tables do not contain repeating groups, their non-key attributes depend on the whole primary key, and there are no transitive dependencies in which one non-key attribute determines another. The strongest project-specific example is `users`: the unique `email` attribute supports authentication lookup, while `password`, `preferences`, `allergies`, and `created_at` all remain directly dependent on the row identity. The same reasoning applies to `generated_recipes` and `ocr_extractions`, where `user_id` expresses ownership but does not determine the remaining descriptive attributes of the row.

The schema does contain intentional denormalization in several `TEXT` columns that store JSON-serialized arrays or objects rather than being decomposed into separate child tables. Specifically, the `preferences` and `allergies` columns in both `users` and `generated_recipes` store serialized JSON arrays of strings. The `ingredients`, `additional_ingredients`, `steps`, `warnings`, and `nutrition` columns in `generated_recipes`, as well as `structured_nutrition` in `ocr_extractions`, similarly store JSON-encoded structured data. A strictly normalized design would decompose each of these into separate child relations. However, this denormalization is a deliberate design choice driven by the application's access patterns: every API endpoint that retrieves a recipe or extraction returns these fields as complete JSON structures in a single response, and the application never filters or aggregates on individual elements within them at SQL level. Decomposing them would introduce extra join cost and mapping complexity for every read while providing little benefit to the actual queries used by the system.

Index strategy and query design

The index design reflects the dominant access paths of the application.

- A unique index on `users.email` supports fast lookup during registration and login.
- An index on `generated_recipes.fingerprint` supports recipe-result traceability.
- An index on `generated_recipes.image_cache_key` supports thumbnail reuse.
- A composite index on `generated_recipes(user_id, is_saved, saved_at)` supports the saved-recipes listing query.
- A composite index on `generated_recipes(user_id, created_at)` supports recipe history pagination.
- A composite index on `ocr_extractions(user_id, created_at)` supports OCR history pagination.

These indexes are justified directly by the route behavior. The principal query patterns of the system are the following:

- **Authentication lookup:** find a user by unique `email` during registration, login, and JWT resolution.
- **Ownership-scoped recipe save:** find one generated recipe by `id` and `user_id` before setting `is_saved` and `saved_at`.
- **Saved-recipe listing:** filter by `user_id` and `is_saved`, then order primarily by `saved_at`.
- **Recipe-history listing:** filter by `user_id` and order by descending `created_at`.
- **OCR-history listing:** filter by `user_id` and order by descending `created_at`.

Saved recipes are filtered by owner and save state, then ordered by `saved_at` and `created_at`. Recipe history and OCR history are filtered by owner and ordered by descending creation time. The indexes were chosen from these concrete queries, not added in advance without a matching access pattern.

ORM mapping rationale

The relational model is reflected in the backend through ORM entity classes. This was preferred over hand-written SQL-only access because the application repeatedly converts between request schemas, database objects, and response schemas. Using ORM-mapped entities keeps the persistent structure close to the application domain and reduces duplication between route logic and storage definitions. The design still remains relationally explicit because the ER diagram, relational schema, and index strategy are documented independently of the ORM syntax.

Vector-store design

The retrieval subsystem stores recipe documents in a persistent vector collection. Each stored recipe has two design layers:

- **Document text**, which is embedded and used for semantic similarity search.
- **Structured metadata**, which stores title, ingredients, tags, steps, summary, and source information used after retrieval.

This matters because retrieval is not the final result shown to the user. The vector store supplies context for later generation. Once a candidate recipe has been retrieved, its metadata becomes just as important as the similarity score, because filtering, ranking, and prompt formatting all depend on those fields.

Media-storage design

The media layer stores OCR uploads and recipe thumbnails under backend-managed directories and exposes them through a static route. This design is intentionally simple because the project focuses on local deployment and demonstrable workflows. A file-based approach is enough here because images support the main features, but they are not queried as central domain objects.

3.1.4 UI/UX Design

Low-fidelity wireframes

The wireframes in Figures 3.10 and 3.11 capture the intended screen composition before the final interface was polished. Their role is to show page zoning, information

hierarchy, and the separation of the main user tasks, not the final visual details of the implemented interface. This is especially important in this project because recipe generation, OCR verification, persistence, and account management are distinct workflows with different information needs on screen.

These wireframes already show the three UI decisions that most strongly influenced the implementation. First, the Generate page reserves space for dietary constraints near the ingredient input because those constraints directly shape the generation workflow and warning logic. Second, recipe warnings deserve a visible zone in the result card because they communicate safety-relevant information rather than merely cosmetic status messages. Third, the Food Labels page shows raw OCR text and the structured nutrition result within the same task view so that the user can verify how the extraction was interpreted. Finally, save actions are attached to the result views rather than hidden in secondary menus, because persistence is a continuation of the primary task rather than a separate administrative action.

Navigation structure

The user interface is organized around a small number of stable routes. This reduces navigation complexity and makes the main features visible from the top-level navigation bar.

- Home page for overview and entry.
- Generate page for recipe-generation tasks.
- Food Labels page for OCR extraction tasks.
- Saved page for bookmarked recipes.
- History page for all generated recipes.
- Login / Account page for authentication and default settings.

The navigation structure is intentionally task-oriented. Generation, OCR, persistence, and profile management each have a dedicated visible location instead of being merged into a single crowded interface.

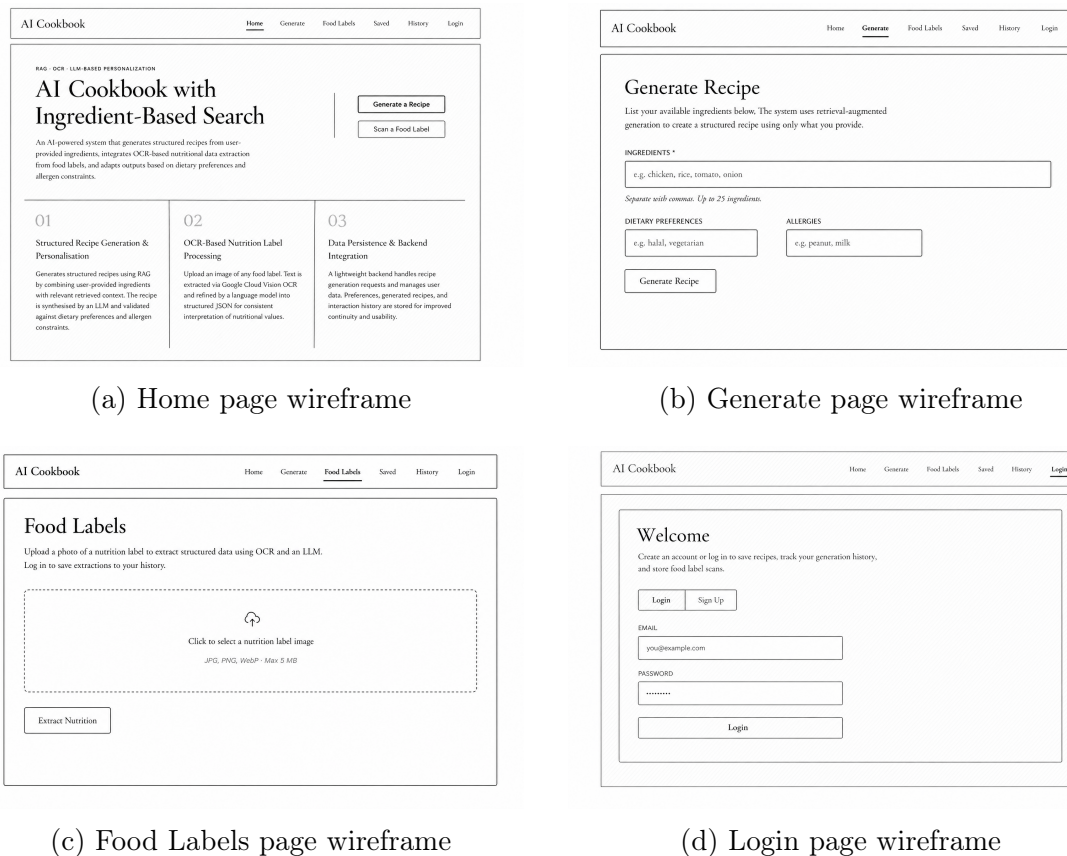
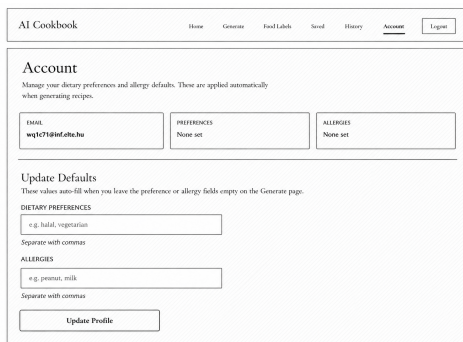
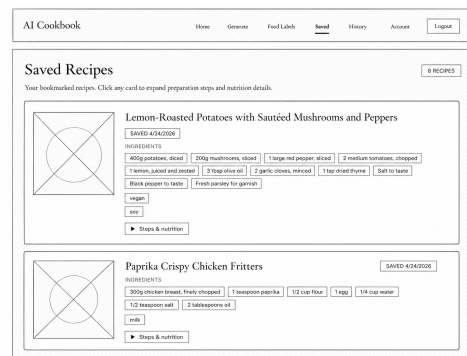


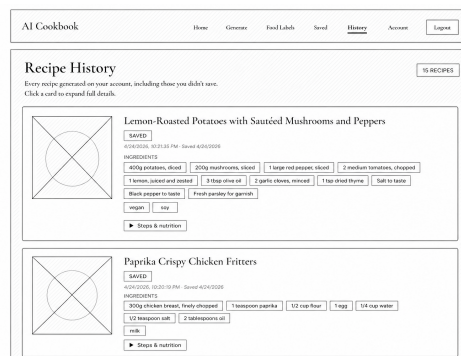
Figure 3.10: Primary design-stage wireframes of the main entry and task-oriented pages



(a) Account page wireframe



(b) Saved recipes page wireframe



(c) History page wireframe

Figure 3.11: Secondary design-stage wireframes for persistence and account-related pages

Screen-flow design

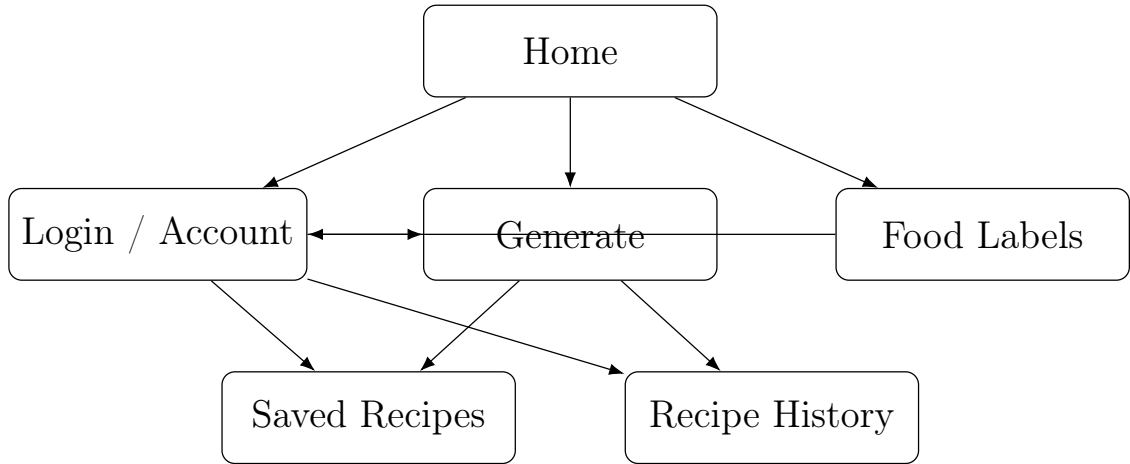


Figure 3.12: Navigation and screen-transition map of the main frontend pages

Interaction design decisions

Three interaction decisions are especially important at design level.

First, recipe-generation warnings are shown prominently in the result view because they communicate safety-related information about excluded ingredients, dietary conflicts, and missing thumbnails.

Second, the OCR page shows both the raw extracted text and the structured nutrition result. This makes the transformation visible to the user and supports trust, debugging, and manual verification.

Third, saved recipes and full recipe history are separated into different pages because they represent different user intentions. Saved recipes act as a curated personal collection, while history acts as a chronological activity log.

3.1.5 Design alternatives considered

Several alternative designs were considered, but the final choices were shaped by the scope of the thesis and by what needed to be implemented and evaluated clearly.

The first alternative was to use a single-prompt language-model approach for recipe generation without retrieval grounding. That design would have been simpler to implement, but it would also have made the generation process less controllable and less tied to an explicit recipe knowledge base. Retrieval-augmented generation was chosen because it creates a stronger connection between user input, reusable

corpus content, and the final prompt while still remaining manageable within the thesis scope.

The second alternative was to process nutrition labels through one end-to-end image-to-JSON step. The chosen design instead separates OCR from semantic structuring. This produces a clearer workflow, supports debugging through raw OCR text, and makes failure causes easier to locate when the image-processing and interpretation stages behave differently.

The third alternative was to use fully cloud-oriented persistence for the relational database, vector store, and media assets. The project instead keeps SQLite, Chroma persistence, and local media directories. This local-storage design is appropriate for a thesis because it keeps deployment reproducible, reduces infrastructure overhead, and still demonstrates the architectural responsibilities clearly.

The fourth alternative was to require authentication for every feature. The chosen guest-plus-authenticated model supports easier exploration of the main AI-supported workflows while still reserving personal history and profile persistence for authenticated users. This makes the system easier to demonstrate and test without weakening the ownership model for private data.

3.1.6 Section Summary

The design chapter establishes the AI Cookbook as a modular AI-assisted web application with explicit requirements, a layered architecture, and a mixed persistence strategy. The use-case, component, class, sequence, state-transition, data-flow, ER, relational schema, navigation, and wireframe figures document the system from complementary viewpoints instead of relying on a single diagram type.

The chapter also makes the project-specific rationale explicit: recipe generation is implemented as a constrained RAG pipeline rather than as a single prompt call, OCR is split into optical extraction and semantic structuring, relational persistence is kept intentionally compact, and the user interface is organized around a small set of dedicated task pages. This design forms the basis for the implementation chapter, where these abstractions are mapped to the final modules and code.

3.2 Implementation

This section shows how the design from Section 3.1 appears in the codebase. It connects the class, sequence, component, and data design artifacts to the implemented backend and frontend modules so that the design decisions can be checked against the actual source code.

3.2.1 Development environment and technology stack

The project is implemented as a monorepo with separate backend and frontend runtimes. The backend uses Python 3 and FastAPI 0.133.1 [1, 4], SQLAlchemy 2.0.48 and Alembic 1.7.7 [5, 6], ChromaDB 1.5.5, OpenAI 2.26.0, and Google Cloud Vision 3.11.0 [7, 8, 9], and Pillow 12.0.0, Pydantic 2.12.5, Pydantic Settings 2.12.5, and Uvicorn 0.41.0 [10, 11, 12, 13]. The frontend uses React 19.2.0, React Router DOM 7.13.1, and Vite 7.3.1 [14, 15, 16]. These exact versions are defined in `backend/requirements.txt` and `frontend/package.json`, which makes the implementation reproducible instead of describing only a generic stack.

The development environment was intentionally lightweight and local. Source code was edited primarily in Visual Studio Code [17], while Git [18] was used for version control across both frontend and backend changes. The backend runtime was managed through a Python virtual environment and the dependency manifest in `backend/requirements.txt`. The frontend runtime was managed through Node.js and npm [19, 20] and the dependency manifests `frontend/package.json` and `frontend/package-lock.json`. Database schema evolution was handled through Alembic migration files.

The technology choices follow the layered architecture from Section 3.2. FastAPI supports thin route modules with built-in request validation. SQLAlchemy and Alembic realize the relational model in code. React and Vite support the page-oriented frontend structure, and ChromaDB makes retrieval part of the working system instead of a design idea described only on paper.

3.2.2 Repository structure and code organization

The repository structure makes the architectural decomposition visible before reading individual modules:

```
1 ai-cookbook-thesis/
2 |-- backend/
3 |   |-- main.py
4 |   |-- config.py
5 |   |-- database.py
6 |   |-- dependencies.py
7 |   |-- models.py
8 |   |-- schemas.py
9 |   |-- security.py
10 |   |-- rate_limit.py
11 |   |-- routes/
12 |   |   |-- auth.py
13 |   |   |-- ocr.py
14 |   |   |-- rag_debug.py
15 |   |   '-- recipes.py
16 |   |-- services/
17 |   |   |-- embedding_service.py
18 |   |   |-- llm_service.py
19 |   |   |-- ocr_image_processing.py
20 |   |   |-- ocr_provider.py
21 |   |   |-- ocr_service.py
22 |   |   |-- recipe_image_provider.py
23 |   |   |-- recipe_image_service.py
24 |   |   |-- rag_service.py
25 |   |   '-- vector_store_service.py
26 |   |-- scripts/
27 |   |   |-- prepare_recipes.py
28 |   |   '-- index_recipes.py
29 |   |-- tests/
30 |   '-- requirements.txt
31 |-- frontend/
32 |   |-- package.json
33 |   '-- src/
34 |       |-- App.jsx
35 |       |-- context/AuthContext.jsx
36 |       |-- lib/api.js
37 |       |-- components/NutritionGrid.jsx
38 |       '-- pages/
39 |           |-- GenerateRecipePage.jsx
40 |           |-- NutritionPage.jsx
41 |           |-- SavedRecipesPage.jsx
```

```

42 |         |-- RecipeHistoryPage.jsx
43 |         '-- LoginPage.jsx
44 '-- alembic/
45     |-- env.py
46     '-- versions/

```

Code 3.1: Repository structure of the implemented system

Table 3.8 summarizes the purpose of the main directories and shows how the repository structure reflects the implemented architecture.

Directory or area	Purpose in the project
backend/	Contains the server-side application, including routing, validation, business logic, persistence, security, and integration with retrieval and external AI services.
backend/routes/	Defines the HTTP entry points for authentication, recipe generation, recipe persistence, OCR extraction, and debugging-oriented retrieval access.
backend/services/	Holds the multi-step workflow logic for recipe generation, retrieval, OCR, image handling, and provider abstraction so that route handlers remain thin.
backend/scripts/	Contains offline preparation utilities used to clean the recipe corpus and build the persistent retrieval index.
backend/tests/	Stores automated backend tests for routes, validation, security behavior, and service-level logic.
frontend/	Contains the browser-facing application, its dependency manifest, and the React source used for the main user workflows.
frontend/src/pages/	Implements the top-level task pages such as recipe generation, OCR extraction, saved recipes, history, and login/account management.
frontend/src/components/	Stores reusable presentation elements shared across pages, such as structured result display components.
frontend/src/context/	Maintains shared frontend state, most importantly authentication state and current-profile information.
frontend/src/lib/	Groups client-side utility code for API access and URL construction so that request logic is not duplicated across pages.
alembic/ and alembic/versions/	Store migration configuration and revision history for relational schema changes, keeping database evolution explicit and reproducible.

Table 3.8: Purpose of the main repository directories

This file system mirrors the layered design from Section 3.1. Route modules act as a thin API boundary, service modules contain business logic and provider orchestration, schema classes define validated data contracts, and ORM models implement persistence. This is also the point where the service-orchestration boundary described in the design chapter becomes visible: `routes/recipes.py` exposes one endpoint to the client, while multiple service modules collaborate behind that boundary.

Development workflow and repository discipline

The development workflow was organized around small, feature-oriented changes rather than large monolithic rewrites. Backend services, API routes, migrations,

frontend pages, and tests were versioned together in the same repository so that every non-trivial feature could be traced as one coherent change set. In practice, this was especially important for features that crossed multiple layers, such as profile-default-aware recipe generation or OCR history persistence, because they required coordinated updates in route logic, schemas, frontend state handling, and tests.

Version-control work followed the same feature-oriented structure. Changes were developed as focused units around one task or one related set of tasks instead of mixing unrelated edits into the same revision. For example, a persistence-related change typically combined the route logic, schema updates, frontend integration, and migration file needed for that feature. This kept the history easier to review and reduced the risk of partial changes that would compile but leave one layer out of sync with the others.

Dependency and environment management were also repository-driven. The backend dependency set is declared in `backend/requirements.txt`, frontend dependencies in `frontend/package.json`, and environment-dependent settings are centralized through the backend configuration layer and the `.env` file format. Database evolution is versioned explicitly through Alembic revision files instead of implicit schema changes. This workflow keeps the implementation reproducible: a given repository state contains not only application code, but also the dependency manifests, migration history, and indexing scripts needed to reconstruct the working system.

3.2.3 Core application logic and complex features

This section focuses on the implementation of the main complex features promised in the project proposal. The first two are the structured recipe-generation workflow and the OCR-based nutrition-label workflow. The third, data persistence and backend integration, is covered in detail later in this section because it spans relational schema design, ORM mapping, migration history, and query behavior.

Application bootstrap and request entry points

The backend entry point is `backend/main.py`. It does more than create a FastAPI object: it prepares the runtime environment, installs cross-cutting middle-

ware, mounts media storage, and registers feature routers [4]. This directly realizes the component diagram from Figure 3.2.

```
1 app = FastAPI(title="AI Cookbook API")
2 Path("backend/media").mkdir(parents=True, exist_ok=True)
3 Path(settings.recipe_thumbnail_directory).mkdir(parents=True,
4     exist_ok=True)
5 Path(settings.ocr_upload_directory).mkdir(parents=True, exist_ok=
6     True)
7
8 app.add_middleware(
9     CORSMiddleware,
10    allow_origins=[settings.frontend_origin],
11    allow_credentials=True,
12    allow_methods=["*"],
13    allow_headers=["*"],
14 )
15
16 app.add_middleware(
17    RateLimitMiddleware,
18    default_rule=RateLimitRule(
19        max_requests=settings.default_rate_limit_requests,
20        window_seconds=settings.default_rate_limit_window_seconds,
21    ),
22    route_rules={
23        "/signup": RateLimitRule(...),
24        "/generate-recipe": RateLimitRule(...),
25        "/ocr/extract": RateLimitRule(...),
26    },
27 )
28
29 app.mount("/media", StaticFiles(directory="backend/media"), name="
30     media")
31
32 app.include_router(auth.router)
33 app.include_router(recipes.router)
34 app.include_router(ocr.router)
35 app.include_router(rag_debug.router)
```

Code 3.2: FastAPI bootstrap in backend/main.py

This bootstrap code mainly provides the shared runtime context for the feature-specific workflows discussed next. It registers the route groups and installs the middleware and media handling that those workflows rely on.

User-specific defaults and authenticated request context

Authentication is implemented in three modules that work together: `backend/security.py` contains Argon2-backed password hashing and JWT helpers [21, 22, 23], `backend/dependencies.py` resolves the current user from the token, and `backend/routes/auth.py` exposes the HTTP endpoints. The hashing and verification calls are implemented as follows:

```
1 password_hash = PasswordHash.recommended()
2
3 def hash_password(password: str) -> str:
4     return password_hash.hash(password)
5
6 def verify_password(plain_password: str, hashed_password: str) ->
    bool:
7     return password_hash.verify(plain_password, hashed_password)
```

Code 3.3: Password hashing utilities in `backend/security.py`

The dependency that enforces ownership of user-specific data is defined in `backend/dependencies.py`:

```
1 oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/token")
2
3 def get_current_user(
4     token: str = Depends(oauth2_scheme),
5     db: Session = Depends(get_db),
6 ) -> User:
7     user = _get_user_from_token(token, db)
8     if user is None:
9         raise credentials_exception
10    return user
```

Code 3.4: JWT-based current-user dependency

This function is the implementation-level counterpart of the ownership relations in the use-case and class diagrams. Endpoints for saving recipes, browsing history, and saving OCR results depend on the resolved `User` instance instead of trusting client-supplied identifiers.

Profile defaults are persisted in `backend/routes/auth.py` and later reused by the frontend when generation fields are left empty:

```
1 @router.put("/me", response_model=UserResponse)
2 def update_current_user_profile(...):
3     current_user.preferences = json.dumps(profile.preferences)
4     current_user.allergies = json.dumps(profile.allergies)
5     db.add(current_user)
6     db.commit()
7     db.refresh(current_user)
8     return _user_to_response(current_user)
```

Code 3.5: Profile update persistence in `backend/routes/auth.py`

Structured recipe generation and personalization

Recipe generation is the most complex workflow in the system. It combines request validation, safety filtering, retrieval, prompt construction, output verification, optional image generation, and persistence. The route in `backend/routes/recipes.py` acts as a thin facade: it delegates generation to the language-model service and thumbnail generation to the image service, then optionally persists the result for the authenticated user.

Personalization enters the workflow at more than one point. User preferences and allergies can arrive directly in the request or through stored profile defaults, and they influence both the initial filtering step and the later prompt construction.

The first non-trivial step is conflict detection. The service checks whether user-provided ingredients violate allergy or preference constraints before retrieval or generation starts:

```
1 allergy_conflicts = find_conflicting_ingredients(
2     request.ingredients,
3     request.allergies,
4 )
5 preference_conflicts = find_preference_conflicting_ingredients(
6     request.ingredients,
7     request.preferences,
8 )
9 safe_ingredients = _remove_conflicting_ingredients(
10     request.ingredients,
11     conflicting_ingredients,
12 )
13
```

```
14 if not safe_ingredients:
15     raise HTTPException(
16         status_code=status.HTTP_400_BAD_REQUEST,
17         detail="All provided ingredients conflict with the selected
18             allergies or dietary preferences.",
19     )
```

Code 3.6: Conflict analysis in `backend/services/llm_service.py`

This safety-oriented preprocessing is the concrete implementation of the restriction logic discussed in Section 3.1. It is also one of the points where the sequence diagram can be verified against real code.

After filtering, the backend retrieves grounded context from the Chroma collection [7]. The retrieval call is implemented in `backend/services/rag_service.py`, and the query vector is created through the embeddings service [24]:

```
1 collection = get_collection()
2 query_text = build_retrieval_query(request)
3 query_embedding = create_embeddings([query_text])[0]
4 response = collection.query(
5     query_embeddings=[query_embedding],
6     n_results=settings.rag_candidate_count,
7     include=["metadatas", "distances"],
8 )
```

Code 3.7: Vector retrieval query in `backend/services/rag_service.py`

The prompt sent to the LLM through the OpenAI API [25] is not generic. It is assembled from the filtered ingredient list, user constraints, and retrieved recipe context. A shortened version of the prompt builder shows this clearly:

```
1 return f"""
2 You are generating a new recipe for an AI cookbook application.
3
4 User ingredients: {ingredient_list}.
5 Dietary preferences: {"", ".join(request.preferences) if request.
6     preferences else "none"}.
7 Allergies to avoid: {"", ".join(request.allergies) if request.
8     allergies else "none"}.
9
10 Retrieved recipe context (use only as structural inspiration, do
11     NOT copy ingredients):
```

```
9 {context_block}
10
11 Return only valid JSON with this exact structure:
12 {{
13     "title": "string",
14     "ingredients": ["string", "string"],
15     "preferences": ["string"],
16     "allergies": ["string"],
17     "steps": ["string", "string", "string"],
18     "nutrition_estimate": {{
19         "calories": 0,
20         "protein": "string",
21         "carbs": "string",
22         "fat": "string"
23     }}
24 }}
25 """.strip()
```

Code 3.8: Prompt construction in backend/services/llm_service.py

The implementation does not trust the first model response blindly. It checks for restriction violations and performs one correction cycle when necessary:

```
1 messages: list[dict] = [{"role": "user", "content": prompt}]
2
3 for _ in range(2):
4     response = client.chat.completions.create(
5         model=settings.openai_model,
6         messages=messages,
7         temperature=0.7,
8     )
9     recipe_text = _strip_code_fences(response.choices[0].message.
10                                     content or "")
11     recipe_json = json.loads(recipe_text)
12     last_violations = _find_output_restriction_violations(
13         recipe_json, restrictions)
14
15     if not last_violations:
16         additional = _find_ingredient_drift(recipe_json,
17                                             safe_ingredients)
18         recipe_json["additional_ingredients"] = additional
19         recipe_json["warnings"] = warnings
```

```
17     return RecipeResponse.model_validate(recipe_json)
18
19     violation_note = (
20         "The previous output violated these rules and must be
           corrected: "
21         + "; ".join(last_violations)
22     )
23     messages.append({"role": "assistant", "content": recipe_text})
24     messages.append({"role": "user", "content": violation_note})
```

Code 3.9: Output validation and retry loop in
backend/services/llm_service.py

This retry loop is the strongest implementation evidence that recipe generation is realized as a fixed multi-stage workflow rather than a single opaque model call. Validation, retrieval, prompt construction, output checking, corrective retry, and response shaping appear as explicit stages in code, which is the implementation-level counterpart of the workflow diagrams and design rationale from Section 3.1.

The implementation also includes dedicated RAG inspection endpoints in `backend/routes/rag_debug.py`. These endpoints are not part of the normal browser workflow, but they were useful during development because they expose the same retrieval pipeline that recipe generation uses internally. The `/rag/debug` endpoint returns the generated retrieval query text, the candidates returned by Chroma, the allergy-conflict filtering result, the ranking score assigned to each candidate, and which items were ultimately selected as context. The `/rag/collection-stats` endpoint provides a lighter diagnostic view over the indexed corpus. In practice, these endpoints made it possible to verify that failures or weak recipe outputs came from retrieval quality, filtering, or prompt behavior rather than from a single opaque model response.

OCR-based nutrition label processing

The OCR subsystem is implemented as a two-stage pipeline: optical text recognition followed by semantic structuring into validated nutrition fields. The abstraction boundary for OCR providers is defined in `backend/services/ocr_provider.py` [9]:

```
1 class OcrProvider(ABC):
2     @abstractmethod
3     def extract_text(self, image_bytes: bytes) -> str:
```

```
4         """Extract raw OCR text from an image payload."""
5
6 class GoogleVisionOcrProvider(OcrProvider):
7     def extract_text(self, image_bytes: bytes) -> str:
8         return extract_text_with_google_vision(image_bytes)
9
10 @lru_cache(maxsize=1)
11 def get_ocr_provider() -> OcrProvider:
12     if settings.ocr_provider == "google_vision":
13         return GoogleVisionOcrProvider()
```

Code 3.10: OCR provider strategy in backend/services/ocr_provider.py

This implements the provider-substitution mechanism described in the design chapter. The OCR workflow depends on an application-level interface and can switch concrete providers without changing the route or higher-level service logic.

Before OCR is executed, images are validated and normalized in backend/services/ocr_image_processing.py with Pillow [10]:

```
1 def prepare_image_for_ocr(file_bytes: bytes) -> bytes:
2     image = Image.open(BytesIO(file_bytes))
3     image.load()
4
5     normalized = ImageOps.exif_transpose(image)
6     grayscale = ImageOps.grayscale(normalized)
7     contrast_ready = ImageOps.autocontrast(grayscale)
8     enlarged = contrast_ready.resize(
9         (contrast_ready.width * 2, contrast_ready.height * 2)
10    )
11
12    buffer = BytesIO()
13    enlarged.save(buffer, format="PNG")
14    return buffer.getvalue()
```

Code 3.11: Image preprocessing for OCR

The second OCR stage turns raw text into structured JSON [26]. The LLM prompt shows that the expected schema is explicit and validated later through Pydantic [11]:

```
1 prompt = f"""
2 You are given OCR text from a food nutrition label.
```

```
3 Extract only the nutrition information you can confidently identify
4
5 Return only valid JSON in this exact structure:
6 {{
7     "product_name": "string or null",
8     "serving_size": "string or null",
9     "calories": 0,
10    "protein_g": 0,
11    "carbs_g": 0,
12    "fat_g": 0,
13    "sugar_g": 0,
14    "sodium_mg": 0,
15    "fiber_g": 0
16 }}
17 """.strip()
```

Code 3.12: Nutrition-structuring prompt in backend/services/ocr_service.py

Supporting image generation and caching

Recipe thumbnails are generated only after the textual recipe has already been accepted. The implementation first derives a deterministic cache key from normalized ingredient data:

```
1 def build_recipe_image_cache_key(
2     title: str,
3     ingredients: list[str],
4     preferences: list[str],
5     allergies: list[str],
6 ) -> str:
7     normalized_ingredients = sorted(
8         _normalize_text(item) for item in ingredients if item.strip()
9     )
10    digest_source = ",".join(normalized_ingredients)
11    return hashlib.sha256(digest_source.encode("utf-8")).hexdigest()[:24]
```

Code 3.13: Thumbnail cache key derivation in
backend/services/recipe_image_service.py

The cache lookup and provider delegation are implemented in the same service:

```
1 thumbnail_path = _get_thumbnail_path(cache_key)
2
3 if thumbnail_path.exists():
4     return {
5         "image_cache_key": cache_key,
6         "image_path": str(thumbnail_path.as_posix()),
7         "image_url": _to_public_image_url(cache_key),
8         "image_prompt": prompt,
9     }
10
11 image_bytes = get_recipe_image_provider().generate_png(prompt)
12 thumbnail_path.write_bytes(image_bytes)
```

Code 3.14: Thumbnail reuse and generation logic

The concrete provider for image generation is again represented through an abstract interface and a provider-specific implementation [27, 25]:

```
1 class RecipeImageProvider(ABC):
2     @abstractmethod
3     def generate_png(self, prompt: str) -> bytes:
4         """Generate a PNG image for the given prompt."""
5
6 class OpenAIRecipeImageProvider(RecipeImageProvider):
7     def generate_png(self, prompt: str) -> bytes:
8         response = self._client.images.generate(
9             model=settings.openai_image_model,
10            prompt=prompt,
11            size="1024x1024",
12            quality="low",
13            output_format="png",
14        )
```

Code 3.15: Recipe image provider in
backend/services/recipe_image_provider.py

This code provides supporting infrastructure for the recipe-generation feature. Thumbnail generation is not the primary feature outcome, but it improves the usability of generated results and saved-history views without changing the main textual workflow.

Request protection

Request throttling is implemented in `backend/rate_limit.py` as custom middleware instead of an external gateway dependency. The middleware keeps an in-memory bucket per client and route:

```
1 @dataclass(frozen=True)
2 class RateLimitRule:
3     max_requests: int
4     window_seconds: int
5
6 class RateLimitMiddleware(BaseHTTPMiddleware):
7     async def dispatch(self, request: Request, call_next):
8         rule = self._resolve_rule(request)
9         client_identifier = self._get_client_identifier(request)
10        key = f"{request.method}:{request.url.path}:{
11            client_identifier}"
12
13        allowed, retry_after = self.rate_limiter.check(key, rule)
14
15        if not allowed:
16            return JSONResponse(
17                status_code=429,
18                content={"detail": "Rate limit exceeded. Please try
19                    again later."},
20                headers={"Retry-After": str(retry_after)},
21            )
```

Code 3.16: Rate-limiting middleware structure

The route-specific limits visible earlier in Listing 3.2 show that authentication, recipe generation, and OCR extraction are protected more strictly than ordinary reads. This is a concrete example of a cross-cutting concern being realized in middleware rather than inside every individual route function.

3.2.4 Frontend implementation

Application shell and routing

The frontend is organized as a React single-page application whose top-level router maps directly to the main workflows from the use-case and navigation diagrams. The routes are declared in `frontend/src/App.jsx`, while each major use

case is implemented as its own page component under `frontend/src/pages/`. This keeps the frontend aligned with the screen-flow diagram from Section 3.4 and follows the React and React Router application model [14, 15].

The page organization is intentionally task-oriented. The Home page provides entry points into the two primary AI-supported workflows, Generate and Food Labels. The Login/Account page combines authentication-related interaction with profile-default management, so the user does not have to navigate to a separate settings area. The Saved and History pages then separate two different persistence views: one for explicitly bookmarked recipes and one for the full generation history. This mirrors the frontend route structure visible in `App.jsx` and keeps each page responsible for one coherent user task.

Centralized API access

HTTP communication [28] is centralized in `frontend/src/lib/api.js`. This module provides a single base URL resolver and a reusable `apiFetch()` wrapper that turns non-success responses into JavaScript errors with backend-supplied detail messages when available. This design keeps page components focused on user interaction and local state instead of repeating low-level request boilerplate. It also gives the frontend one consistent place to adapt API-base configuration across development environments.

Authentication state management

Shared authentication state is implemented through the React context in `frontend/src/context/AuthContext.jsx` [14]:

```
1 const AuthContext = createContext(null);
2
3 export function AuthProvider({ children }) {
4   const [token, setToken] = useState(() => localStorage.getItem("
      token"));
5   const [profile, setProfile] = useState(null);
6   const [profileLoading, setProfileLoading] = useState(false);
7
8   useEffect(() => {
9     if (!token) {
10       setProfile(null);
```

```
11     return;
12   }
13
14   const loadProfile = async () => {
15     const data = await apiFetch("/me", {
16       headers: { Authorization: 'Bearer ${token}' },
17     });
18     setProfile(data);
19   };
20
21   loadProfile();
22 }, [token]);
23 }
```

Code 3.17: Authentication context in `frontend/src/context/AuthContext.jsx`

This module is the frontend counterpart of the backend `get_current_user` dependency. The context stores the token, loads the user profile from `/me`, and exposes state that other pages use to decide whether save actions or account-aware defaults are available.

Recipe-generation page

The most interesting frontend behavior is the fallback from manual fields to stored profile defaults. It is implemented directly in `frontend/src/pages/GenerateRecipePage.jsx`:

```
1 const manualPreferences = preferences.split(",").map((i) => i.trim()
   ().filter(Boolean);
2 const manualAllergies = allergies.split(",").map((i) => i.trim()).
   filter(Boolean);
3
4 const payload = {
5   ingredients: parsedIngredients,
6   preferences: manualPreferences.length > 0 ? manualPreferences :
     profile?.preferences || [],
7   allergies: manualAllergies.length > 0 ? manualAllergies : profile
     ?.allergies || [],
8 };
```

Code 3.18: Profile-default fallback on the generation page

This short block is an important implementation detail because it shows how profile management and recipe generation are connected across the frontend and backend. The feature was discussed in the design chapter, but here it becomes visible as a concrete data-flow decision.

The same page also shows how the generated recipe identifier is used to persist a bookmarked recipe only after successful generation:

```
1 await apiFetch("/recipes", {
2   method: "POST",
3   headers: {
4     "Content-Type": "application/json",
5     Authorization: `Bearer ${token}`,
6   },
7   body: JSON.stringify({ generated_recipe_id: recipe.
8     generated_recipe_id }),
9 });
```

Code 3.19: Saving a generated recipe from the frontend

The same page also illustrates the frontend state pattern used throughout the application: input state is kept locally, asynchronous actions toggle explicit loading flags, and result or error state is rendered conditionally. Recipe generation, OCR extraction, profile updates, and history loading all follow this same approach, which keeps page behavior easy to reason about without introducing a heavier global state solution.

OCR and history-oriented pages

The Food Labels page combines upload interaction, OCR result rendering, save behavior, and paginated history loading in a single task-oriented view. After the user selects an image, the page constructs a `FormData` payload for `/ocr/extract`, renders the raw OCR text together with the structured nutrition result, and conditionally enables saving when the user is authenticated. The page also loads saved OCR history through page-number and page-size query parameters, which makes the backend pagination contract visible in the frontend.

The Saved Recipes page and Recipe History page follow the same data-access pattern. Both request paginated backend data, maintain local `loading`, `error`, `page`, and `totalPages` state, and render pagination controls only when more than one page

exists. The difference is semantic rather than structural: the Saved page filters for explicitly bookmarked recipes, while the History page displays all generated recipe entries, including unsaved ones and their warnings. This close structural similarity is useful because it keeps persistence-related pages consistent for the user while still preserving their different purposes.

Structured result presentation

Structured nutritional output is rendered through the reusable `frontend/src/components/NutritionGrid.jsx` component. The component accepts a nutrition object and an optional field list, which allows the same grid to display the compact recipe nutrition estimate on the Generate, Saved, and History pages, and the richer OCR nutrition schema on the Food Labels page. This is a small but meaningful frontend abstraction: presentation logic for structured nutrition data is kept in one component instead of being duplicated across multiple pages.

3.2.5 Persistence implementation

Relational schema in executable form

The relational design from Section 3.1 is not only diagrammed; it is materialized through ORM mappings and migration files. The resulting tables remain small in number but rich enough to capture ownership, save-state transitions, timestamps, and the denormalized structured payloads returned by the application. The following DDL-style excerpt shows the schema shape that the ORM and Alembic revisions realize [5, 6]:

```
1 CREATE TABLE users (  
2     id INTEGER PRIMARY KEY,  
3     email VARCHAR NOT NULL UNIQUE,  
4     password VARCHAR NOT NULL,  
5     preferences TEXT NOT NULL DEFAULT '[]',  
6     allergies TEXT NOT NULL DEFAULT '[]',  
7     created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
8 );  
9  
10 CREATE TABLE generated_recipes (  
11     id INTEGER PRIMARY KEY,
```

```
12     fingerprint VARCHAR NOT NULL,
13     title VARCHAR NOT NULL,
14     ingredients TEXT NOT NULL,
15     additional_ingredients TEXT NOT NULL DEFAULT '[]',
16     preferences TEXT NOT NULL DEFAULT '[]',
17     allergies TEXT NOT NULL DEFAULT '[]',
18     steps TEXT NOT NULL,
19     nutrition TEXT NOT NULL,
20     warnings TEXT NOT NULL DEFAULT '[]',
21     image_url VARCHAR,
22     image_path VARCHAR,
23     image_cache_key VARCHAR,
24     image_prompt TEXT,
25     is_saved BOOLEAN NOT NULL DEFAULT FALSE,
26     saved_at DATETIME,
27     created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
28     user_id INTEGER REFERENCES users(id)
29 );
30
31 CREATE TABLE ocr_extractions (
32     id INTEGER PRIMARY KEY,
33     source_filename VARCHAR NOT NULL,
34     raw_text TEXT NOT NULL,
35     structured_nutrition TEXT NOT NULL,
36     image_path VARCHAR,
37     image_url VARCHAR,
38     created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
39     user_id INTEGER NOT NULL REFERENCES users(id)
40 );
```

Code 3.20: Representative relational DDL for the implemented schema

This representation is useful because it makes the key decisions visible in one place: the ownership model is implemented through foreign keys, recipe persistence permits a nullable owner because guest generation exists conceptually, and structured variable-length content is stored as serialized text rather than decomposed into many auxiliary tables.

The `fingerprint` field in `generated_recipes` is also an implementation detail with practical value rather than a decorative schema element. In `backend/utils.py`, the application builds this fingerprint by normalizing the recipe title, ingredients,

preferences, allergies, and steps, serializing the normalized payload, and hashing it with SHA-256. This makes the identity of a generated recipe depend on its meaningful content instead of surface formatting differences such as extra whitespace or input order. The result is a stable identifier that can be stored together with the generated row and reused in later comparisons or debugging.

ORM entities

The relational model from Section 3.1 is implemented in `backend/models.py` with SQLAlchemy [5]. The `GeneratedRecipe` entity is the most important example because it combines ownership, save state, thumbnail metadata, and several intentionally serialized fields:

```
1 class GeneratedRecipe(Base):
2     __tablename__ = "generated_recipes"
3
4     id = Column(Integer, primary_key=True, index=True)
5     fingerprint = Column(String, nullable=False, index=True)
6     title = Column(String, nullable=False)
7     ingredients = Column(Text, nullable=False)
8     additional_ingredients = Column(Text, nullable=False, default="
9         []")
10    preferences = Column(Text, nullable=False, default="[]")
11    allergies = Column(Text, nullable=False, default="[]")
12    steps = Column(Text, nullable=False)
13    nutrition = Column(Text, nullable=False)
14    warnings = Column(Text, nullable=False, default="[]")
15    image_url = Column(String, nullable=True)
16    image_path = Column(String, nullable=True)
17    image_cache_key = Column(String, nullable=True, index=True)
18    is_saved = Column(Boolean, nullable=False, default=False,
19        server_default="false")
20    saved_at = Column(DateTime(timezone=True), nullable=True)
21    created_at = Column(DateTime(timezone=True), server_default=
22        func.now())
23    user_id = Column(Integer, ForeignKey("users.id"), nullable=True
24        )
25
26    owner = relationship("User", back_populates="generated_recipes"
27        )
```

Code 3.21: GeneratedRecipe ORM mapping

This mapping closes the loop between the ER diagram, the relational schema, and the implementation. The JSON-serialized text fields are a deliberate denormalization tradeoff already justified in Section 3.3.4: they simplify storage of variable-length ingredient, warning, and step lists without introducing multiple auxiliary tables for a thesis-scale application.

Schema migration history

Schema evolution is managed through Alembic [6]. The migration `a3f9c2d1e804_ocr_image_fields_and_pagination_indexes.py` is a good example because it adds both new columns and composite indexes:

```
1 def upgrade() -> None:
2     op.add_column("ocr_extractions", sa.Column("image_path", sa.
3         String(), nullable=True))
4     op.add_column("ocr_extractions", sa.Column("image_url", sa.
5         String(), nullable=True))
6
7     with op.batch_alter_table("generated_recipes") as batch_op:
8         batch_op.create_index(
9             "ix_generated_recipes_user_saved_at",
10            ["user_id", "is_saved", "saved_at"],
11        )
12         batch_op.create_index(
13             "ix_generated_recipes_user_created_at",
14            ["user_id", "created_at"],
15        )
```

Code 3.22: Alembic revision adding image fields and pagination indexes

This revision demonstrates that the relational design did not remain static after the ERD was drawn. The database schema evolved together with the implementation needs of OCR history and paginated history views.

The same evolution also shows where the implementation became more precise than the first conceptual sketch. The design stabilized around three adjustments in particular. First, recipe history and explicit saved recipes were kept in the same table instead of being split into two separate persistence models, because save-state

is an attribute of a generated result rather than a different domain object. Second, OCR history gained image-path and image-URL fields only once the frontend history view made image recall part of the user-visible workflow. Third, pagination support led directly to composite indexes on `generated_recipes` and `ocr_extractions`, so the final schema reflects not only domain entities but also the access patterns that emerged during implementation.

Query and indexing implementation

The composite indexes above are used by the saved-recipes and history queries in `backend/routes/recipes.py`. The saved-recipes query is shown below:

```
1 base_query = db.query(GeneratedRecipe).filter(  
2     GeneratedRecipe.user_id == current_user.id,  
3     GeneratedRecipe.is_saved.is_(True),  
4 )  
5 rows = (  
6     base_query  
7     .order_by(GeneratedRecipe.saved_at.desc(), GeneratedRecipe.  
8         created_at.desc())  
9     .offset((page - 1) * page_size)  
10    .limit(page_size)  
11    .all()  
12 )
```

Code 3.23: Saved-recipes query using the ownership and save-state fields

This query is not only persistence code but also evidence that the index strategy described in Section 3.1 is connected to an actual access pattern in the system.

The same query can be expressed at SQL level as follows:

```
1 SELECT  
2     id,  
3     title,  
4     ingredients,  
5     additional_ingredients,  
6     preferences,  
7     allergies,  
8     steps,  
9     nutrition,  
10    image_url,
```



```
11     image_cache_key ,
12     saved_at ,
13     created_at
14 FROM generated_recipes
15 WHERE user_id = :current_user_id
16     AND is_saved = TRUE
17 ORDER BY saved_at DESC , created_at DESC
18 LIMIT :page_size OFFSET :offset;
```

Code 3.24: SQL form of the saved-recipes listing query

This query uses the same ownership and save-state conditions that appear in the ORM code and benefits directly from the composite index on (`user_id`, `is_saved`, `saved_at`). The ordering also reflects a domain choice: once a recipe becomes part of the saved collection, the most relevant timestamp is the save action rather than the original generation time.

Although the user-facing application mainly executes ownership-scoped single-table reads, the same relational model also supports more analytical queries when needed. For example, the following aggregation query can summarize per-user recipe activity by combining a join, conditional aggregation, and descending ranking:

```
1 SELECT
2     u.email ,
3     COUNT(gr.id) AS generated_recipe_count ,
4     SUM(CASE WHEN gr.is_saved = TRUE THEN 1 ELSE 0 END) AS
5         saved_recipe_count ,
6     MAX(gr.created_at) AS latest_generation_at
7 FROM users AS u
8 LEFT JOIN generated_recipes AS gr
9     ON gr.user_id = u.id
10 GROUP BY u.id, u.email
11 ORDER BY generated_recipe_count DESC , latest_generation_at DESC;
```

Code 3.25: Analytical SQL over the implemented recipe-history schema

This query is not part of the current browser workflow, but it is still important documentation evidence because it shows that the schema supports not only transactional reads and writes but also higher-level reporting over the same ownership model.

The retrieval corpus is indexed offline through `backend/scripts/index_recipes.py`:

```
1 def index_recipe_corpus() -> None:
2     recipes = load_recipe_corpus(corpus_path)
3     collection = get_collection()
4     existing = collection.get(include=[])
5     existing_ids = existing.get("ids", [])
6     if existing_ids:
7         collection.delete(ids=existing_ids)
8
9     for start in range(0, len(recipes), batch_size):
10        batch = recipes[start : start + batch_size]
11        documents = [build_recipe_document(recipe) for recipe in
12                      batch]
13        embeddings = create_embeddings(documents)
14        collection.add(
15            ids=[str(recipe["recipe_id"]) for recipe in batch],
16            documents=documents,
17            embeddings=embeddings,
18            metadatas=[build_recipe_metadata(recipe) for recipe in
19                      batch],
20        )
```

Code 3.26: Offline recipe-corpus indexing for ChromaDB

The indexed recipe corpus is based on a cleaned subset of the Food.com dataset obtained from Kaggle and stored locally in `backend/data/recipes_subset.json` [29]. This indexing script shows that the retrieval layer is not abstract or purely conceptual. The corpus preparation and embedding workflow [24] are implemented as executable code that can rebuild the persistent Chroma store [7] used during generation.

However, the indexing script is only the second half of the retrieval pipeline. Before any embeddings are created, the raw Food.com export is cleaned by `backend/scripts/prepare_recipes.py`. This script parses the CSV source, decodes list-like fields, normalizes ingredient text, removes duplicates, filters invalid recipes, and builds a short instruction summary for each retained item. The resulting `recipes_subset.json` file is therefore not a raw dataset dump but a project-specific intermediate corpus shaped specifically for later retrieval. This preprocessing

step is important implementation evidence because the quality of retrieval depends not only on the embedding model but also on how consistently the local corpus is cleaned and reduced before it enters Chroma.

The vector-store layer itself is implemented in `backend/services/vector_store_service.py`. That module creates the persistent Chroma collection, converts each recipe into a compact document containing the title, ingredients, tags, and instruction summary, and stores the richer structured fields as metadata alongside the embedding [7]. This means that the retrieval representation used by the project is deliberately asymmetric: the document text is optimized for semantic matching, while metadata keeps the structured recipe fields available for later filtering, ranking, and prompt construction.

3.2.6 Known implementation limitations

Several implementation limitations remain and are important for honest evaluation.

Known bugs and unresolved issues

The first unresolved issue is that the quality of recipe generation and nutrition-label extraction depends partly on upstream AI-provider behavior. Even when the local validation and control flow operate correctly, the application may still receive weak OCR text, malformed model output, or structurally poor generated content from the external services it relies on. In such cases, the backend may reject the response, return a lower-quality result, or surface warnings instead of producing a fully satisfactory output. This is not a control-flow bug in the local application logic, but it remains an unresolved reliability issue because output quality cannot be guaranteed purely by local code.

A second unresolved issue concerns OCR upload persistence. In `backend/services/ocr_service.py`, the system attempts to store the uploaded image before OCR extraction begins, but a failure during that media-write step does not stop the extraction workflow. As a result, OCR processing may still succeed and return structured nutrition data even when the uploaded image itself was not persisted correctly. This behavior keeps the core extraction feature

available in the presence of non-critical storage errors, but it also means that a saved history entry may lack a usable stored-image reference.

A third practical weakness is the limited recovery strategy around AI-generated recipe validity. The recipe workflow validates generated content against restriction rules and performs one corrective retry when the first result violates allergy or dietary constraints. However, if both attempts fail, the request is rejected. This keeps the safety model strict, but it also means that transient or provider-specific output instability can still cause complete request failure even when the input itself is valid.

Technical debt and maintainability concerns

The default relational database is SQLite [30], as configured in `backend/config.py`. This is appropriate for a local thesis-scale deployment, but it is not designed for larger multi-user or high-concurrency scenarios. The same applies to the custom rate limiter in `backend/rate_limit.py`, which stores counters in process memory. Its limits are therefore reset when the backend restarts and are not shared across multiple application instances.

Authentication state on the frontend is currently stored in browser `localStorage`, as implemented in `frontend/src/context/AuthContext.jsx`. This keeps the frontend logic simple, but it is a weaker session-management approach than using refresh tokens, `httpOnly` cookies, or server-managed session renewal. Similarly, account management remains intentionally minimal: the current implementation supports signup, login, logout, and profile-default updates, but it does not include password reset, email verification, or role-based authorization features.

The persistence model also contains an intentional denormalization tradeoff. Several structured values, such as recipe ingredients, steps, warnings, and OCR nutrition payloads, are stored as serialized text rather than being decomposed into separate relational child tables. This keeps the schema compact and matches the JSON-shaped API contracts [26] well, but it reduces query flexibility and would become harder to maintain if the system evolved toward more complex reporting or cross-record analytics.

Media handling is likewise optimized for simplicity rather than for a strong deployment model. OCR upload images and generated recipe thumbnails are stored on the local filesystem under the backend workspace. This is sufficient for local de-

velopment and demonstration, but it would need to be replaced by a more robust object-storage strategy in a larger deployment.

The thumbnail cache also reflects a pragmatic compromise. In `backend/services/recipe_image_service.py`, the cache key is derived mainly from normalized ingredient input. This reduces repeated image-generation calls and API cost, but it can also cause requests with the same base ingredient set to reuse the same thumbnail even when preference or allergy context differs.

The application also remains dependent on external AI services for its main intelligent features. Recipe generation, embedding creation [24], OCR text structuring, and recipe-image generation all require upstream API access and therefore do not currently support a fully offline execution mode. This is acceptable for the current thesis-scale deployment model, but it defines a clear operational boundary for the implementation.

Planned but unimplemented improvements

Several improvements were deliberately left outside the thesis scope. The recipe workflow does not yet support editing and re-saving generated recipes after the initial result is stored. The system does not yet provide editing or deletion for saved recipes or saved OCR history entries. The OCR subsystem exposes a provider abstraction, but only the Google Vision-based implementation [9] is currently available in production form. Likewise, recipe image generation is represented through a provider interface, but only the OpenAI-backed path [27, 25] is implemented.

The current application also lacks more advanced administrative and analytical functionality. For example, there is no dedicated dashboard for aggregate usage statistics, no administrator view over user activity patterns, and no richer moderation or review workflow for generated content. These omissions do not prevent the core thesis goals from being demonstrated, but they remain natural extension points.

Future development directions

Future work should first strengthen operational robustness. The most important infrastructure-oriented improvements would be replacing the in-memory rate limiter

with a shared backing store, migrating from SQLite to a stronger multi-user database when needed, and making media persistence more explicit and failure-aware.

On the AI side, future development should broaden provider support, improve evaluation coverage, and strengthen output control. Useful next steps would include adding alternative OCR backends, expanding the recipe and OCR evaluation datasets, refining retrieval and prompt construction, and introducing more flexible post-generation revision controls for users.

From a product perspective, the most valuable user-facing improvements would be edit/delete support for stored content, stronger account management, and clearer lifecycle handling for generated artifacts and saved OCR media. These directions follow directly from the current implementation and would extend the project without requiring a fundamental redesign.

3.2.7 Section Summary

The design section presented the architecture and design rationale of the project; this implementation section showed their realization in code. The FastAPI bootstrap, route modules, service classes, and React context/page structure demonstrate that the UML and architectural diagrams reflect the actual repository. The persistence layer, Alembic revisions, and Chroma indexing script likewise show that the database and retrieval designs were implemented as executable program components rather than remaining design-only artifacts. This concrete implementation basis is used in the testing section to evaluate functional behavior, robustness, and output quality.

3.3 Testing

This section documents the executed tests for the implemented system. The testing evidence covers software correctness, user-interface behavior, API integration, operational robustness, and the quality boundary of the AI-assisted features. The executed validation includes automated backend tests, automated frontend tests, screenshot-based UI verification, API-level integration checks in Postman, manual workflow tests, reliability-oriented backend tests, and AI-specific evaluation.

3.3.1 Testing Strategy and Methodology

The testing structure follows the architecture of the implemented system. The backend contains route handlers, validation schemas, persistence logic, RAG retrieval, OCR preprocessing, and provider abstractions. The frontend contains page-level workflows for authentication, recipe generation, recipe history, saved recipes, and nutrition-label extraction. The executed tests are therefore grouped into multiple layers with different goals.

- **Unit and functional testing** validates backend route behavior, schema validation, service logic, and frontend rendering behavior under controlled conditions.
- **UI testing** verifies that the main user-facing pages are usable and display the expected workflow states.
- **Integration and end-to-end API testing** verifies that the HTTP layer, authentication flow, persistence layer, and paginated history endpoints work together correctly.
- **Advanced reliability testing** checks whether the backend remains stable under repeated requests and small mocked bursts without measuring uncontrollable external-provider latency.

Deterministic behavior such as authentication and history scoping is validated with automated assertions. User-visible workflow states are documented through rendered pages and screenshots. Integration behavior is validated through HTTP requests in Postman. Reliability checks use mocked provider dependencies so that the observed results reflect local system behavior rather than external service latency.

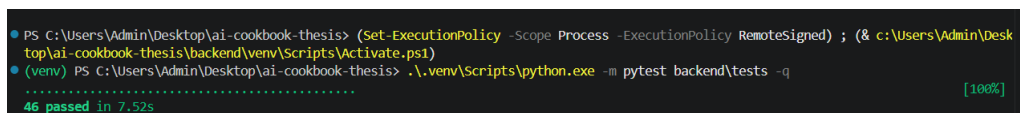
Coverage was used as a supporting indicator together with functional evidence. The backend focus was the public route groups, authentication guards, persistence flows, schema validation, OCR preprocessing, and RAG support logic. The frontend focus was the main pages and their most important guest and authenticated branches.

3.3.2 Unit and Functional Testing

Backend automated test suite

The backend test suite was executed with `pytest` against the FastAPI application and its supporting modules. The tests cover authentication, protected-route behavior, recipe generation endpoints, OCR persistence endpoints, schema validation, service-provider selection, RAG retrieval logic, image preprocessing, rate limiting, and mocked reliability scenarios. The executed suite therefore covers both internal helper logic and the request/response contracts exposed by the backend to the frontend.

The backend suite completed successfully with **46 passing tests**. The terminal output is shown in Figure 3.13.



```
PS C:\Users\Admin\Desktop\ai-cookbook-thesis> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\Admin\Desktop\ai-cookbook-thesis\backend\venv\Scripts\Activate.ps1)
(venv) PS C:\Users\Admin\Desktop\ai-cookbook-thesis> .\venv\Scripts\python.exe -m pytest backend\tests -q
.....
46 passed in 7.52s [100%]
```

Figure 3.13: Successful backend automated test execution with `pytest`

The executed backend tests are concentrated in four areas:

- **Authentication and authorization:** signup, login, token validation, invalid-token rejection, expired-token rejection, and protected-route access control.
- **Persistence and user scoping:** saving generated recipes, listing recipe history, saving OCR extractions, and ensuring that one user cannot see another user's saved content.
- **Validation and preprocessing:** request-schema cleaning, image-type checks, image-size limits, and OCR preprocessing rules.

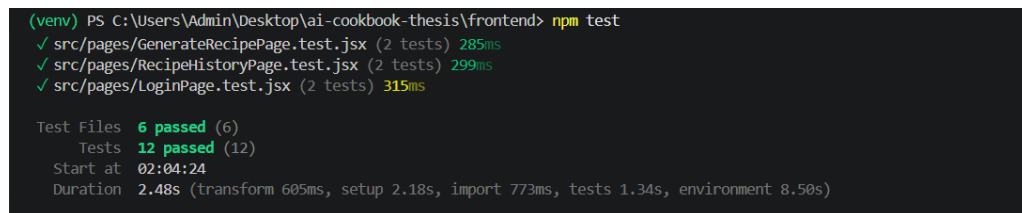
- **Retrieval and orchestration logic:** ingredient conflict detection, preference conflict detection, retrieval query formation, candidate ranking behavior, and provider selection.

The backend evidence covers security, persistence, and intermediate business logic in addition to basic route responses.

Frontend automated functional tests

The frontend test suite was executed with **Vitest** and the **jsdom** environment. These tests validate the behavior of the React pages and components, including form handling, conditional rendering, authenticated page states, and structured result display.

The frontend suite completed successfully with **12 passing tests across 6 test files**, as shown in Figure 3.14. The tested areas include the login page, generate-recipe page, saved-recipes page, recipe-history page, nutrition-label page, and the shared nutrition-grid component.



```
(venv) PS C:\Users\Admin\Desktop\ai-cookbook-thesis\frontend> npm test
✓ src/pages/GenerateRecipePage.test.jsx (2 tests) 285ms
✓ src/pages/RecipeHistoryPage.test.jsx (2 tests) 299ms
✓ src/pages/LoginPage.test.jsx (2 tests) 315ms

Test Files  6 passed (6)
Tests      12 passed (12)
Start at   02:04:24
Duration   2.48s (transform 605ms, setup 2.18s, import 773ms, tests 1.34s, environment 8.50s)
```

Figure 3.14: Successful frontend automated test execution with **Vitest**

The frontend tests verify several behaviors that are visible to the end user:

- validation errors are shown when required fields are missing,
- saved profile defaults are reused correctly during recipe generation,
- login and profile-update requests are sent with the expected payloads,
- protected pages display correct guest-only guidance when the user is not authenticated,
- history entries can be rendered and expanded correctly,
- nutrition data is rendered consistently in both recipe and OCR contexts.

The backend and frontend automated suites provide the main deterministic correctness evidence for the application.

Measured test coverage

In addition to counting passing tests, the executed suites were also measured for implementation coverage. Backend coverage was measured with `coverage.py` over the `pytest` run, while frontend coverage was measured with the `Vitest` V8 coverage provider. Table 3.9 summarizes the resulting indicators.

Layer	Tool	Metric	Result
Backend	<code>coverage.py</code> + <code>pytest</code>	Statement coverage	77% overall (1138 of 1474 statements executed).
Frontend	<code>Vitest</code> V8 coverage	Statement / line coverage	72.98% statements and 74.90% lines.
Frontend	<code>Vitest</code> V8 coverage	Branch / func- tion coverage	55.18% branches and 64.93% functions.

Table 3.9: Coverage indicators measured for the automated test suites

These percentages should be interpreted together with the covered modules. The backend coverage is strongest in the directly relevant application layers: `main.py` reached 100%, route modules such as `auth.py`, `ocr.py`, and `recipes.py` reached 98%, 97%, and 94%, `schemas.py` reached 98%, and `rag_service.py` reached 91%. The lower backend totals are mainly caused by thin provider-adapter and externally dependent modules such as `llm_service.py`, `ocr_google_vision_provider.py`, `recipe_image_service.py`, and `vector_store_service.py`, which are harder to execute comprehensively in a deterministic local test environment without turning the thesis suite into a network- dependent integration harness.

The frontend coverage similarly reflects the actual design of the application. The most important page-level flows are exercised, but some secondary branches, especially inside the saved-recipes page, remain less covered because the current suite prioritizes core navigation, validation, authenticated state handling, and result rendering over exhaustive UI edge-case permutation.

Representative executed test cases

Table 3.10 presents a representative selection of executed test cases, including the test steps, expected result, and observed result.

Layer	Case	Test Steps	Expected Result	Observed Result
Frontend automated	Empty recipe submission validation	Open the Generate page, leave the ingredient field empty, and press Generate Recipe .	A validation error is shown and no API request is sent.	The executed Vitest case rendered the message “Please enter at least one ingredient” and confirmed that the mocked API function was not called.
Backend automated	Protected profile access without token	Send GET /me without an authorization header.	The endpoint rejects the request with 401 Unauthorized .	The executed pytest case returned status code 401, confirming correct protected-route enforcement.
Backend automated	Authenticated recipe generation persists history	Generate a recipe as an authenticated user, then request /recipes/history .	The generated recipe is stored and later visible in history with the expected metadata.	The executed backend test verified one history item with the generated identifier, the expected title, and unsaved state metadata.
Frontend automated	Saved defaults reused during generation	Render the Generate page for a logged-in user with stored preferences and allergies, enter only ingredients, and submit.	The request payload automatically includes the saved defaults and the page renders a structured recipe response.	The executed Vitest case confirmed both the outgoing payload and the displayed recipe result.
Manual scenario	Successful login and account view	Open the Login page, submit valid credentials, and navigate to the Account page.	Authentication succeeds and profile information is displayed.	The executed manual run displayed the account page together with the saved dietary-preference and allergy sections.
Manual + OCR evaluation	OCR extraction on a clear nutrition label	Upload nutrition-facts.jpg and execute extraction on the Food Labels page.	Structured nutrition values and raw OCR text are returned without page failure.	The executed case returned calories 257, protein 8g, carbohydrates 39.8g, fat 9.4g, sugar 2.1g, sodium 41mg, fiber 10g, and readable raw OCR output.

Table 3.10: Representative executed test cases with steps, expected result, and observed result

3.3.3 UI Testing

Purpose of screenshot-based UI verification

Automated frontend tests confirm logic and rendering decisions, but visual verification was also documented for the main user-facing workflows. The application contains multiple screens where the arrangement of controls, messages, cards, and structured outputs is part of the observed result. Screenshot-based UI verification therefore documents the implemented pages and the expected sequence of user actions.

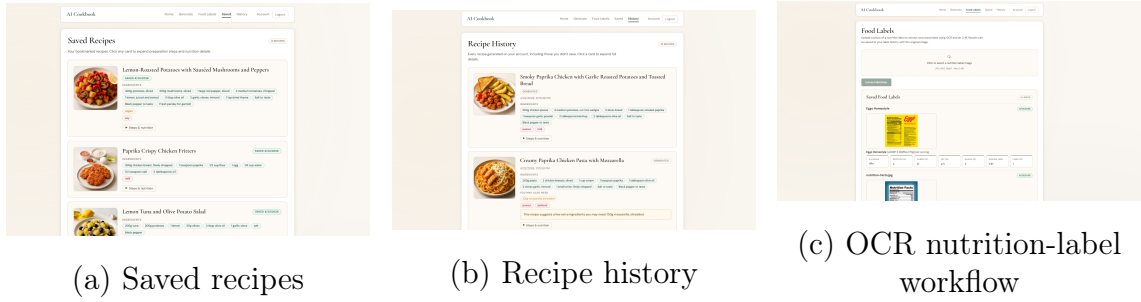


Figure 3.17: Persistence and OCR-related UI screens

These UI screenshots document three observed properties of the frontend: the implemented workflows are present, guest and authenticated states are separated clearly, and the outputs of recipe generation and OCR extraction are presented in a readable task-oriented format.

3.3.4 Integration and End-to-End Testing

Postman as API-integration evidence

To validate the interaction between the HTTP layer, authentication, backend persistence, and feature-specific routes, the project uses a dedicated Postman collection. This collection provides request-level evidence that the implemented API can be exercised outside the frontend and behaves correctly through direct HTTP calls.

The collection itself is shown in Figure 3.18. It documents that the API checks were organized as a repeatable test set rather than as isolated ad hoc requests.

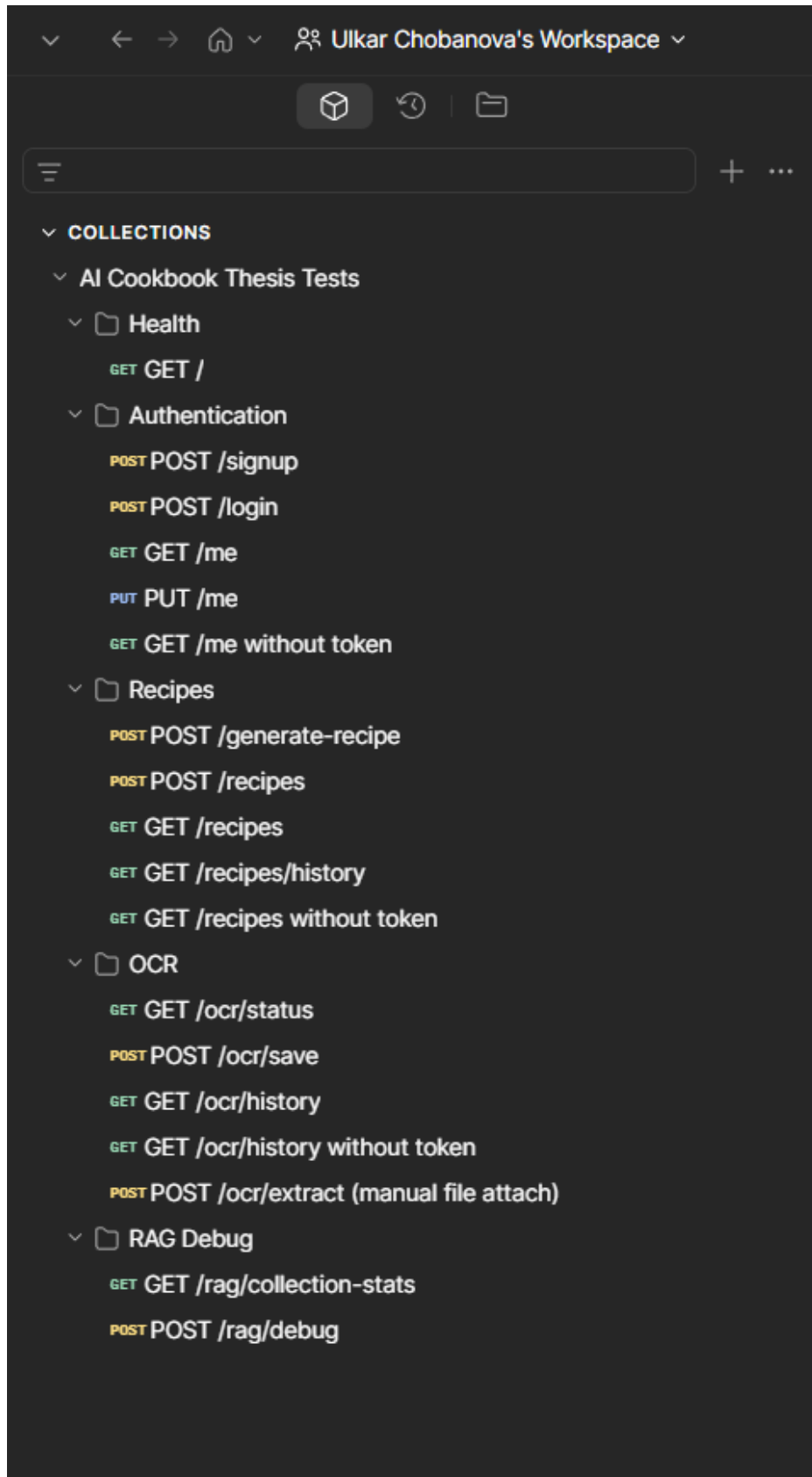
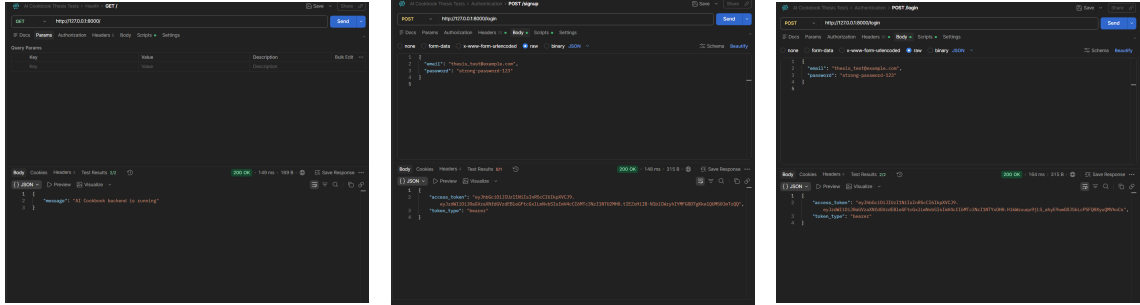


Figure 3.18: Postman collection used for API-level integration testing

The Postman checks were executed in a sequence that follows the real system

lifecycle: health verification, account creation, login, authenticated profile access, main feature calls, persistence actions, and history retrieval. Figure 3.19 shows the initial health and authentication requests.



(a) Health check

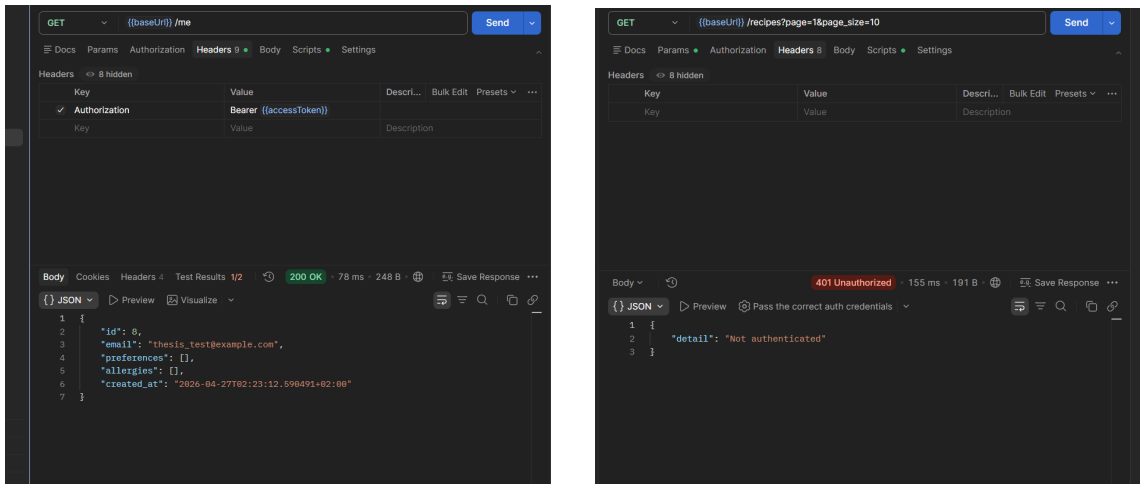
(b) Signup request

(c) Login request

Figure 3.19: Postman evidence for backend availability and authentication

The health-check response confirms that the backend is running and reachable. The signup and login responses confirm that account creation and token issuance work correctly through the public API boundary.

Figure 3.20 shows the profile request and an unauthorized access attempt. This pair documents both successful authenticated access and rejection of unauthenticated access to protected resources.



(a) Authenticated /me request

(b) Unauthorized protected request

Figure 3.20: Postman evidence for protected-route behavior

The main recipe-generation integration path is shown in Figure 3.21. The first request demonstrates successful structured recipe generation through the API. The second request demonstrates that a generated recipe can then be saved through a

separate persistence endpoint. The third request demonstrates that the saved or generated item later appears in recipe history.

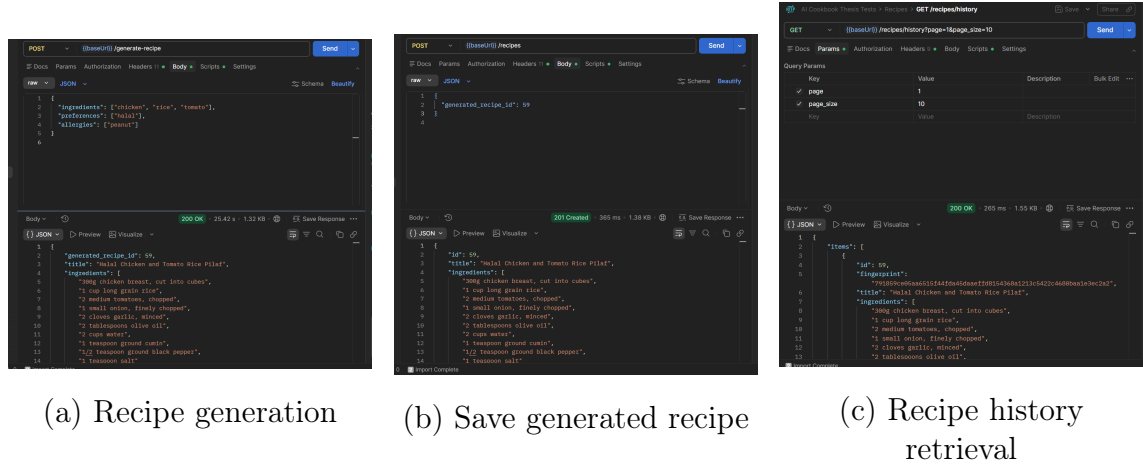
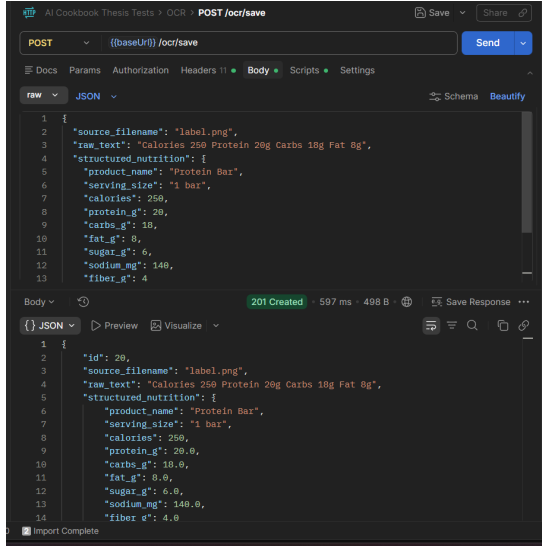


Figure 3.21: Postman evidence for recipe-generation and persistence integration

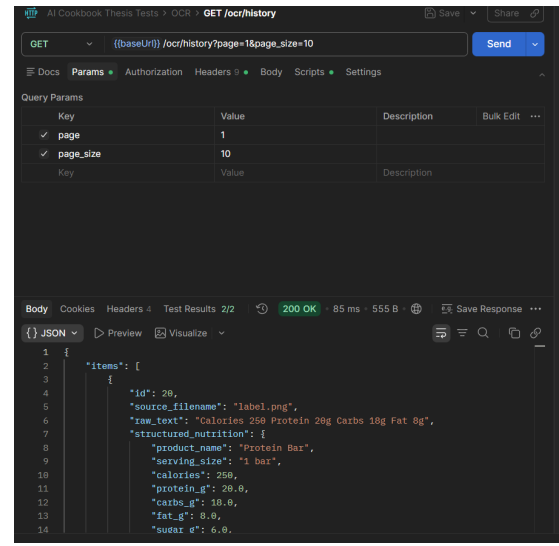
This sequence validates a full integration path:

1. authenticated user context is accepted,
2. the recipe-generation endpoint returns structured output,
3. a generated recipe identifier can be reused by the save endpoint,
4. the persistence layer stores the record correctly,
5. the history endpoint returns the stored data through paginated API access.

The OCR persistence path is shown in Figure 3.22. These requests confirm that structured OCR-related data can be saved and later retrieved through the user-scoped history endpoint.



(a) Save OCR extraction



(b) OCR history retrieval

Figure 3.22: Postman evidence for OCR persistence and retrieval

Finally, the retrieval subsystem itself is represented by the collection statistics endpoint shown in Figure 3.23. This does not test recipe quality directly, but it does verify that the RAG layer is implemented as a real subsystem with a persistent collection and explicit configuration parameters.

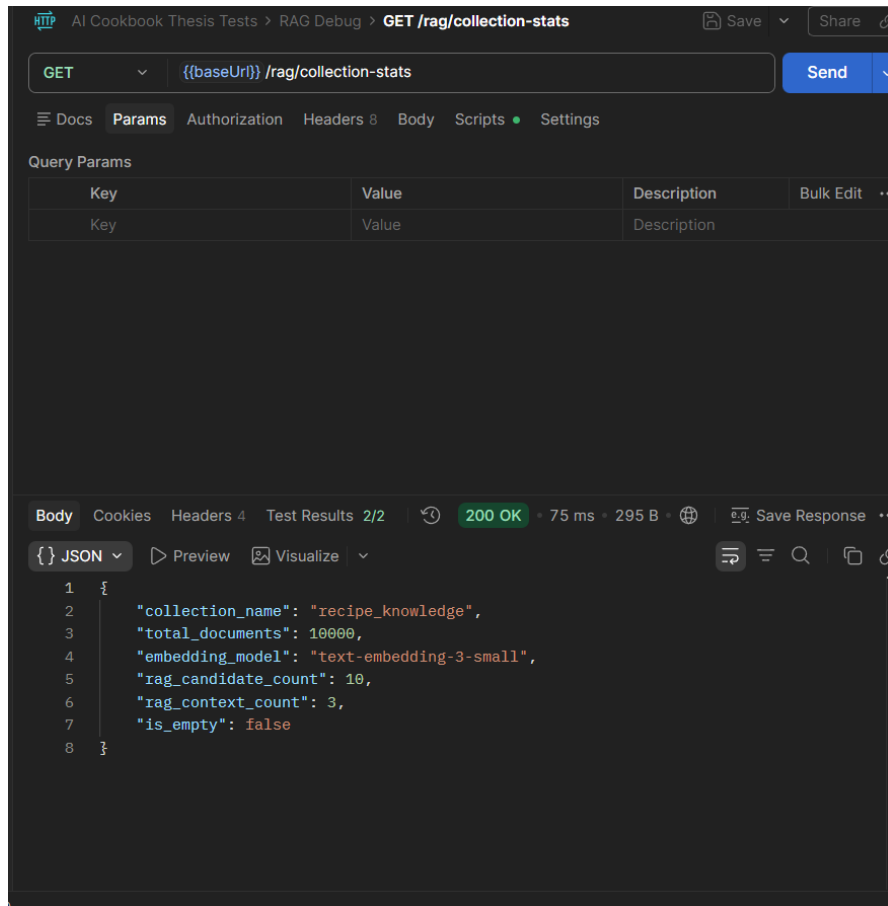


Figure 3.23: Postman evidence for the retrieval collection statistics endpoint

Taken together, the Postman evidence validates not just isolated routes, but real backend integration across authentication, persistence, and feature-specific workflow boundaries.

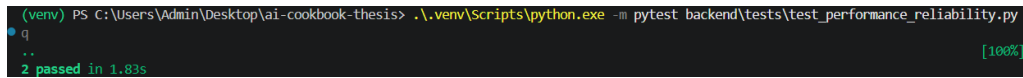
3.3.5 Advanced Reliability Testing

Reliability-oriented backend verification

In addition to correctness and integration, the backend was also verified from a reliability perspective. The implemented reliability tests focus on repeated requests and a small concurrent burst against the recipe-generation endpoint while external providers are replaced by deterministic mocks. This approach is appropriate because the project depends on networked AI services. If those calls were left real during reliability testing, the results would be dominated by provider latency and availability rather than the stability of the local application.

The executed reliability-oriented test result is shown in Figure 3.24. The successful run confirms that the backend can process multiple repeated mocked recipe-

generation requests consistently and can tolerate a small concurrent burst without failing the internal request path.

A terminal window showing a command prompt. The command is `pytest backend\tests\test_performance_reliability.py`. The output shows a progress bar at 100% and the result `2 passed in 1.83s`.

```
(venv) PS C:\Users\Admin\Desktop\ai-cookbook-thesis> .\venv\Scripts\python.exe -m pytest backend\tests\test_performance_reliability.py
..
2 passed in 1.83s [100%]
```

Figure 3.24: Successful reliability-oriented backend test execution

This reliability layer should be interpreted carefully. It does *not* prove real-world latency guarantees for OpenAI or OCR providers, and it does not replace production load testing. What it does show is that the internal orchestration of validation, request handling, and persistence remains stable when the nondeterministic external dependencies are controlled.

Role of security-oriented checks

Security-oriented behavior also appears in the automated and integration tests. The unauthorized protected-request screenshot in Figure 3.20 and the backend authentication tests jointly verify that protected resources are not exposed without a valid token. The backend rate-limiting tests also validate that repeated authentication attempts can be restricted at middleware level. For a project of this scope, this is a meaningful security-oriented testing layer because it addresses the most relevant risks in the implemented deployment model: unauthorized access to user-specific saved content and abusive repeated requests against sensitive endpoints.

3.3.6 Manual Scenario-Based Testing

Role of manual testing

Automated backend and frontend tests already confirm a large part of the deterministic behavior of the application. However, they do not fully replace scenario-based validation from the perspective of a human user. Manual testing remains necessary because the project contains multi-screen workflows that combine authentication state, page navigation, persistence, generated output display, and OCR result presentation. These aspects are easier to evaluate coherently when the system is used as an end user would use it.

For this reason, the evaluation includes a dedicated set of manual test cases covering the main user-facing flows of the application rather than only isolated

controls. The goal of this layer is to confirm that the implemented features can be completed successfully as end-to-end user tasks and to support the thesis with screenshot-based evidence.

Executed manual cases

The currently executed manual cases cover authentication, profile management, recipe generation, recipe persistence, history access, OCR extraction, OCR persistence, and access control. The strongest executed cases are summarized in Table 3.11. The table follows a compact thesis-oriented format similar to classical manual test documentation: each row identifies the test type, the validated behavior, the test steps, the expected result, and the observed result. The concrete evidence for these scenarios is distributed across the earlier figures in this chapter: Figures 3.15, 3.16, and 3.17 document the visible page-level results of login, account, generation, history, and OCR workflows from the perspective of the end user.

3. Developer Documentation

Type	Description	Test Steps	Expected Result	Observed Result
UI Scenario	Login page access	Open the frontend, navigate to the Login page, and observe the visible controls.	Login and sign-up tabs are visible, email and password fields are displayed, and the page loads correctly.	The login page loaded successfully and displayed both authentication modes and the required input fields without rendering failure.
Functionality	Successful user registration	Open the Login page, switch to Sign Up, enter a valid new email and password, and submit the form.	The account is created successfully, a success message is shown, and the page returns to login mode.	Registration succeeded with valid credentials and the interface returned to the normal authentication flow.
Functionality	Successful login and account view	Enter valid user credentials on the Login page, submit the form, and open the Account page.	The user is authenticated successfully and account/profile information is displayed.	The account page displayed the logged-in profile together with the saved preferences and allergies section.
Functionality Persistence	Profile default update	Open the Account page, enter dietary preferences and allergies, submit the update form, then refresh or revisit the page.	The profile update succeeds and the new values remain visible after reload.	Updated preference and allergy values remained visible after saving and page revisit, confirming persistence.
Functionality AI workflow	Recipe generation with direct input	Open the Generate page, enter a comma-separated ingredient list, and click Generate Recipe.	A structured recipe is returned with title, ingredients, steps, and nutrition estimate, and the page remains stable.	The page returned a complete recipe card and remained stable during response rendering.
Functionality Personalization	Recipe generation using saved defaults	Open the Generate page, enter only ingredients, leave preference and allergy fields empty, and generate the recipe.	The stored profile defaults are applied automatically and the generated output reflects those saved constraints.	The generated output reflected the stored personalization settings even though the optional form fields were left blank.
Integration Persistence	Save generated recipe	Generate a recipe while logged in, click Save Recipe, then navigate to the Saved page.	The save operation succeeds and the generated recipe appears in the saved recipes list.	The saved recipes page displayed the same recipe after the save action completed.
Integration Persistence	Recipe history availability	Generate one or more recipes, open the History page, and inspect the listed entries.	Generated recipes are listed in history, their state is shown, and stored details can be reviewed.	The history page listed the generated entries correctly and allowed their details to be reviewed.
Functionality OCR	OCR extraction with valid nutrition label	Open the Food Labels page, upload a valid nutrition-label image, and submit extraction.	The extraction completes successfully and the page displays both raw OCR text and structured nutrition information.	The OCR workflow returned both readable raw text and the expected nutrition field grid for the uploaded label.
Integration Persistence	Save OCR extraction	Perform a successful OCR extraction while logged in, save the result, and review the saved label history.	The extraction is stored successfully and appears in the saved OCR history section.	The stored OCR result was later visible in the user-scoped OCR history view.
Authentication Security	Protected pages require login	Log out, open the Saved page, then open the History page and observe both responses.	The pages instruct the user to log in and no private saved or historical data is exposed.	Both pages remained inaccessible without authentication and did not expose user-specific content.
UI Usability	Expand and collapse saved recipe details	Open the Saved page, expand one saved recipe, then collapse the detail section again.	Steps and nutrition become visible when expanded and are hidden again when collapsed.	The detail section opened and closed correctly while preserving the surrounding page state.

Table 3.11: Manual test cases executed for the AI Cookbook application

The executed manual results show that the application behaves correctly as a complete user-facing system. Registration and login work in combination with the account page. Stored preferences and allergies are reused during recipe generation. Generated recipes can be persisted and reviewed in the saved-recipes and history views. OCR extraction results can be produced, saved, and reviewed from history. Protected pages remain restricted when no authenticated user context is present.

Relationship to UI and integration testing

The manual testing layer complements the earlier UI and Postman sections. UI testing focuses on visible page states and layout-level evidence. Postman testing focuses on API contracts and HTTP-level integration. Manual testing documents that a user can complete the full workflow successfully in the running application.

3.3.7 Advanced AI-Related Evaluation

In addition to deterministic correctness and workflow validation, the project also includes evaluation of AI-assisted output quality. The repository therefore contains an evaluation pack for recipe generation and OCR-based nutrition extraction.

Recipe-generation evaluation methodology

The recipe-generation evaluation focuses on the combined behavior of the RAG retrieval layer, the prompt-driven language-model response, the restriction-handling logic, and the final structured output returned to the user. The prepared evaluation cases cover baseline success, preference satisfaction, allergy filtering, failure handling, and ingredient drift. Each output was assessed using the following criteria:

- **Structure validity:** whether title, ingredients, steps, and nutrition estimate were all present in usable form.
- **Allergy safety:** whether prohibited ingredients were successfully excluded or avoided.
- **Preference satisfaction:** whether the generated recipe respected the selected dietary rule.
- **Ingredient coverage:** how well the recipe used the provided ingredients as its main basis.

- **Additional ingredient count:** how many non-user ingredients had to be introduced beyond the basic pantry-like support.
- **Practical quality:** a manual score from 1 to 5 for overall usefulness and coherence.

Executed recipe-evaluation results

Based on the collected recipe outputs, the currently completed evaluation covers all twelve prepared recipe cases from the repository worksheet. Eleven of the twelve cases returned a structured recipe successfully, while one case — where all provided ingredients conflicted with the active vegan and milk-avoidance constraints — was correctly rejected by the system before unsafe generation could occur.

The aggregated results are summarized in Table 3.12.

Metric	Value	Interpretation
Total evaluated recipe cases	12	Full prepared recipe set was executed.
Successful responses	11 / 12	Only the fully conflicting input was rejected.
Structure validity rate	91.7%	Most cases returned complete structured recipe output.
Allergy safety rate	100%	Allergy-constrained cases stayed safe or were correctly rejected.
Preference satisfaction rate	100%	Preference-constrained cases respected the declared rule set.
Average ingredient coverage score	0.75	Most outputs used the provided ingredients well, though conflict-removal cases naturally reduced coverage.
Average additional ingredient count	0.33	Extra ingredients were generally limited.
Average practical quality score	4.4 / 5	The outputs were usually coherent and usable in practice.

Table 3.12: Aggregated metrics for recipe-generation AI evaluation

The structure validity rate shows that the model usually returns output in the required application format. The safety and preference metrics show that the combination of conflict filtering, restriction-aware prompting, and post-generation validation was effective in the executed cases. Lower ingredient-coverage scores were expected in cases where one of the user-provided ingredients conflicted with an allergy or dietary preference, because the system favored safety over maximal ingredient reuse.

Representative recipe-evaluation observations

The executed outputs also reveal several meaningful qualitative patterns.

- In baseline cases such as vegetarian pasta or simple salad input, the generated recipes remained coherent, simple, and strongly centered on the user ingredients.
- In allergy-constrained cases such as peanut, milk, and shellfish avoidance, the system issued explicit warnings and removed the conflicting ingredient while still attempting to preserve recipe usability.
- In preference-conflict cases such as vegetarian input containing chicken, the system again preferred rule compliance over preserving the conflicting ingredient.
- In some simple-input cases, the system suggested one or two extra ingredients, which improved practicality but slightly increased ingredient drift.
- In the fully conflicting case, the system correctly rejected the request instead of generating an unsafe or nonsensical output.

These observations show that the surrounding rule logic influences the final quality and safety of the generated result.

Executed OCR evaluation results

The OCR-specific evaluation was executed on three representative images: a clear nutrition label, a different supplement-style label layout, and a deliberately invalid non-nutrition document. This evaluation measures whether OCR text is returned and whether the downstream structuring stage produces correct nutrition fields or rejects unsuitable input.

Table 3.13 summarizes the per-case outcomes. For the two valid label images, the system extracted all core numeric nutrition fields correctly. The third case reveals the current failure mode: the OCR stage successfully read the document text, but the nutrition-structuring layer still returned a zero-filled nutrition object instead of rejecting the non-target image as unsupported input.

Case	Input type	OCR success	Structured output	Correct fields	Observation
O-01	Clear nutrition label (nutrition-facts.jpg)	Yes	Yes	9 / 9	Serving size and all numeric nutrition values were extracted correctly from the clean label image.
O-02	Alternate supplement-label layout (unnamed.jpg)	Yes	Yes	9 / 9	The different visual layout still produced correct serving-size and nutrition values across the core fields.
O-03	Non-nutrition certificate photo (WhatsApp Image 2026-04-14 at 13.56.17.jpeg)	Yes	No	0 / 9	OCR text was extracted from the document, but the nutrition layer should have rejected the image instead of returning zero-valued nutrition fields.

Table 3.13: Executed OCR evaluation cases and outcomes

The aggregate OCR metrics are summarized in Table 3.14. These values show that the OCR pipeline is strong on valid label images, but still needs a better front-end screening or classifier step for non-label uploads.

Metric	Value	Interpretation
Total evaluated OCR cases	3	Two valid label images and one invalid non-label document were executed.
OCR text extraction success rate	100%	Text was extracted from all three images, including the invalid document.
Structured-output success rate	66.7%	Only the two real nutrition-label images produced semantically correct structured nutrition output.
Calories exact-match rate	66.7%	Calories were correct for both valid labels and incorrect for the invalid document case.
Macro-field exact-match rate	66.7%	Protein, carbohydrate, fat, sugar, sodium, and fiber fields were correct on the valid labels and failed on the invalid document.
Average correctly extracted fields per image	6.0 / 9	The strong valid cases are offset by the complete failure of the non-target document case.
High-quality extraction share (≥ 5 correct fields)	66.7%	Two of the three evaluated images produced practically useful structured results.

Table 3.14: Aggregated metrics for OCR evaluation

These OCR results show that the implemented OCR plus structuring pipeline works well on clear nutrition labels of more than one visual layout. They also expose a current limitation: the system needs an explicit rejection rule or document-type classifier before the nutrition structuring stage, because raw OCR success alone is not enough to guarantee semantic correctness.

3.3.8 Threats to Validity

The current testing evidence has several limitations that should be stated explicitly.

- External AI providers can change behavior over time, so backend correctness does not imply identical future qualitative outputs.
- The reliability tests use mocked dependencies and therefore do not measure real-world latency of OpenAI or OCR services.
- Screenshot-based UI verification demonstrates usability and correct rendering, but it is not a substitute for exhaustive browser automation across all edge cases.
- The manual scenario evidence is strong for the main workflows, but it still prioritizes representative end-user tasks over exhaustive permutation of every negative and pagination edge case.
- The OCR evaluation currently uses only three executed images, so its metrics already reveal meaningful behavior, but they should still be interpreted as a small-sample evaluation rather than a production- scale benchmark.

These limitations define the interpretation boundary of the evidence presented in this chapter.

3.3.9 Section Summary

The testing evidence consists of multiple complementary layers. Automated backend tests verified authentication, persistence, validation, retrieval support logic, and OCR-related services. Automated frontend tests verified page behavior and conditional rendering. Screenshot-based UI verification documented the implemented user workflows. Postman-based integration checks verified API behavior, token-based authentication, persistence endpoints, recipe history, OCR history, and retrieval statistics across component boundaries. Reliability-oriented backend tests showed stable local request handling under repeated mocked requests and small bursts. Coverage measurement showed that the automated suites exercised a large part of the implemented backend and frontend logic, especially the security-sensitive

and user-visible layers. Manual scenario-based testing confirmed that the main end-user workflows can be completed in the running application. AI-specific evaluation showed that recipe generation performed well across the prepared worksheet and that OCR worked well on valid labels while still exposing a meaningful failure mode on non-nutrition input.

The chapter therefore documents both the functional correctness of the implemented system and the current quality boundary of its AI-assisted features.

Chapter 4

Conclusion

This thesis presented the design and implementation of an *AI Cookbook with Ingredient-Based Search*, a web application that combines personalized recipe generation, OCR-based nutrition-label processing, and persistent user-specific history. The project was motivated by two practical problems: deciding what to cook from available ingredients under dietary constraints, and converting nutrition-label information from food packaging into structured digital data without manual transcription.

The final system addresses these problems through a full-stack web application architecture. Users can submit ingredient lists and receive structured recipes grounded by retrieval-augmented generation, while food-label images can be processed through an OCR pipeline that returns both raw extracted text and structured nutritional fields. Authenticated users can additionally store dietary defaults, save generated recipes, and preserve OCR extraction history for later review.

4.1 Summary of achieved goals versus original proposal commitments

The project achieved the main objectives defined in the introduction and covered the core commitments of the original proposal.

- A working recipe-generation workflow was implemented with support for ingredients, dietary preferences, allergy constraints, retrieval context, warnings, and structured output.

- A user account system was implemented with registration, login, profile-default storage, recipe history, and saved-recipe support.
- An OCR-based nutrition-label workflow was implemented with image validation, preprocessing, OCR extraction, structured nutrition conversion, and optional persistence.
- The system was supported by a real persistence layer, migration history, vector retrieval subsystem, and media handling logic.
- A multi-layer testing strategy was prepared combining automated tests, manual workflow tests, and AI-specific evaluation materials.

Taken together, these results show that the final system is a working application with multiple integrated features rather than an isolated demonstration of a language-model call.

4.2 Lessons learned and challenges encountered

During the project, the main technical challenge was integrating several different concerns into one coherent application:

- retrieval-augmented generation for recipe personalization,
- rule-supported safety filtering for allergies and dietary preferences,
- OCR combined with language-model-based semantic structuring,
- persistence of both classical user data and AI-generated content,
- a modular architecture separating routes, services, models, schemas, and provider abstractions.

One of the main lessons learned was that AI-assisted features required supporting software-engineering structure around them. Validation, persistence, testing, and explicit limitation handling were necessary for turning recipe generation and OCR extraction into stable application features rather than isolated API calls.

Another lesson was that system boundaries had to remain explicit. External AI providers made output quality and availability partly dependent on services outside the local application. This affected both implementation and testing choices, especially for OCR extraction, recipe generation, and evaluation of output quality.

4.3 Current limitations

Despite the achieved results, the final system still has several important limitations.

- The quality of recipe generation, OCR text extraction, and structured nutrition output depends on external AI providers.
- The nutritional estimates attached to generated recipes are approximate and should not be interpreted as certified dietary analysis.
- Allergy and dietary handling is safety-oriented but not medically complete, since it is based on rule-based filtering and prompt-level constraints rather than formal nutritional reasoning.
- The default storage and media-serving model is appropriate for local thesis deployment, but it is not intended as a final production deployment architecture.
- The current vector retrieval corpus is intentionally limited in scope and therefore cannot represent every culinary style or dietary context.

These limitations define the boundary between what the system currently demonstrates and what would still be required for broader real-world deployment.

4.4 Proposed future work and development directions

The following improvements would be natural continuations of the project:

- Expanding the recipe corpus and improving retrieval quality with better metadata and reranking strategies.

- Introducing richer user modelling, such as cuisine preferences, cooking-time preferences, or meal-type preferences.
- Replacing local media storage and process-local rate limiting with more deployment-ready infrastructure.
- Improving OCR robustness through broader preprocessing options and more diverse evaluation datasets.
- Extending the nutritional model beyond a small fixed schema toward richer product comparison and dietary tracking features.

These directions would extend the system beyond the current thesis scope while building directly on the implemented architecture and testing results.

Bibliography

- [1] Python Software Foundation. *Python Setup and Usage*. Python Software Foundation. 2026. URL: <https://docs.python.org/3/using/index.html>.
- [2] Node.js Contributors. *Download Node.js*. Node.js. 2026. URL: <https://nodejs.org/en/learn/getting-started/how-to-install-nodejs>.
- [3] Sebastián Tiangolo. *Deployment - FastAPI*. FastAPI. 2026. URL: <https://fastapi.tiangolo.com/deployment/>.
- [4] Sebastián Tiangolo. *FastAPI Documentation*. FastAPI. 2026. URL: <https://fastapi.tiangolo.com/>.
- [5] SQLAlchemy Authors. *SQLAlchemy 2.0 Documentation*. SQLAlchemy. 2026. URL: <https://docs.sqlalchemy.org/20/>.
- [6] SQLAlchemy Authors. *Alembic Documentation*. Alembic. 2026. URL: <https://alembic.sqlalchemy.org/>.
- [7] Chroma Authors. *Chroma Documentation*. Chroma. 2026. URL: <https://docs.trychroma.com/>.
- [8] OpenAI. *OpenAI Platform Documentation*. OpenAI. 2026. URL: <https://platform.openai.com/docs/overview>.
- [9] Google Cloud. *Cloud Vision Documentation*. Google Cloud. 2026. URL: <https://cloud.google.com/vision/docs>.
- [10] Pillow Authors. *Pillow Documentation*. Pillow. 2026. URL: <https://pillow.readthedocs.io/>.
- [11] Samuel Colvin and contributors. *Pydantic Documentation*. Pydantic. 2026. URL: <https://docs.pydantic.dev/>.

- [12] Samuel Colvin and contributors. *Pydantic Settings Documentation*. Pydantic. 2026. URL: https://docs.pydantic.dev/latest/concepts/pydantic_settings/.
- [13] Encode OSS Ltd. *Uvicorn Documentation*. Uvicorn. 2026. URL: <https://www.uvicorn.org/>.
- [14] React Team. *React Documentation*. Meta. 2026. URL: <https://react.dev/>.
- [15] React Router Team. *React Router Documentation*. React Router. 2026. URL: <https://reactrouter.com/>.
- [16] Vite Team. *Vite Documentation*. Vite. 2026. URL: <https://vite.dev/guide/>.
- [17] Microsoft. *Visual Studio Code Documentation*. Microsoft. 2026. URL: <https://code.visualstudio.com/docs>.
- [18] Git Project. *Git Documentation*. Git. 2026. URL: <https://git-scm.com/doc>.
- [19] Node.js Contributors. *Node.js Documentation*. Node.js. 2026. URL: <https://nodejs.org/docs/latest/api/>.
- [20] npm, Inc. *npm Documentation*. npm. 2026. URL: <https://docs.npmjs.com/>.
- [21] Argon2-cffi Authors. *Argon2-cffi Documentation*. Argon2-cffi. 2026. URL: <https://argon2-cffi.readthedocs.io/>.
- [22] python-jose Contributors. *python-jose Documentation*. python-jose. 2026. URL: <https://python-jose.readthedocs.io/>.
- [23] Michael Jones, John Bradley, and Nat Sakimura. *RFC 7519: JSON Web Token (JWT)*. IETF. 2015. URL: <https://www.rfc-editor.org/rfc/rfc7519>.
- [24] OpenAI. *Embeddings FAQ*. OpenAI. 2026. URL: <https://help.openai.com/en/articles/6824809-embeddings-faq>.
- [25] OpenAI. *OpenAI API Reference*. OpenAI. 2026. URL: <https://platform.openai.com/docs/api-reference>.
- [26] Tim Bray. *RFC 8259: The JavaScript Object Notation (JSON) Data Interchange Format*. IETF. 2017. URL: <https://www.rfc-editor.org/rfc/rfc8259>.
- [27] OpenAI. *Image Generation Guide*. OpenAI. 2026. URL: <https://platform.openai.com/docs/guides/image-generation>.

- [28] Roy Fielding, Mark Nottingham, and Julian Reschke. *RFC 9110: HTTP Semantics*. IETF. 2022. URL: <https://www.rfc-editor.org/rfc/rfc9110>.
- [29] Shuyang Li and collaborators. *Food.com Recipes and Interactions*. Kaggle. 2019. URL: <https://www.kaggle.com/datasets/shuyangli94/food-com-recipes-and-user-interactions>.
- [30] SQLite Consortium. *SQLite Documentation*. SQLite. 2026. URL: <https://www.sqlite.org/docs.html>.
- [31] Encode OSS Ltd. *HTTPX Documentation*. HTTPX. 2026. URL: <https://www.python-httpx.org/>.
- [32] python-dotenv Authors. *python-dotenv Documentation*. python-dotenv. 2026. URL: <https://saurabh-kumar.com/python-dotenv/>.
- [33] Passlib Authors. *Passlib Documentation*. Passlib. 2026. URL: <https://passlib.readthedocs.io/>.
- [34] pytest Development Team. *pytest Documentation*. pytest. 2026. URL: <https://docs.pytest.org/>.
- [35] Ned Batchelder and contributors. *Coverage.py Documentation*. coverage.py. 2026. URL: <https://coverage.readthedocs.io/>.
- [36] Vitest Team. *Vitest Documentation*. Vitest. 2026. URL: <https://vitest.dev/guide/>.
- [37] Testing Library Authors. *React Testing Library Documentation*. Testing Library. 2026. URL: <https://testing-library.com/docs/react-testing-library/intro/>.
- [38] Testing Library Authors. *jest-dom Documentation*. Testing Library. 2026. URL: <https://testing-library.com/docs/ecosystem-jest-dom/>.
- [39] jsdom Contributors. *jsdom Documentation*. jsdom. 2026. URL: <https://github.com/jsdom/jsdom>.
- [40] Postman, Inc. *Postman Learning Center*. Postman. 2026. URL: <https://learning.postman.com/>.
- [41] Python Software Foundation. *Python 3 Documentation*. Python Software Foundation. 2026. URL: <https://docs.python.org/3/>.

- [42] Google. *Google Chrome Help*. Google. 2026. URL: <https://support.google.com/chrome/>.

List of Figures

2.1	Home page of the AI Cookbook application	19
2.2	Login and registration page	20
2.3	Account page after login, showing saved dietary defaults	21
2.4	Recipe-generation page with a generated result	22
2.5	Food label extraction page with OCR result	23
2.6	Saved recipes page	24
2.7	Recipe history page	25
3.1	Use case diagram of the main user-visible system functions	37
3.2	Component diagram of the system architecture	45
3.3	Class diagram of the domain and backend-service structure, showing implemented entity classes, provider abstractions, and their relation- ships	48
3.4	Class diagram of the infrastructure and schema layer, showing API contracts, configuration, middleware, and rate-limiting components .	48
3.5	Sequence diagram of the recipe-generation workflow, illustrating the call chain from user input through context retrieval, LLM invocation, and conditional database persistence	50
3.6	State-transition diagram of the recipe request lifecycle, showing the path from initial submission through input validation, ingredient fil- tering, retrieval-augmented generation, output verification, thumbnail generation, and persistence	51
3.7	Level 1 data flow diagram of the recipe-generation pipeline, showing processes, data stores, external entities, and field-level flows derived from the application codebase	52
3.8	Entity-relationship diagram of the AI Cookbook system database, showing all entities, attributes, primary keys, foreign keys, and car- dinality constraints	57

3.9	Relational schema derived from the ER model, showing all tables with primary keys, foreign keys, constraints, indexes, and inter-table relationships	59
3.10	Primary design-stage wireframes of the main entry and task-oriented pages	64
3.11	Secondary design-stage wireframes for persistence and account-related pages	65
3.12	Navigation and screen-transition map of the main frontend pages . .	66
3.13	Successful backend automated test execution with pytest	97
3.14	Successful frontend automated test execution with Vitest	98
3.15	Entry and authentication screens of the implemented frontend	101
3.16	Authenticated profile management and recipe-generation UI	101
3.17	Persistence and OCR-related UI screens	102
3.18	Postman collection used for API-level integration testing	103
3.19	Postman evidence for backend availability and authentication	104
3.20	Postman evidence for protected-route behavior	104
3.21	Postman evidence for recipe-generation and persistence integration . .	105
3.22	Postman evidence for OCR persistence and retrieval	106
3.23	Postman evidence for the retrieval collection statistics endpoint . . .	107
3.24	Successful reliability-oriented backend test execution	108

List of Tables

3.1	Actors and responsibilities in the AI Cookbook system	36
3.2	Measurable acceptance criteria for the functional requirements	39
3.3	Concrete design targets for the non-functional requirements	40
3.4	Layer responsibilities in the implemented system	46
3.5	Direct correspondence between the design artifacts and the imple- mentation units	49
3.6	Main API areas and their responsibilities	53
3.7	Recurring design mechanisms used throughout the system	54
3.8	Purpose of the main repository directories	70
3.9	Coverage indicators measured for the automated test suites	99
3.10	Representative executed test cases with steps, expected result, and observed result	100
3.11	Manual test cases executed for the AI Cookbook application	110
3.12	Aggregated metrics for recipe-generation AI evaluation	112
3.13	Executed OCR evaluation cases and outcomes	114
3.14	Aggregated metrics for OCR evaluation	114

List of Codes

2.1	Cloning the project repository	10
2.2	Creating the Python virtual environment	11
2.3	Activating the virtual environment in PowerShell	11
2.4	Activating the virtual environment on Linux or macOS	11
2.5	Installing backend dependencies	11
2.6	Installing frontend dependencies	12
2.7	Creating the local environment file	12
2.8	Creating the local environment file on Linux or macOS	12
2.9	Example local environment configuration	13
2.10	Verifying the local toolchain	14
2.11	Verifying key local files and directories	14
2.12	Applying database migrations	15
2.13	Indexing the recipe corpus for retrieval	15
2.14	Starting the FastAPI backend	16
2.15	Starting the React frontend	16
3.1	Repository structure of the implemented system	68
3.2	FastAPI bootstrap in <code>backend/main.py</code>	72
3.3	Password hashing utilities in <code>backend/security.py</code>	73
3.4	JWT-based current-user dependency	73
3.5	Profile update persistence in <code>backend/routes/auth.py</code>	73
3.6	Conflict analysis in <code>backend/services/llm_service.py</code>	74
3.7	Vector retrieval query in <code>backend/services/rag_service.py</code>	75
3.8	Prompt construction in <code>backend/services/llm_service.py</code>	75
3.9	Output validation and retry loop in <code>backend/services/llm_service.py</code>	76
3.10	OCR provider strategy in <code>backend/services/ocr_provider.py</code>	77

3.11	Image preprocessing for OCR	78
3.12	Nutrition-structuring prompt in <code>backend/services/ocr_service.py</code>	78
3.13	Thumbnail cache key derivation in <code>backend/services/recipe_image_service.py</code>	79
3.14	Thumbnail reuse and generation logic	80
3.15	Recipe image provider in <code>backend/services/recipe_image_provider.py</code>	80
3.16	Rate-limiting middleware structure	81
3.17	Authentication context in <code>frontend/src/context/AuthContext.jsx</code>	82
3.18	Profile-default fallback on the generation page	83
3.19	Saving a generated recipe from the frontend	84
3.20	Representative relational DDL for the implemented schema	85
3.21	<code>GeneratedRecipe</code> ORM mapping	87
3.22	Alembic revision adding image fields and pagination indexes	88
3.23	Saved-recipes query using the ownership and save-state fields	89
3.24	SQL form of the saved-recipes listing query	89
3.25	Analytical SQL over the implemented recipe-history schema	90
3.26	Offline recipe-corpus indexing for ChromaDB	91